

Hafta 3 – Veri Güvenliği: Aktarımda, Beklemede, Kullanımda

Tarih	02.10.2026
Öğrenme çıktıları	ÖÇ.2 (şifreleme yöntemleri ve güvenli iletişim ilkeleri) · ÖÇ.4 (güvenli iletişim kanalı kurma)
Süre	3 saat (3 × 50 dakika)
Ön bilgi	C'de işaretçi, dizi, dosya; Hafta 1 'den bellek düzeni ve güvenli silme; Linux/WSL terminalinde <code>cd</code> , derleme
Uygulamalar	code/week-03 – 9 demo; Windows'ta <code>.\demo.ps1</code> , WSL/Linux'ta <code>sh demo.sh</code>



Bu haftanın sonunda şunları yapabileceksiniz

1. Bir verinin üç durumunu — **aktarımda, beklemede, kullanımda** — ayırt etmek ve her biri için doğru koruma aracını seçmek.
2. **Simetrik/asimetrik şifreleme, özet (hash), MAC ve kimlik doğrulamalı şifrelemeyi (AEAD)** birbirinden ayırmak; AES-GCM ile bir dosyayı şifreleyip **tek bir bit değişince** çözmenin reddettiğini göstermek.
3. **IV/nonce ve tuzun** ne işe yaradığını ve **nonce yeniden kullanımının** neden felaket olduğunu; **ECB kipinin** neden desen sızdığını çalışan örnekte göstermek.
4. Paroladan anahtar türetmeyi (**PBKDF2, scrypt, Argon2id**) ve ana sırdan **oturma anahtarı türetmeyi (HKDF)** ile **ileri gizliliği** açıklamak.
5. **TLS 1.3 el sıkışmasını** çözmek; **sertifika ve ana makine adı doğrulamanın, sertifika sabitlemenin (pinning)** ve **TOFU'nun** neden gerektiğini; doğrulamasız bir istemciye karşı **localhost MITM** kurup sabitleme ile durdurmak.
6. **Beklemede** (dosya/veritabanı şifreleme, veri maskeleye) ve **kullanımda** (bellekte kısa ömür, güvenli silme, cihaz bağlama) veriyi korumak.
7. **Güvenlik kabuğu** kavramını — bir sırrın taşıma, depolama ve kullanım boyunca **iç içe katmanlarla** sarılması — kurmak ve "yalnız TLS'e güvenmeme" ilkesini savunmak.
8. **Whitebox kriptografinin** neden var olduğunu (motivasyon) bir cümleyle söylemek.
9. Güvenlikle ilgili rastgele değerleri **CSPRNG**'den üretmek; OpenSSL EVP ve Windows CNG ile AES-GCM'yi **adım adım** doğru çağırarak, etiketi doğrulamadan açık metne dokunmamak ve sabit zamanlı karşılaştırma yapmak.
10. Bir anahtarın **yaşam döngüsünü, hiyerarşisini, zarflamayı** ve anahtar sürümüyle yenilemeyi açıklamak; bir TLS istemcisini satır satır kurup **fail-open** hatalarını tanımak; **maskeleye, tokenizasyon ve takma adlandırmayı** doğru yerde kullanmak.

Ders akışı (öğretim üyesi için zaman planı)

Zaman	Bölüm	Ne yapılıyor
0:00–0:10	Giriş	Üç veri durumu, güvenlik kabuğu çerçevesi, laboratuvar kontrolü
0:10–0:30	2	Simetrik/asimetrik/özet/MAC/AEAD; Demo 1 (AES-GCM)
0:30–0:50	3–5	Rastgele sayılar, kript API'lerini adım adım kullanmak; IV/nonce/tuz, Demo 2–3
0:50–1:00	Ara	
1:00–1:25	6–8	PBKDF2/Argon2id Demo 4 ; HKDF ve ileri gizlilik Demo 5 ; anahtar yaşam döngüsü ve hiyerarşisi
1:25–1:50	9–10	TLS 1.3, doğrulama, sabitleme, Demo 6 ; TLS'i kodda kurmak, fail-open – sınıf etkinliği
1:50–2:00	Ara	
2:00–2:25	11–13	Beklemede veri Demo 7 ; veri maskeleyme; kullanımda veri Demo 8
2:25–2:45	14–15	Demo 9 (kabuk) ve sahadaki kabuk matrisi; whitebox motivasyonu
2:45–3:00	17–21	Proje (S5/S7), kendini sinama, kapanış

Laboratuvarı önceden hazırlayın (bu hafta kript var)

Demolar hem **Windows** (Visual Studio 2022 derleyicisi, ek kurulum yok) hem **WSL/Linux** (GCC + OpenSSL) üzerinde **aynı kaynaktan** derlenir. Kripto işlevleri ortak bir başlık üzerinden gelir: Linux'ta **OpenSSL**, Windows'ta işletim sisteminin kendi **BCrypt (CNG)** kütüphanesi. Bu yüzden Windows'ta öğrencinin OpenSSL kurmasına gerek yoktur.

```
cd code
.\build.ps1
cd week-03\01-aes-gcm-file
.\demo.ps1      # ya da demo.cmd dosyasına çift tıklayın

cd code
./build.sh
cd week-03/01-aes-gcm-file
sh demo.sh
```

Dosya > Aç > Klasör > code/ deyin; üstteki yapılandırmadan "**Windows (MSVC)**" ya da "**WSL (GCC)**" seçin; **Derle > Tümünü Derle**. Binary dosyalar her demonun bin\windows ya da bin/linux klasörüne düşer.

Kurulum ayrıntıları ve güvenlik kuralları: [code/README.md](#).

**Etik kural — bu derste her hafta geçerli**

Bu derste saldırıları **görev** öğreniyoruz. Bütün demolar yalnız kendi klasörlerinde dosya üretir, yönetici yetkisi istemez, sistem sertifika deposuna dokunmaz ve **ağı yalnız localhost (127.0.0.1)** üzerinde kullanır. TLS demosundaki "araya girme" (MITM) benzetimi tamamen kendi bilgisayarınızda, **kendi ürettiğimiz sertifikalarla** çalışır. Öğrendiğiniz teknikleri **yalnız kendi bilgisayarınızda ve verilen demolar üzerinde** deneyin. Başkasının trafiğini izinsiz dinlemek ya da araya girmek suçtur.

0. Başlamadan önce

Bu bölüm haftaya hazırlık içindir. Önce bu haftanın dayandığı önceki konuları kısaca hatırlatır; sonra bu haftanın kavramlarını birer cümleyle tanımlayıp her birini ayrıntılı anlatıldığı bölüme bağlar.

Önceki haftalardan gelenler

- **Beyaz kutu saldırgan modeli** — cihazın ya da sunucunun sahibi olan kullanıcı aynı zamanda potansiyel saldırgandır; belleği okuyabilir, hata ayıklayıcı bağlayabilir ([Hafta 1, §3](#)). Bu hafta bu modeli **kullanımda veri** (§13) ve **whitebox kriptografi** motivasyonu**nda (§15) yeniden kullanıyoruz.
- **Bellekten güvenli silme** — bir sırrı, derleyicinin kaldıramayacağı bir işlevle (`explicit_bzero`, `OPENSSL_cleanse`) üzerine yazarak silmek ([Hafta 1, §17](#)). Bu hafta §13'te buna döküm engelleme, bellek kilitleme ve cihaz bağlamayı ekliyoruz.
- **Entropi ölçümü** — bir dosyanın baytlarındaki düzensizliği ölçüp şifreli/paketli içeriği ayırt etme fikri ([Hafta 2, §6](#)). §3'te aynı fikri, bu kez rastgele üreticinin **girdisindeki** düzensizlik (entropi) açısından kullanıyoruz.

Bu haftanın kavram haritası

Kavram	Bir cümlede	Ayrıntısı
Verinin üç hâli	Bir sır yaşam döngüsü boyunca aktarımda (ağda), beklemede (diskte) ve kullanımda (bellekte) olmak üzere üç farklı durumda bulunur; her durumun tehdidi ve savunması farklıdır.	§1
Şifreleme	Şifreleme, açık metni bir anahtar kullanarak okunamaz şifreli metne dönüştürme işlemidir; çözme aynı (ya da eşleşen) anahtarla bunu tersine çevirir ve güvenlik anahtarın gizliliğine dayanır.	§2
AEAD	Kimlik doğrulamalı şifreleme, veriyi tek bir çağrıda hem şifreleyip hem de değişmediğini kanıtlayan bir etiket üreterek gizlilik ve bütünlüğü birlikte sağlar.	§2.1
Simetrik ve asimetrik şifreleme	Simetrik şifreleme iki tarafın paylaştığı tek bir gizli anahtarla hızlı çalışır; asimetrik şifreleme bir açık ve bir özel anahtar çiftiyle anahtar dağıtımını kolaylaştırır ama yavaştır; gerçek sistemler ikisini birlikte (hibrit) kullanır.	§2.2
Özet (hash)	Özet, veriden hesaplanan sabit boyutlu, tek yönlü bir parmak izidir; anahtarsız olduğu için tek başına bütünlük kanıtı sayılmaz.	§2.4
MAC ve imza	MAC paylaşılan bir gizli anahtarla mesajın değişmediğini kanıtlar; dijital imza açık/özel anahtar çiftiyle aynı şeyi inkâr edilemezlikle birlikte sağlar.	§2.4
Rastgelelik ve CSPRNG	CSPRNG, işletim sisteminin sunduğu kriptografik olarak güvenli rastgele sayı üreticidir; anahtar, IV, nonce ve tuz gibi güvenlikle ilgili her değer buradan üretilmeli, <code>rand()</code> gibi istatistiksel üreteçlerden gelmemelidir.	§3
IV / nonce / tuz	IV, nonce ve tuz; gizli olmayan ama tekrarsız ya da rastgele olması zorunlu başlangıç değerleridir; nonce'un aynı anahtarla tekrarı anahtar akışını sızdırır.	§5
Anahtar türetme (KDF)	KDF, bir paroladan (PBKDF2/scrypt/Argon2id ile, tuzlu ve yavaş) ya da bir ana sırdan (HKDF ile) yeni anahtarlar üreten fonksiyondur.	§6
İleri gizlilik	Her oturum için türetilen anahtarın kullanımdan sonra silinmesiyle, bugünkü anahtar sızsa bile geçmiş oturumların çözülmemesidir.	§7.1
Anahtar hiyerarşisi	Tek bir anahtar her işte kullanılmaz; bir ana anahtardan amaca özel türev anahtarlar (veri anahtarı, oturum anahtarı) üretilir ve her birinin kendi krypto-periyodu vardır.	§8
TLS	TLS, iki uç arasında anahtar anlaşmasını, AEAD ile şifrelemeyi ve sertifika doğrulamasını birleştirip ağ üzerinde güvenli iletişim sağlayan protokoldür (HTTPS'in altındaki katman).	§9

1. Verinin üç hâli ve güvenlik kabuğu

Geçen iki hafta "hata → saldırı → düzeltme" ile programın **kendisini** koruduk. Bu hafta odağı **veriye** çeviriyoruz. Bir sır — parola, ödeme anahtarı, kişisel bilgi — yaşam döngüsü boyunca **üç ayrı hâlde** bulunur ve her hâlin tehdidi de savunması da farklıdır:



Kısa tarihçe: veriyi korumanın araçları

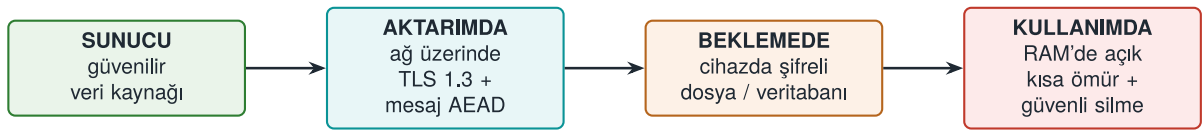
- **1976** — Diffie & Hellman **açık anahtarlı** kriptografiyi başlatır; **1977 RSA** ve **DES** standardı gelir.
- **1997–2001** — NIST'in açık **AES yarışması**; kazanan Belçikalı **Rijndael** algoritması, DES'in yerini alan **AES** standardı olur (2001). Bugün simetrik şifrelemenin omurgasıdır.
- **2004** — David McGrew ve John Viega, **GCM** (Galois/Counter Mode) kipini yayımlar; **2007**'de NIST **SP 800-38D** ile resmî standart olur: gizlilik **ve** bütünlüğü birlikte veren **AEAD** çağı başlar (bu haftanın ana aracı).
- **1995 → 2018** — Netscape'in **SSL 2.0**'ı (1995) → IETF'nin **TLS 1.0**'ı (1999, RFC 2246) → **TLS 1.3** (2018, RFC 8446): aktarımdaki verinin standardı.
- **2014** — **Heartbleed** (OpenSSL'de bir arabellek aşırı okuma açığı), TLS uygulamasındaki tek bir hatanın sunucu belleğindeki anahtarları ve sırları nasıl sızdırabileceğini bütün dünyaya gösterdi.

Dersin kuralı buradan çıkar: "kendi kriptonu yazma, **AEAD** kullan, anahtarı doğru yönet".

Hâl	Nerede?	Tehdit	Tipik savunma
Aktarımda (in transit)	Ağ üzerinde, iki uç arasında	Dinleme, araya girme (MITM), yeniden oynatma	TLS 1.3, sertifika doğrulama + sabitleme, mesaj düzeyi AEAD
Beklemede (at rest)	Disk, veritabanı, yedek	Dosyanın/DB'nin çalınması	Dosya/alan şifreleme (AEAD), cihaza bağlı anahtar, maskeleye
Kullanımda (in use)	RAM, önbellek, yazmaç	Bellek dökümü, hata ayıklayıcı, takas alanı	Kısa ömür, güvenli silme, bellek kilitleme, whitebox

Verinin üç hâli

sunucudan çıkıp kullanıcının GÜVENİLMEZ cihazına gider



Her hâlin tehdidi ve savunması farklıdır; üçü birden korunmazsa zincir kopar.



Bu haftanın ana fikri: güvenlik kabuğu

Tek bir önlem yetmez. Hassas bir varlığı, yaşam döngüsünün **her aşamasında** iç içe geçmiş **koruma katmanlarıyla** (kabuklarla) sararız: en dışta kanal güvenliği (TLS), onun içinde mesaj düzeyi şifreleme, onun içinde depolama şifrelemesi, en içte cihaza bağlı bir anahtar. Saldırganın sırda ulaşması için **hepsini sırayla** kırması gerekir. Bu, **Hafta 1**'de tanıttığımız **derinlemesine savunmanın** somut hâlidir; Bölüm 14'da bir anahtarı dört kabukla sarıp açacağız.

Dönem boyunca kullandığımız **örnek mimariyi** (bir mobil ödeme uygulaması + onun kritik işlerini yapan bir güvenlik kütüphanesi) hatırlayın. Bu hafta o mimarideki üç arayüzü dolduruyoruz: telefon ↔ sunucu (aktarımda), telefondaki şifreli

veritabanı (beklemede) ve ödeme anında bellekte açılan anahtar (kullanımda).

Bu mimaride telefon **güvenilmez ortamdır** (kullanıcı — yani potansiyel saldırgan — cihazın sahibidir; root alınmış, hata ayıklayıcı bağlanmış olabilir). Sunucu tarafı güvenilirdir. Aradaki her arayüz için şu soruları sorarız: hangi veri geçiyor, hangi hâlde (aktarım/depolama/kullanım), kimlik nasıl doğrulanıyor, gizlilik mi bütünlük mü yoksa ikisi birden mi gerekiyor? Bu haftanın demoları tam olarak bu soruların cevaplarını kurar:

- **Telefon ↔ sunucu (aktarımda):** TLS 1.3 + sertifika doğrulama + sabitleme, üstüne mesaj düzeyi AEAD (Demo 6, 1, 5).
- **Yerel veritabanı (beklemede):** hassas alanlar cihaza bağlı bir anahtarla AES-GCM (Demo 7, 4).
- **Ödeme anında bellek (kullanımda):** anahtar en kısa süre açık, kullanınca güvenli silme, cihaz bağlama (Demo 8, 9).

Bütün örnek değerler **sentetiktir** (uydurma); gerçek anahtar, kart numarası, cihaz kimliği ya da IP kullanılmaz.

2. Şifreleme temelleri: hangi araç neyi korur?

Şifreleme (encryption), okunur veriyi (**açık metin**) bir **anahtar** kullanarak okunamaz hale (**şifreli metin**) getirme işlemidir; **çözme (decryption)** aynı (ya da eşleşen) anahtarla bunu tersine çevirir. Güvenlik, algoritmanın değil **anahtarın** gizliliğine dayanır. Ama kriptografi tek bir şey değildir; **farklı hedefler için farklı araçlar** vardır. En sık karıştırılan dört aileyi netleştirelim.

Kriptografinin dört ailesi: hangi araç neyi korur?

tek bir "şifreleme" yoktur; hedefe göre araç seçilir

Simetrik şifreleme

GİZLİLİK

AES-GCM, ChaCha20

Özet (hash)

PARMAK İZİ

SHA-256 — anahtar yok

MAC

BÜTÜNLÜK + KİMLİK

HMAC — paylaşılan anahtar

Dijital imza

BÜTÜNLÜK + İNKÂR EDİLEMEZLİK

Ed25519 — özel anahtar

En sık hata: gizlilik ile bütünlüğü karıştırmak. Şifreleme tek başına bütünlük vermez.

Araç	Ne sağlar?	Anahtar	Örnek	Bu derste
Simetrik şifreleme	Gizlilik	Tek gizli anahtar (her iki taraf)	AES, ChaCha20	Dosya/DB/oturum şifreleme
Asimetrik şifreleme	Gizlilik + anahtar dağıtımı	Açık/özel çift	RSA-OAEP, ECIES	Anahtar kurulumu, TLS
Özet (hash)	Bütünlük parmak izi (anahtarsız)	Yok	SHA-256, SHA-3	Parmak izi, HKDF/HMAC yapıtaşı
MAC	Bütünlük + kimlik (paylaşılan anahtar)	Tek gizli anahtar	HMAC-SHA-256, CMAC	Mesajın değişmediğini kanıtlama
Dijital imza	Bütünlük + kimlik + inkâr edilemezlik	Açık/özel çift	Ed25519, RSA-PSS	Sertifikalar, sürüm imzalama (H10)
AEAD	Gizlilik ve bütünlük birlikte	Tek gizli anahtar	AES-GCM, ChaCha20-Poly1305	Bu haftanın ana aracı



En sık yapılan iki hata

- 1. Yalnız şifrelemek, bütünlüğü unutmak.** Sadece şifreli metin (ör. AES-CTR) saldırganın **fark edilmeden** bitleri değiştirmesini engellemez. Şifreleme gizliliği sağlar, **bütünlüğü değil**.
- 2. Kendi şifrenizi/kipinizi yazmak.** Kerckhoffs ilkesi: güvenlik algoritmanın gizliliğine değil **anahtarın** gizliliğine dayanmalı. Standart, denenmiş yapıları kullanın.

2.1 Kimlik doğrulamalı şifreleme (AEAD): tek çağrıda gizlilik + bütünlük

Doğru cevap neredeyse her zaman **AEAD**'dir (Authenticated Encryption with Associated Data). AEAD tek bir işlemle iki şey yapar: veriyi şifreler (**gizlilik**) ve şifreli metnin (isteğe bağlı olarak ek "ilişkili veri"nin) değişmediğini kanıtlayan bir **etiket (tag)** üretir (**bütünlük + kimlik**). Çözerken etiket **doğrulanır**: tek bir bit bile oynadıysa çözme **reddedilir**.

İki modern AEAD:

- AES-GCM** — donanım AES hızlandırması olan işlemcilerde (çoğu masaüstü/sunucu) çok hızlı. NIST SP 800-38D.
- ChaCha20-Poly1305** — AES donanımı olmayan cihazlarda (bazı mobil/gömülü) daha hızlı ve yan kanala dirençli. RFC 8439.



Kitap tarifi ve güncelleme

Ders kitabı (Viega & Messier) simetrik şifrelemeyi **tarif 5.x**'te işler ama 2003 tarihlidir: DES/3DES/RC4 ve ayrı MAC önerir. Biz bunları **AES-GCM / ChaCha20-Poly1305** (AEAD) ile güncelliyoruz. "Şifreleme ile bütünlüğü birlikte çalıştırma" fikri **tarif 6.18**'de geçer; AEAD bunun tek çağrıyla inmiş hâlidir.

2.2 Simetrik mi, asimetrik mi? Cevap: ikisi birden (hibrit)

Simetrik şifreleme (AES) hızlıdır ama iki tarafın **aynı gizli anahtarı** bilmesini gerektirir; bu anahtarı güvensiz bir ağdan nasıl paylaşırsınız? Asimetrik şifreleme (RSA, eliptik eğri) bu sorunu çözer: herkesin bir **açık** anahtarı (dağıtılır) ve bir **özel** anahtarı (gizli kalır) vardır. Ama asimetrik işlemler yavaştır ve büyük veriye uygun değildir.

Gerçek sistemler bu yüzden **hibrit** çalışır: asimetrik kriptoyla yalnız **küçük bir simetrik anahtarı** güvenli taşımak (ya da anlaşılmak) için kullanılır; asıl veri o simetrik anahtarla AEAD ile şifrelenir. TLS tam olarak budur: el sıkışmada eliptik eğri Diffie-Hellman ile bir oturum anahtarı üzerinde anlaşılır, sonra bütün trafik AES-GCM ya da ChaCha20-Poly1305 ile şifrelenir.

Özellik	Simetrik (AES)	Asimetrik (RSA/EC)
Hız	Çok hızlı	Yavaş (100–1000×)
Anahtar	Tek, paylaşılan gizli	Açık/özel çift
Kullanım	Büyük veri şifreleme	Anahtar taşıma, imza
Bu haftada	Dosya/DB/oturum	TLS anahtar kurulumu (H10)

2.3 Blok mü, akış mı? Kip (mode) neden önemli?

AES bir **blok şifresidir**: tek seferde 16 baytlık bir bloğu şifreler. Bloktan uzun veriyi şifrelemek için bir **kip** (mode of operation) gerekir ve **kip seçimi güvenliği belirler**:

- **ECB** — her bloğu bağımsız şifreler; **desen sızdırır** (Demo 3). Kullanmayın.
- **CBC** — her blok bir öncekine zincirlenir; rastgele IV ister, tek başına bütünlük **vermez**, padding oracle'a açık.
- **CTR** — bloğu bir akış şifresine çevirir (anahtar akışı $\oplus$ veri); nonce tekrarına **çok** hassas (Demo 2).
- **GCM** — CTR + Galois MAC; **AEAD**. Modern varsayılan.

ChaCha20 zaten bir **akış şifresidir** (blok kavramı yoktur); Poly1305 MAC'yle birleşince AEAD olur. Sonuç aynı: **gizlilik + bütünlük tek pakette**.

2.4 Özet, MAC ve imza: hangisi ne zaman?

Bu üçü **bütünlük** ailesindendir ama farklı şeyler kanıtlarlar; karıştırmak yaygın bir hatadır:

- **Özet (hash)**, veriden hesaplanan **sabit boyutlu, tek yönlü bir parmak izidir** ve anahtarsızdır. Yalnız verinin **değişip değişmediğini** gösterir. Ama saldırgan veriyi değiştirip **özeti de yeniden hesaplayabilir**; bu yüzden özet tek başına bütünlük **kanıtı değildir**. Özet, HMAC ve HKDF gibi yapıların yapıtaşdır ve parmak izi/karşılaştırma için kullanılır. Eski MD5/SHA-1 çakışmaya açıktır; SHA-256/SHA-3 kullanın.
- **MAC** (ör. HMAC-SHA-256) **paylaşılan gizli anahtar** ister. "Bu mesajı, anahtarı bilen biri gönderdi ve değişmedi" der. İki taraf da anahtarı bildiği için MAC **inkâr edilemezlik sağlamaz** (hangi taraf ürettiği ayırt edilemez). Sırları MAC ile karşılaştırırken **sabit zamanlı** karşılaştırma kullanın (`CRYPTO_memcmp`), düz `memcmp` zamanlama sızdırır.
- **Dijital imza** (ör. Ed25519, RSA-PSS) **açık/özel anahtar** çiftiyle çalışır. Yalnız özel anahtar sahibi imza üretebilir, herkes açık anahtarla doğrular; bu yüzden **inkâr edilemezlik** sağlar. Sertifikalar ve sürüm imzalama ([Hafta 10](#)) bununla çalışır.

Soru	Doğru araç
"Bu dosya bozuldu mu?" (anahtarsız parmak izi)	Özet (SHA-256)
"Bu mesajı anahtarı bilen biri mi yolladı ve değişmedi mi?"	MAC (HMAC)
"Bunu kesinlikle şu kişi/sunucu imzaladı, inkâr edemez"	İmza (Ed25519)
"Bunu hem gizle hem bütünlüğünü kanıtla"	AEAD (AES-GCM)

AEAD çoğu zaman doğru cevaptır

Uygulama içinde veri saklıyor ya da bir kanaldan geçiriyorsanız, ayrı ayrı "şifrele + MAC ekle" kurmak yerine **AEAD** kullanın; iki işi tek çağrıda, doğru sırada (encrypt-then-MAC) ve daha az hatayla yapar. Ayrı MAC yalnız şifrlenmeyecek ama bütünlüğü gereken veriler için (ya da AEAD'nin AAD alanıyla) gerekir.

Demo 1 — AES-256-GCM ile dosya şifreleme ve kurcalama tespiti

Demo 1 · `code/week-03/01-aes-gcm-file` · AEAD, NIST SP 800-38D

Program bir dosyayı AES-256-GCM ile şifreliyor; çıktı `[12 bayt nonce][şifreli metin][16 bayt etiket]` biçiminde. Çözerken etiket doğrulanıyor. Sonra şifreli metnin bir baytını, ardından yalnız etiketin bir baytını değiştirip çözmenin **reddedildiğini** görüyoruz.

Şifreleme ve çözmenin özü (kripto çağrıları ortak `cen429_crypto.h` başlığından gelir; Linux'ta OpenSSL EVP, Windows'ta BCrypt):

aes_gcm_file.c (özet)

```
/* Encrypt: out = [nonce][ciphertext][tag] */
crypto_random(nonce, 12); /* a fresh nonce per message */
memcpy(out, nonce, 12);
crypto_gcm_encrypt(key, nonce, 12, NULL, 0,
                  plain, plain_len, out + 12, tag);
memcpy(out + 12 + plain_len, tag, 16);

/* Decrypt: the tag is verified; ok == 0 if it does not match */
int ok = crypto_gcm_decrypt(key, nonce, 12, NULL, 0,
                           ct, ct_len, tag, plain);
if (!ok) { /* REJECT: the data was tampered with, or the key is wrong */ }
```

Çalıştırın (Windows'ta `.\demo.ps1`):

```
cd code/week-03/01-aes-gcm-file
sh demo.sh
```

Gerçek çıktı (Windows `.\demo.ps1`, kısaltılmış):

demo.ps1 — çıktı**STEP 2 - Encrypt**

Encrypted: output\secret.txt -> output\secret.enc (54 bytes of ciphertext + 12 nonce + 16 tag)

First 32 bytes of the ciphertext file (nonce + ciphertext):

```
df 05 81 06 c9 4a c6 75 da 63 ed ad f3 8a b9 1a 21 8c 81 aa 55 5e 3f 8e e7 27 9e e7 1c c8 b1 08
```

STEP 3 - Decrypt correctly (the tag matches)

Decrypted and VERIFIED: output\secret.enc -> output\decrypted.txt (54 bytes)

```
IBAN: TR00 0000 0000 0000 0000 0000 00
```

```
Balance: 12345
```

STEP 4 - Attack: change byte 20 of the ciphertext

VERIFICATION FAILED: the tag did not match, the data was tampered with, or the key is wrong.

Decryption REJECTED.

```
(exit code 2 - decryption rejected)
```

STEP 5 - Attack: change only the last byte of the TAG

VERIFICATION FAILED: the tag did not match, the data was tampered with, or the key is wrong.

Decryption REJECTED.

```
(exit code 2 - the tag did not match)
```

Satır satır ne oldu:

- **ADIM 2:** Rastgele bir 12 baytlık nonce üretildi ve dosyanın başına yazıldı. Nonce **gizli değildir** (dosyada açık durur); yalnız **benzersiz** olması gerekir.
- **ADIM 3:** Aynı anahtar ve nonce ile çözüldü, etiket tuttu, düz metin geri geldi.
- **ADIM 4:** Şifreli metnin 20. baytını bir XOR ile değiştirdik. GCM etiketi bütün şifreli metnin üzerinde hesaplandığı için **tek bayt bile** etiketi bozar; çözme reddedildi (çıkış kodu 2).
- **ADIM 5:** Bu kez veriye değil **yalnız etikete** dokunduk; yine reddedildi. Etiket, veriyle birlikte anahtarın da fonksiyonudur; saldırgan doğru etiketi anahtar olmadan üretemez.

✓ Kural

Bir şeyi şifreliyorsanız **AEAD kullanın** (AES-GCM ya da ChaCha20-Poly1305). Her mesaja **benzersiz nonce** verin. Etiket **her zaman doğrulayın** ve doğrulama başarısızsa **hiçbir çıktı üretmeyin** (fail-closed). Bütünlüğü ayrı bir adıma bırakmayın.

🔧 Sahada nasıl uygulanır?

Mobil ödeme kütüphanesinde yerel veritabanındaki her hassas alan ve sunucuya giden her mesaj AEAD ile korunur. "İlişkili veri" (AAD) alanına şifrelenmesi gerekmeyen ama **bağlanması** gereken bilgiler (sürüm no, kayıt kimliği, kart şeması etiketi) konur; böylece bir kaydın şifreli gövdesi başka bir bağlama **taşınamaz**.

? Değerlendirici nasıl test eder?

Bağımsız bir değerlendirici şifreli dosyada/DB'de bir baytı değiştirir ve uygulamanın **sessizce yanlış veri döndürmediğini, açıkça reddettiğini** doğrular. Ayrıca aynı düz metnin iki kez şifrelendiğinde **farklı** şifreli metin verdiğini (nonce'un tekrar etmediğini) kontrol eder.



Sık hatalar ve kontrol listesi

- ✓ Nonce her mesajda benzersiz mi? (Sabit nonce = felaket, bkz. Demo 2)
- ✓ Etiket saklanıyor ve çözerken doğrulanıyor mu?
- ✓ Doğrulama başarısızsa çıktı üretmeden mi çıkılıyor?
- ✓ Anahtar koda gömülü değil, güvenli bir kaynaktan mı geliyor?
- ✓ AES-CBC/CTR'yi tek başına (MAC'siz) kullanmıyorsunuz, değil mi?

Aşağıdaki animasyonda şifreleme, dosya biçimi `[nonce][şifreli metin][etiket]` ve etiket doğrulamasının adım adım nasıl işlediğini — hem başarılı çözme hem de kurcalanmış bir baytın nasıl reddedildiğini — izleyin.

plain[] = S e c r e t M e s s a g e 0 1

1. Düz metin, 15 bayt: "SecretMessage01".

plain[] = S e c r e t M e s s a g e 0 1

ct[] = C6 EA 74 20 05 4F BA F6 B1 A2 E2 6A 71 E3 04

2. `crypto_gcm_encrypt(...)` — şifreli metin hesaplandı (anahtar akışıyla XOR).

plain[] = S e c r e t M e s s a g e 0 1

ct[] = C6 EA 74 20 05 4F BA F6 B1 A2 E2 6A 71 E3 04

tag = D3147264 [nonce][şifreli metin][etiket]

3. `memcpy(out + NONCE_LEN + plain_len, tag, TAG_LEN)` — dosya `[nonce][şifreli metin][etiket]` olarak yazıldı.

plain[] = S e c r e t M e s s a g e 0 1

ct[] = C6 EA 74 20 05 4F BA F6 B1 A2 E2 6A 71 E3 04

tag = D3147264 [nonce][şifreli metin][etiket]
yeniden hesaplanan etiket = D3147264

4. Çöz: `crypto_gcm_decrypt(...)` yeniden hesaplanan etiket = saklanan etiket — DOĞRULANDI, düz metin geri verildi.

plain[] = S e c r e t M e s s a g e 0 1

ct[] = C6 EA 74 20 05 4F BA F6 B1 A2 E2 6A 71 E3 04

tag = D3147264 [nonce][şifreli metin][etiket]
yeniden hesaplanan etiket = D3147264

çözülen = S e c r e t M e s s a g e 0 1

5. Sonuç: "SecretMessage01" — orijinal düz metinle birebir aynı.

Örnek seçiciden **doğru çözme** (uyar/zor) ile **şifreli metin kurcalandı** ve **yalnız etiket kurcalandı** (uç durumlar) senaryolarını deneyin — ya da 🎲 ile rastgele bir mesaj üretin, ya da kendi metninizi yazın.

3. Rastgele sayılar: kriptografinin görünmez temeli

Şimdiye kadar konuştuğumuz her şey (anahtar, IV, nonce, tuz, oturum kimliği) tek bir varsayıma dayanır: bu değerler **tahmin edilemez**. Dünyanın en iyi şifreleme algoritması, anahtarı tahmin edilebilir bir üreteçle oluşturulmuşsa hiçbir şey korumaz. Kitabın 11. bölümü baştan sona bu konuya ayrılmıştır; burada bugün geçerli olan kısmını adım adım işleyeceğiz.

Üç tür “rastgele”: hangisi nerede kullanılır?

anahtar tahmin edilebilirse algoritmanın gücü anlamsızdır

Sözde rastgele (PRNG)

`rand()`, `Math.random()` — hızlı, tahmin edilebilir; ASLA güvenlikte

Kriptografik (CSPRNG)

`getrandom()`, `BCryptGenRandom` — anahtar, IV, nonce, tuz için

Gerçek rastgelelik (TRNG)

donanım gürültüsü — işletim sisteminin havuzunu besler

Kural: güvenlikle ilgili her değer CSPRNG'den gelir; kendi üreticinizi yazmayın.

Üç tür “rastgele”

Tür	Nasıl üretilir?	Tahmin edilebilir mi?	Nerede kullanılır?
İstatistiksel sözde rastgele	<code>rand()</code> , <code>std::mt19937</code> , Java <code>Random</code>	Evet: birkaç çıktıdan iç durum hesaplanabilir	Oyun, simülasyon, test verisi
Kriptografik sözde rastgele (CSPRNG)	İşletim sisteminin üretici: <code>getrandom</code> , <code>BCryptGenRandom</code> , <code>OpenSSL</code> <code>RAND_bytes</code>	Hayır (tohum gizli kaldıkça)	Anahtar, IV, nonce, tuz, belirteç
Gerçek rastgele (entropi kaynağı)	Donanım gürültüsü, zamanlama belirsizliği	Hayır	CSPRNG'yi tohumlamak için

Kural basittir: **güvenlikle ilgili her rastgele değer CSPRNG'den gelir**. `rand()` ve `mt19937` istatistiksel olarak “rastgele görünür”, ama güvenlik açısından tamamen tahmin edilebilirdir. Mersenne Twister'ın 624 ardışık çıktısını gören biri, sonraki bütün çıktıları hesaplayabilir.

Bu **tahmin edilemezlik**, [Hafta 2](#)'de gördüğümüz **entropi** fikrinin diğer yüzüdür: orada bir dosyanın baytlarındaki düzensizliği ölçüp şifreli/paketli içeriği ayırt ediyorduk; burada üreticin **girdisindeki** düzensizliğin (entropinin) yeterli olmasını isteyip **tahmin edilemez** çıktı üretmeye çalışıyoruz. Üreticin havuzu yeterince tohumlanmamışsa (aşağıdaki Hata 2'de göreceğimiz gibi) çıktı da tahmin edilebilir hale gelir.

Hata 1: zamanla tohumlamak

Hatalı: saniye cinsinden zaman = tahmin edilebilir tohum

```
srand(time(NULL));
unsigned char key[16];
for (int i = 0; i < 16; i++)
    key[i] = rand() & 0xFF;
```

Saldırgan anahtarın hangi gün üretildiğini biliyorsa, olası tohum sayısı bir günde yalnız 86.400'dür. Bir dizüstü bilgisayar bunların hepsini saniyeler içinde dener. `rand()` 'ın kendi iç durumu da küçüktür; tohum bilinirse bütün dizi bilinir.

Hata 2: entropiyi yanlışlıkla silmek

Debian OpenSSL felaketi (2008)

Bir dağıtım paketleyicisi, bellek denetim aracının verdiği bir uyarıyı susturmak için OpenSSL'in rastgele sayı havuzuna entropi ekleyen iki satırı kaldırdı. Sonuç: iki yıl boyunca o dağıtımda üretilen SSH ve SSL anahtarlarının tek entropi kaynağı **süreç kimliği** (en fazla 32.768 değer) oldu. Her anahtar türü ve boyu için olası anahtar sayısı birkaç on bine düştü; saldırganlar hepsini önceden üretip listeledi (CVE-2008-0166). Ders: kriptografik koda **anlamadan dokunmayın** ve rastgele üreticinin çıktısını istatistiksel olarak test etmek, tahmin edilemezliğini kanıtlamaz.

Doğru yol: işletim sisteminin üretici

```
#include <sys/random.h>
#include <errno.h>

int random_bytes(void *buffer, size_t n)
{
    unsigned char *p = buffer;
    while (n > 0) {
        ssize_t r = getrandom(p, n, 0); /* blocks until the pool is ready */
        if (r < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        p += r; n -= (size_t)r;
    }
    return 0;
}
```

`getrandom` (Linux 3.17+, glibc 2.25+), çekirdeğin CSPRNG'sinden okur ve açılışta havuz yeterince tohumlanmadıysa **bekler**. Eski yöntem olan `/dev/urandom` dosyasını açmak da güvenlidir, ama dosya tanıtıcıları tükenmiş ya da `chroot` içinde dosya yoksa başarısız olabilir. `/dev/random` 'ı kullanmak için bugün bir neden yoktur.

```
#include <windows.h>
#include <bcrypt.h>
#pragma comment(lib, "bcrypt.lib")

int random_bytes(void *buffer, size_t n)
{
    NTSTATUS s = BCryptGenRandom(NULL, (PUCHAR)buffer, (ULONG)n,
                                BCRYPT_USE_SYSTEM_PREFERRED_RNG);
    return BCRYPT_SUCCESS(s) ? 0 : -1;
}
```

`BCryptGenRandom`, kitabın Tarif 11.4'te anlattığı eski `CryptGenRandom` 'ın yerini almıştır. Algoritma tanıtıcısı yerine `NULL` ve `BCRYPT_USE_SYSTEM_PREFERRED_RNG` vermek en sade kullanımdır.

```
#include <openssl/rand.h>
if (RAND_bytes(buffer, (int)n) != 1) { /* error: NEVER continue */ }
```

⚠ Dönüş değerini denetleyin

Rastgele üreteç **başarısız olabilir**. Dönüş değeri denetlenmezse, tampon ilklendirilmemiş (ya da sıfır) kalır ve program "rastgele" sandığı sabit bir anahtarla çalışmaya devam eder. Hata durumunda tek doğru davranış, işlemi durdurmaktır.

Hata 3: aralığa indirirken sapma (modulo bias)

Bir zar atmak için `r % 6` yazmak masum görünür. Ama `r` 0–255 arasında tekdüze ise, 256 sayısı 6'ya tam bölünmediği için 0–3 değerleri 43'er kez, 4–5 değerleri 42'şer kez çıkar. Zar için önemsiz; ama rastgele bir parola ya da tek kullanımlık kod üretirken bazı karakterlerin daha sık çıkması tahmin edilebilirliği artırır. Doğru yöntem **reddetmedir** (Tarif 11.11):

[0, upper) aralığında sapmasız rastgele tamsayı

```
#include <stdint.h>

uint32_t random_in_range(uint32_t upper)
{
    uint32_t threshold = (uint32_t)(-upper) % upper; /* 2^32 mod upper: this many values are
"extra" */
    uint32_t r;
    do {
        random_bytes(&r, sizeof r);
    } while (r < threshold); /* discard the extra, draw again */
    return r % upper;
}
```

Fikir: 2^{32} değerini, `ust` 'e tam bölünen en büyük kısmını kullanıp kalanı atmak. Böylece her sonuç tam olarak aynı sayıda girdiden gelir.

Rastgele değerleri nerede kullanıyoruz? Bir kontrol listesi

Değer	Boyut	Gizli mi?	Benzersiz mi?	Not
Simetrik anahtar	16–32 bayt	Evet	Evet	CSPRNG'den ya da KDF'den
GCM nonce	12 bayt	Hayır	Evet (aynı anahtarla asla tekrar etmemeli)	Rastgele üretiliyorsa bir anahtarla en fazla $\sim 2^{32}$ mesaj
CBC IV	16 bayt	Hayır	Evet	Tahmin edilemez olmalı
Parola tuzu	≥ 16 bayt	Hayır	Evet	Kullanıcı başına
Oturum belirteci	≥ 16 bayt	Evet	Evet	Tahmin edilirse oturum çalınır
Tek kullanımlık kod (OTP)	6–8 hane	Evet	—	Sapmasız aralık üretimi + deneme sınırı

Rastgele nonce mu, sayaç mı?

GCM'de 12 baytlık nonce'u rastgele üretmek pratiktir, ama doğum günü olasılığı nedeniyle aynı anahtarla çok sayıda mesaj şifrelenecekse (NIST SP 800-38D'nin önerisi: rastgele nonce ile en fazla 2^{32} mesaj) ya anahtar sık yenilenir ya da nonce bir **sayaç** olarak üretilir. Sayaç kullanılıyorsa, sayacın cihaz yeniden başladığında sıfırlanmaması (kalıcı saklanması) gerekir. Bir sonraki bölümde göreceğimiz nonce tekrarı felaketi tam olarak bu noktada yaşanır.

Değerlendirici nasıl test eder?

Kaynak kodda `rand`, `srand`, `random`, `mt19937`, `java.util.Random` ve `Math.random` çağrılarını arar; güvenlikle ilgili bir değer bunlardan gelip gelmediğine bakar. Rastgele üreticinin dönüş değerinin denetlenip denetlenmediğini ve aralığa indirgemenin sapmasız yapıp yapılmadığını inceler. Mobil platformlarda rastgele üreticinin **kancalanabilir** olup olmadığını (ör. sabit değer döndürecek şekilde değiştirilebilir mi) sorar.

Sahada nasıl uygulanır?

Hassas bir kütüphanenin kılavuzunda rastgele sayıların **nerede kullanıldığı ve nerede kullanılmadığı** ayrıca yazılır: ödeme verisi için istemcide rastgele sayı üretilmez (değerler sunucudan gelir), Java tarafında platformun güvenli üretici kullanılır, native tarafta rastgelelik yalnız bellek doldurmada kullanılır. Böylece değerlendirici, rastgele üreticinin zayıflamasının hangi varlığı etkileyeceğini tek bakışta görür.

4. Kripto API'lerini doğru kullanmak: adım adım AES-GCM

Demo 1'de AES-256-GCM ile bir dosyayı şifreledik ve kurcalamayı yakaladık. Demo, işi `code/common/cen429_crypto.h` içindeki yardımcı fonksiyonlara bıraktı. Bu bölümde o fonksiyonların **içini** açıyoruz: OpenSSL'in EVP arayüzü ve Windows'un CNG (BCrypt) arayüzüyle AES-GCM'yi adım adım yazacağız. Amaç, bir kripto kütüphanesini kullanırken nerede hata yapılabileceğini görmektir. Kural hâlâ geçerlidir: **kendi algoritmanızı yazmazsınız**, ama hazır algoritmayı **doğru çağırmak** da bir beceridir.

AEAD'in dört girdisi ve iki çıktısı

AEAD (AES-256-GCM): girdiler ve çıktılar

tek işlemde gizlilik + bütünlük



Nonce aynı anahtarla İKİ KEZ kullanılırsa gizlilik ve bütünlük birlikte çöker.

AAD (ek doğrulanmış veri), şifrelenmeyen ama değiştirilirse fark edilmesi gereken bilgidir: dosya başlığı, sürüm numarası, kayıt kimliği. [1. haftadaki](#) parola kasası örneğinde türetme parametrelerini AAD yapmıştık; biri yineleme sayısını değiştirirse etiket tutmaz.

OpenSSL EVP ile şifreleme

`aes_gcm.c` — şifreleme (OpenSSL 1.1.1 / 3.x)

```
#include <openssl/evp.h>

/* Returns the ciphertext length on success, -1 on error. */
int aes_gcm_encrypt_example(const unsigned char key[32], const unsigned char nonce[12],
                           const unsigned char *aad, int aad_n,
                           const unsigned char *plain, int n,
                           unsigned char *ciphertext, unsigned char tag[16])
{
    EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
    int written, total = -1;
    if (ctx == NULL) return -1;

    if (EVP_EncryptInit_ex(ctx, EVP_aes_256_gcm(), NULL, NULL, NULL) != 1) goto done; /* 1 */
    if (EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_IVLEN, 12, NULL) != 1) goto done; /* 2 */
    if (EVP_EncryptInit_ex(ctx, NULL, NULL, key, nonce) != 1) goto done; /* 3 */
    if (aad_n > 0 && EVP_EncryptUpdate(ctx, NULL, &written, aad, aad_n) != 1) goto done; /* 4 */
    if (EVP_EncryptUpdate(ctx, ciphertext, &written, plain, n) != 1) goto done; /* 5 */
    total = written;
    if (EVP_EncryptFinal_ex(ctx, ciphertext + total, &written) != 1) { total = -1; goto done; } /* 6 */
    /*
    total += written;
    if (EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_GET_TAG, 16, tag) != 1) total = -1; /* 7 */
done:
    EVP_CIPHER_CTX_free(ctx); /* also clears the key material inside the context */
    return total;
}
```

Adımlar sırasıyla:

1. Algoritmayı seç; anahtarı ve nonce'u **henüz verme**.
2. Nonce uzunluğunu ayarla (GCM için önerilen 12 bayt; varsayılan da budur ama açık yazmak iyidir).
3. Anahtarı ve nonce'u ver.
4. AAD'yi ver: çıktı tamponu `NULL` 'dır, çünkü AAD şifrelenmez, yalnız etikete katılır.
5. Açık metni şifrele. Büyük veride bu adım parça parça tekrarlanabilir.
6. Bitir. GCM akış kipinde çalıştığı için dolgu yoktur; bu çağrı genellikle 0 bayt yazar, ama **çağırılması zorunludur**: etiket burada hesaplanır.
7. Etiket al ve şifreli metinle birlikte sakla.

Her çağrının dönüş değeri denetleniyor ve tek bir çıkış noktasında (`done:`) bağlam serbest bırakılıyor. C'de kaynak sızıntısını önlemenin en okunaklı yolu bu `goto` kalıbıdır.

Çözme: etiket doğrulanmadan açık metne dokunma

aes_gcm.c — çözme

```

/* Returns the plaintext length on success, -1 if the tag doesn't hold or on error. */
int aes_gcm_decrypt_example(const unsigned char key[32], const unsigned char nonce[12],
    const unsigned char *aad, int aad_n,
    const unsigned char *ciphertext, int n,
    const unsigned char tag[16], unsigned char *plain)
{
    EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
    int written, total = -1;
    if (ctx == NULL) return -1;

    if (EVP_DecryptInit_ex(ctx, EVP_aes_256_gcm(), NULL, NULL, NULL) != 1) goto done;
    if (EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_IVLEN, 12, NULL) != 1) goto done;
    if (EVP_DecryptInit_ex(ctx, NULL, NULL, key, nonce) != 1) goto done;
    if (aad_n > 0 && EVP_DecryptUpdate(ctx, NULL, &written, aad, aad_n) != 1) goto done;
    if (EVP_DecryptUpdate(ctx, plain, &written, ciphertext, n) != 1) goto done;
    total = written;
    /* Give the expected tag BEFORE Final */
    if (EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_TAG, 16, (void *)tag) != 1) { total = -1; goto
done; }
    if (EVP_DecryptFinal_ex(ctx, plain + total, &written) != 1) {
        OPENSSL_cleanse(plain, (size_t)n); /* tag did not hold: wipe the half-decrypted data */
        total = -1;
        goto done;
    }
    total += written;
done:
    EVP_CIPHER_CTX_free(ctx);
    return total;
}

```



En sık yapılan AEAD hatası

`EVP_DecryptUpdate` çağırısı açık metni **etiket doğrulanmadan önce** tampona yazar. Etiket `EVP_DecryptFinal_ex` içinde doğrulanır. Bir program Final'in dönüş değerine bakmadan tampondaki veriyi kullanırsa, kurcalanmış veriyi doğrulanmış sanar. Kural: **Final başarılı olmadan açık metin yok sayılır ve silinir.**

Windows CNG ile aynı iş

```
#include <windows.h>
#include <bcrypt.h>

int aes_gcm_encrypt_cng_example(const UCHAR key[32], const UCHAR nonce[12],
                                UCHAR *aad, ULONG aad_n, const UCHAR *plain, ULONG n,
                                UCHAR *ciphertext, UCHAR tag[16])
{
    BCryptAlg_HANDLE alg = NULL;
    BCryptKey_HANDLE keyHandle = NULL;
    BCryptAuthenticatedCipherMode_Info info;
    ULONG written = 0;
    int result = -1;

    if (!BCRYPT_SUCCESS(BCryptOpenAlgorithmProvider(&alg, BCRYPT_AES_ALGORITHM, NULL, 0))) goto done;
    if (!BCRYPT_SUCCESS(BCryptSetProperty(alg, BCRYPT_CHAINING_MODE,
        (PUCHAR)BCRYPT_CHAIN_MODE_GCM, sizeof(BCRYPT_CHAIN_MODE_GCM), 0))) goto done;
    if (!BCRYPT_SUCCESS(BCryptGenerateSymmetricKey(alg, &keyHandle, NULL, 0,
        (PUCHAR)key, 32, 0))) goto done;

    BCryptInitAuthModeInfo(info);
    info.pbNonce = (PUCHAR)nonce; info.cbNonce = 12;
    info.pbAuthData = aad; info.cbAuthData = aad_n;
    info.pbTag = tag; info.cbTag = 16;

    if (BCRYPT_SUCCESS(BCryptEncrypt(keyHandle, (PUCHAR)plain, n, &info, NULL, 0,
        ciphertext, n, &written, 0)))
        result = (int)written;
done:
    if (keyHandle) BCryptDestroyKey(keyHandle);
    if (alg) BCryptCloseAlgorithmProvider(alg, 0);
    return result;
}

/* Aynı hazırlık; info.pbTag beklenen etiketi gösterir. */
NTSTATUS s = BCryptDecrypt(keyHandle, (PUCHAR)ciphertext, n, &info, NULL, 0,
    plain, n, &written, 0);
if (s == STATUS_AUTH_TAG_MISMATCH) { /* 0xC000A002: kurcalama */
    SecureZeroMemory(plain, n);
    return -1;
}
```

İki arayüz farklı görünse de adımlar aynıdır: algoritma ve kip seçimi, anahtar nesnesi, nonce + AAD + etiket, tek çağrıda işlem, kaynakların serbest bırakılması. CNG'nin bir avantajı, etiketin uyuşmamasını ayrı bir durum koduyla bildirmesidir.

Sabit zamanlı karşılaştırma

Etiketi ya da bir MAC'i elle karşılaştırmamız gerekirse (ör. HMAC ile imzalanmış bir mesajda), `memcmp` kullanmayın. `memcmp` ilk farklı baytta durur; karşılaştırmamızın süresi, kaç baytın doğru olduğunu **sızdırır**. Saldırgan etiketi bayt bayt tahmin edip süreyi ölçerek doğru değere ulaşabilir (zamanlama saldırısı).

Sabit zamanlı karşılaştırma

```
int constant_time_equal(const unsigned char *a, const unsigned char *b, size_t n)
{
    unsigned char diff = 0;
    for (size_t i = 0; i < n; i++)
        diff |= a[i] ^ b[i];    /* always walk every byte */
    return diff == 0;
}
```

Hazır karşılıkları: OpenSSL `CRYPTO_memcmp`, Linux çekirdeğinde `crypto_memneq`, libsodium `sodium_memcmp`.

Kripto kütüphanesi kullanırken kontrol listesi

- ✓ Her çağrının dönüş değeri denetleniyor; hata durumunda işlem **durduruluyor**.
- ✓ Nonce her şifrelemede **yeni** (rastgele ya da kalıcı sayaç).
- ✓ Etiket, açık metin kullanılmadan **önce** doğrulanıyor; başarısızlıkta açık metin siliniyor.
- ✓ Etiket ve MAC karşılaştırmaları **sabit zamanlı**.
- ✓ Anahtar tamponları iş bitince siliniyor; bağlam nesneleri serbest bırakılıyor.
- ✓ Hata iletileri saldırgana ayrıntı vermiyor: "etiket hatası" ile "dolgu hatası" ayrı iletiler olursa, bu fark bir **kâhin** olarak kullanılabilir (CBC dolgu kâhini (padding oracle) saldırıları buradan doğmuştur).
- ✓ Eskimiş algoritmalar yok: DES/3DES, RC4, MD5, SHA-1 (imzada), ECB.



Sahada nasıl uygulanır?

Gerçek ürünlerin kılavuzlarında bir **algoritma envanteri** bulunur: kullanılan her algoritma (AES-CBC/CTR, HMAC, SHA-256, TLS, ödeme şemalarının MAC'i), hangi amaçla, hangi anahtar boyuyla ve hangi standarda dayanarak kullanıldığı tablo halinde yazılır. Bu envanter aynı zamanda **eleştirel bir okuma** için iyi bir malzemedir: birkaç yıl önce yazılmış envanterlerde SHA-1 ya da ayrı MAC'li CBC gibi bugün önerilmeyen seçimler görülür. Bugün yazılan bir envantere bunların yerini SHA-256 ve AEAD almalıdır.



Değerlendirici nasıl test eder?

Algoritma envanterini kaynak kodla karşılaştırır: envantere olmayan bir algoritma kullanılıyor mu? Nonce üretimini ve tekrarını inceler; şifreli veriyi bir bit değiştirerek geri verip uygulamanın bunu **reddettiğini** ve reddederken ayrıntılı hata vermediğini doğrular. MAC karşılaştırmasında `memcmp` arar.

5. IV/nonce ve tuz: aynı anahtarı tekrar kullanmak

Üç kavram sürekli karıştırılır. Üçü de **gizli değildir**, ama güvenlik doğru kullanımlarına bağlıdır (kitap **tarif 4.9**):

Tuz, IV ve nonce: üçü de gizli değil, üçü de farklı iş yapar

karıştırmak en sık yapılan kriptu hatasıdır

TUZ (salt)
paroladan anahtar
türetmede

kural: her kayıt için RASTGELE
bozulursa: önhesaplı tablolar işe yarar

IV
CBC gibi kiplerde

kural: öngörülemez olmalı
bozulursa: aynı düz metin aynı görünür

NONCE
GCM gibi AEAD kiplerinde

kural: anahtar başına BİR KEZ
bozulursa: gizlilik VE bütünlük çöker

Üçü de açıkça saklanabilir; güvenlik gizliliklerinden değil, TEKRARLANMAMALARINDAN gelir.

Kavram	Nerede?	Kural	Bozulursa
Tuz (salt)	Paroladan anahtar türetmede	Her parola/kayıt için rastgele	Önhesaplı tablolar (rainbow) işe yarar
IV (başlangıç vektörü)	CBC gibi kiplerde	Rastgele , öngörülemez	İlk blok sızar (BEAST türü)
Nonce (bir kez kullanılan sayı)	CTR/GCM gibi kiplerde	Aynı anahtarla asla tekrar etmesin	Anahtar akışı sızar (aşağıda)

5.1 Nonce yeniden kullanımı neden felakettir?

GCM ve CTR, anahtardan ve nonce'dan bir **anahtar akışı** üretilip düz metinle XOR'lar: $C = P \oplus AA(\text{anahtar}, \text{nonce})$. Aynı anahtar **ve** aynı nonce ile iki mesaj şifrelenirse iki mesajda da **aynı** anahtar akışı kullanılır:

$$\begin{aligned} C1 &= P1 \oplus AA & C2 &= P2 \oplus AA \\ C1 \oplus C2 &= (P1 \oplus AA) \oplus (P2 \oplus AA) = P1 \oplus P2 \end{aligned}$$

Anahtar akışı yok olur; saldırgan iki **düz metnin XOR'unu** ele geçirir. Bir mesajı biliyorsa (ya da tahmin ederse) diğerini tamamen çözer.

Demo 2 — Nonce tekrarının iki düz metnin XOR'unu sızdırması

Demo 2 · `code/week-03/02-nonce-reuse` · nonce yeniden kullanımı

Program iki farklı mesajı önce **aynı** nonce ile, sonra **farklı** nonce ile AES-256-GCM ile şifreliyor ve $C1 \oplus C2$ ile $P1 \oplus P2$ 'yi karşılaştırıyor.

nonce_reuse.exe — gerçek çıktı (hex kısaltıldı)

```

Plaintext 1: "Attack begins at dawn, 06:00 hours!"
Plaintext 2: "Total balance is 45000, PIN is 1234"
=====
BAD CASE - the SAME nonce for both messages:
  C1 xor C2 = 151b00000f4b42030906070d16000807 ... 44404015
  P1 xor P2 = 151b00000f4b42030906070d16000807 ... 44404015
==> C1 xor C2 == P1 xor P2 : the keystream LEAKED!
  If the attacker knows P1, P2 = (C1 xor C2) xor P1 =
    "Total balance is 45000, PIN is 1234"
=====
GOOD CASE - a DIFFERENT nonce for each message:
  C1 xor C2 = 330093b563c381a72339b0180bd1dc2f ... d97fab3
  P1 xor P2 = 151b00000f4b42030906070d16000807 ... 44404015
==> C1 xor C2 != P1 xor P2 : no leak.

```

Kötü durumda $C1 \oplus C2$ ile $P1 \oplus P2$ **birebir aynı** çıktı: anahtar akışı iptal oldu. Program bir mesajı bilen saldırganın diğerini $(C1 \oplus C2) \oplus P1$ ile tamamen çözdüğünü de gösterdi. Farklı nonce'ta ilişki kayboldu.

” Gerçek olaylar

Nonce/IV tekrarı efsanevi hatalardır: WEP'in kısa IV'i (kablosuz ağ şifrelemesinin çöküşü), Sony PlayStation 3'ün ECDSA imzasında **aynı rastgele değeri** iki kez kullanması (özel anahtarın çıkarılması) ve çeşitli TLS kütüphanelerinde GCM nonce tekrarı uyarıları. Kural nettir: **nonce asla tekrar etmesin**.

✓ Kural

Nonce'u ya **rastgele** (12 bayt, çakışma olasılığı ihmal edilebilir) ya da **sayaç** olarak üretin ve **kalıcı olarak** takip edin. Aynı anahtarla aynı nonce'u iki kez asla kullanmayın. Çok fazla mesaj şifreliyorsanız (2^{32} üstü) anahtarı yenileyin ya da XChaCha20 gibi geniş nonce'lu bir yapı seçin.

🔧 Sahada nasıl uygulanır? / Değerlendirici nasıl test eder?

Uzun ömürlü bir anahtarla milyonlarca mesaj şifreleyen sistemlerde (ör. sunucu-istemci oturumu) nonce yönetimi bir tasarım kararıdır: her mesaja artan sayaç + oturum başına yeni anahtar. Değerlendirici, kodda nonce'un nasıl üretildiğine bakar (sabit mi, sayaç kalıcı mı, rastgele mi) ve iki şifreli çıktının nonce alanlarının **asla aynı olmadığını** doğrular. Sabit nonce ya da yeniden başlatınca sıfırlanan sayaç doğrudan bulgudur (CWE-323).

⚠ Sık hatalar ve kontrol listesi

- ✓ Nonce kalıcı mı takip ediliyor (program yeniden başlayınca sıfırlanmıyor)?
- ✓ Rastgele nonce kullanıyorsanız 96 bit yeterli mesaj sayısı için mi?
- ✓ Aynı (anahtar, nonce) çifti iki mesajda kullanılamaz, doğru mu?

Aşağıdaki animasyonda aynı nonce'un iki mesajı nasıl aynı anahtar akışıyla şifrelediğini, $C1 \text{ xor } C2$ 'nin $P1 \text{ xor } P2$ 'ye nasıl eşit çıktığını ve saldırganın bilinen bir düz metinle diğerini nasıl kurtardığını izleyin.



1. İki düz metin, eşit uzunlukta (14 bayt): P1 ve P2.



2. Kötü durum: her iki mesaj da AYNI nonce ile şifrelenecek.



3. `crypto_gcm_encrypt(key, nonce_same, ...)` — C1 hesaplandı.



4. `crypto_gcm_encrypt(key, nonce_same, ...)` — C2 hesaplandı.



5. `xor_bytes(c1, c2, cxor, len)` — şifreli metinlerin XOR'u hesaplandı.



6. $C1 \text{ xor } C2 == P1 \text{ xor } P2$ — anahtar akışı birbirini GÖTÜRDÜ: bu, düz metinlerin XOR'unu SIZDIRIR!



7. Saldırgan P1'i biliyorsa: $P2 = (C1 \text{ xor } C2) \text{ xor } P1$ — P2'yi HİÇ anahtar olmadan tamamen kurtarır.

Örnek seçiciden **aynı nonce ile sızıntı** (uyar/zor) ile **farklı nonce, sızıntı yok** ve **iki mesaj da aynı** (uç durumlar) senaryolarını deneyin — ya da kendi iki eş uzunluklu mesajınızı yazın.

5.2 ECB deseni sızdırır

Nonce/IV kullanmayan bir kip olan **ECB** (Electronic Codebook), her 16 baytlık bloğu **bağımsız** şifreler. Aynı düz metin bloğu **her zaman** aynı şifreli bloğu verir; bu yüzden verideki tekrar eden desenler şifreli metinde de görünür.

Demo 3 — ECB "pengueni": desen sızıntısı

Demo 3 · `code/week-03/03-ecb-pattern` · ECB desen sızıntısı

İki renkli bir "resim" kuruyoruz (her piksel tam bir 16 baytlık blok). AES-128-**ECB** ile şifrelenince şekil hâlâ okunur kalır; AES-256-**GCM** (CTR tabanlı) ile şifrelenince desen kaybolur.

`ecb_pattern.exe` — gerçek çıktı

```

Original picture (plaintext):
#####
# ##      ###      ##      #
... (429 rakamları) ...
#####
=====
Encrypted with AES-128-ECB (each block by its first byte):
=====
==#==####==####==####==####=
... aynı şekil hala GORUNUYOR ...
=====
==> The shape is still VISIBLE: ECB leaks the pattern. DO NOT USE IT.
=====
Encrypted with AES-256-GCM (CTR-based), same picture:
: -.-+-%*-:*%* . ###+*.:+* :..
*.*###:*%+####:#-*=*#-+=%:.*%*
... desen kayboldu, gürültü ...
==> The pattern is gone: GCM/CTR encrypts every block differently.

```

ECB çıktısında yalnız **iki** karakter var (arka plan bloğu → bir şifreli blok, şekil bloğu → başka bir şifreli blok); şekil aynen okunuyor. Bu, ünlü "ECB penguini" örneğinin ASCII hâlidir. GCM'de her blok konuma bağlı farklı bir anahtar akışıyla şifrelendiği için çıktı gürültüdür.

✓ Kural

ECB kullanmayın. Şifreleme için AEAD (GCM, ChaCha20-Poly1305); disk için XTS-AES; asla desen sızdıran ECB değil. "Her blok bağımsız şifrelensin" cazip görünür ama gizliliği bozar.

🔧 Sahada nasıl uygulanır? / Değerlendirici testi

Kod incelemesinde `ECB` geçen her yer bir bulgudur (kitap **tarif 5.x** eski `EVP_*_ecb` çağrılarını içerir; güncellenmelidir). Değerlendirici, aynı düz metnin şifreli metninde tekrar eden 16 baytlık bloklar arayarak ECB'yi otomatik yakalayabilir.

⚠ Sık hatalar ve kontrol listesi

- ✓ Kodda `ecb` geçen hiçbir şifreleme kipi yok, değil mi?
- ✓ Görüntü/yapılandırılmış veri şifrelerken desen sızıntısını düşündünüz mü?
- ✓ Şifreli çıktı, aynı girdi için her seferinde farklı mı (nonce/IV sayesinde)?

Aşağıdaki animasyonda küçük bir resmi hem ECB hem GCM ile şifreleyip desenin ECB'de nasıl **okunur kaldığını**, GCM'de nasıl **kaybolduğunu** izleyin.

Düz metin

1. Küçük iki renkli bir "resim": 24 blok, yalnız 2 farklı değer (arka plan=0x00, şekil=0xFF).

Düz metin **AES-128-ECB**

2. `crypto_aes_ecb_block(key, ..., plain + i*BLOCK, ecb + i*BLOCK)` — ECB çıktısında yalnız 2 FARKLI bayt değeri var: şekil HÂLÂ OKUNUYOR.

Düz metin **AES-128-ECB** **AES-256-GCM (CTR)**

3. `crypto_gcm_encrypt(key32, nonce, ..., plain, ..., good, tag)` — çıktıda 24 FARKLI değer var (24 bloğun 24'i benzersiz): desen KAYBOLDU.

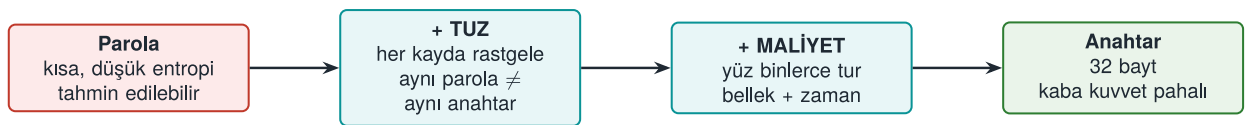
Örnek seçiciden **küçük bir şekil** (uyar/zor) ile **tamamı arka plan** ve **dama tahtası** (uç durumlar) desenlerini deneyin — ya da kendi 0/1 desenli küçük resminizi yazın.

6. Paroladan anahtar türetme

Parola doğrudan anahtar olamaz: kısa, düşük entropili ve tahmin edilebilir. Paroladan anahtar türetme fonksiyonu (KDF) iki iş yapar:

Paroladan anahtar türetme: iki ek, iki fayda

Argon2id / scrypt / PBKDF2



Tuz önhesaplı tabloyu, maliyet ise deneme hızını öldürür. Biri eksikse KDF işe yaramaz.

- Tuz (salt):** Her kullanıcıya/kayda rastgele tuz eklenir. Aynı parola farklı tuzlarla farklı anahtar verir; böylece **önhesaplı tablolar (rainbow table)** işe yaramaz ve iki kullanıcının aynı parolası fark edilmez.
- Yavaşlık (maliyet):** KDF bilerek yüzbinlerce tur çalışır; bir parolayı denemek pahalı olur, kaba kuvvet yavaşlar.

KDF	Maliyet eksen	Ne zaman?	Standart
PBKDF2	Yalnız CPU (tur sayısı)	Uyumluluk/FIPS gerektiğinde	NIST SP 800-132
scrypt	CPU + bellek	Bellek-zor isteniyorsa	RFC 7914
Argon2id	CPU + bellek + paralellik	Yeni sistemlerde birinci tercih	RFC 9106, OWASP



Kısa tarihçe: parola KDF'leri

- **1999** — Niels Provos ve David Mazières, Bcrypt algoritmasını (Blowfish'in anahtar çizelgelemesine dayanan, ayarlanabilir maliyetli bir parola özetleme fonksiyonu) yayımlar.
- **2000** — RSA Laboratories'in **PKCS #5 v2.0** standardı (RFC 2898), **PBKDF2**'yi tanımlar.
- **2009** — Colin Percival, **scrypt**'i tanımlar; amaç özel donanımla (ASIC/FPGA) kaba kuvveti bellek gereksinimiyle pahalılaştırmaktır (RFC 7914).
- **2013–2015** — Password Hashing Competition (PHC); **2015**'te Alex Biryukov, Daniel Dinu ve Dmitry Khovratovich'in **Argon2**'si kazanır (Argon2id, RFC 9106 ile standartlaşır).



Kitap tarifi ve güncelleme

Kitap paroladan anahtar türetmeyi **tarif 4.10**'da PBKDF2 ile, 10.000 tur önererek anlatır. Bugün bu **çok düşüktür**: OWASP (2023) PBKDF2-HMAC-SHA256 için **≥ 600.000 tur** ister; mümkünse **Argon2id** tercih edilir.

Demo 4 — Tur sayısı, süre ve tuzun önemi



Demo 4 · `code/week-03/04-password-to-key` · PBKDF2, tuz, maliyet

Program aynı paroladan PBKDF2 ile anahtar türetir: aynı tuz → aynı anahtar (tabloya açıklık), farklı tuz → farklı anahtar; ve tur sayısı arttıkça geçen süreyi ölçer. WSL'de `argon2` aracı varsa Argon2id karşılaştırması da yapılır.

`password_to_key.exe` — gerçek çıktı (Windows)

```

Password: "dog123"
=====
1) WHY THE SALT MATTERS (rounds = 100000)
  salt A -> key = 8b3d1913b0a104fa643f2b7b36863c65d1deda204e9f6bf6c1b21443930ec892
  salt A -> key = 8b3d1913b0a104fa643f2b7b36863c65d1deda204e9f6bf6c1b21443930ec892
  ^ Same password + the SAME salt -> the SAME key (a table lookup would work)
  salt B -> key = e9a8193a675ec5dd2366aad2a54d253c655f2c914043858b5a4c8d8b53691290
  ^ Same password + a DIFFERENT salt -> a DIFFERENT key (the salt does its job)
=====
2) ROUND COUNT vs TIME (same password, same salt)
  rounds = 1000 -> 1.24 ms
  rounds = 10000 -> 12.55 ms
  rounds = 100000 -> 126.75 ms
  rounds = 600000 -> 824.96 ms
  rounds = 2000000 -> 2808.32 ms
  ^ As the round count grows, every password guess gets more expensive.

```

Argon2id karşılaştırması yalnız WSL/Linux'ta, `argon2` aracı kuruluysa çalışır (`demo.sh`'in 3. adımı); Windows demosu (`.\demo.ps1`) yalnız açıklamayı yazdırır (bkz. aşağıdaki not).

- **Tuz:** Aynı parola + aynı tuz her seferinde **aynı** anahtar verdi (bu yüzden tuz sabitse saldırgan bir kez tablo kurup herkesi arar). Aynı parola + **farklı** tuz tamamen farklı anahtar verdi.
- **Maliyet:** Tur sayısı 1.000'den 2.000.000'a çıkınca süre ~1,2 ms'den ~2,8 saniyeye çıktı. Bu doğrudan çarpandır: saldırganın her parola denemesi de o kadar pahalılaşır. "Kullanıcı girişinde 200 ms" kabul edilebilir; saldırgan için saniyede milyarlarca deneme yerine binlerce deneme demektir.
- **Argon2id:** Ek olarak **bellek** (burada 64 MiB) ister; GPU/ASIC ile paralel kaba kuvveti PBKDF2'den daha çok zorlaştırır.



Windows notu

Windows demosu (`.\demo.ps1`) aynı PBKDF2 ölçümünü yapar (BCrypt üzerinden); süreler makineye göre değişir. `argon2` komut satırı aracı Windows'ta gelmez, bu yüzden Argon2id yalnız açıklanır. Anahtar değerleri iki platformda da **aynıdır** (aynı standart, aynı test vektörleri).



Kural

Parola saklarken **düz metin ya da düz SHA-256 kullanmayın**. Rastgele tuz + yeterli maliyetli bir KDF (Argon2id; değilse yüksek turlu PBKDF2 ya da scrypt) kullanın. Tuzu şifreyle birlikte saklayın (gizli olması gerekmez); maliyeti donanımınıza göre ayarlayıp yıllar içinde artırın.



Değerlendirici nasıl test eder?

Değerlendirici tuzun **rastgele ve kayıt başına benzersiz** olduğunu, tur/bellek parametrelerinin güncel öneriye uyduğunu ve parolanın türetmeden sonra bellekten silindiğini kontrol eder (bkz. Demo 8).

İleri: biber (pepper)

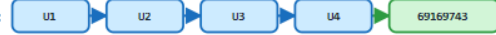
Tuza ek olarak, veritabanında **saklanmayan**, uygulama yapılandırmasında ya da bir donanım güvenlik modülünde (HSM) tutulan gizli bir **biber (pepper)** de eklenebilir. Böylece yalnız veritabanı çalınırsa (biber sızmadıkça) parola özetleri kaba kuvvete karşı bir kat daha korunur. Tuz gizli değildir; biber gizlidir.

Aşağıdaki animasyonda küçük bir tur zincirinin ($U_1 \rightarrow U_2 \rightarrow \dots \rightarrow U_n$) nasıl işlediğini ve aynı parolanın farklı tuzlarla nasıl tamamen farklı anahtarlar verdiğini izleyin; gerçek demodan ölçülen tur-süre tablosu da gösterilir.

parola = "dog123secret"

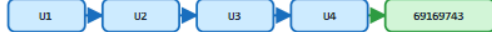
1. Parola: "dog123secret" — kısa ve tahmin edilebilir; doğrudan anahtar OLAMAZ.

parola = "dog123secret"

tuz A: 

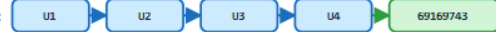
2. `crypto_pbkdf2_sha256(parola, tuz_A, 4 tur, ...)` — her tur bir önceki turun HMAC'ı: $U_1 \rightarrow U_2 \rightarrow \dots \rightarrow U_4$, sonra hepsinin XOR'u anahtarı verir (gerçek PBKDF2 100.000+ tur kullanır; burada 4 tur gösteriliyor).

parola = "dog123secret"

tuz A: 

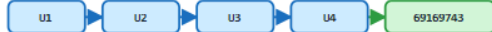
3. Aynı parola + AYNI tuz A ile yeniden hesaplandı: key = 69169743 — BİREBİR AYNI (tabloya açıklık).

parola = "dog123secret"

tuz A: 
tuz B: 

4. Aynı parola + FARKLI tuz B ile: key = F085F412 — tamamen FARKLI bir anahtar.

parola = "dog123secret"

tuz A: 

tuz B: 

Gerçek demodan ölçülen süreler (tur sayısı arttıkça):

1,000	10,000	100,000	600,000	2,000,000
1.24 ms	12.55 ms	126.75 ms	824.96 ms	2,808.32 ms

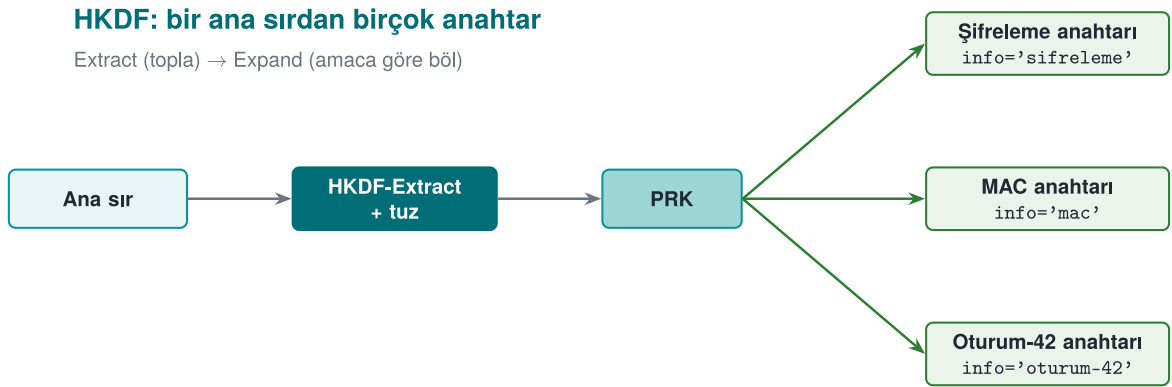
5. Gerçek program çıktısı: tur 1.000 -> 1,24 ms iken tur 2.000.000 -> 2,8 saniye. OWASP (2023): PBKDF2-HMAC-SHA256 için en az 600.000 tur; mümkünse Argon2id.

Örnek seçiciden **4 turluk zincir** (uyar) ile **6 turluk zincir** (zor) ve uç durumlardaki **tek tur** ile ****çok kısa parola****yı deneyin — ya da kendi parola:tur çiftinizi yazın (gösterim için tur sayısı 1-6 ile sınırlıdır; gerçek demo 100.000+ tur kullanır).

7. Ana sırdan anahtar türetme, oturum anahtarları ve ileri gizlilik

Uzun ömürlü bir "ana sır" **doğrudan** kullanılmaz. Ondan, her amaç ve her oturum için **ayrı** anahtarlar türetilir. Araç: **HKDF** (RFC 5869), iki aşamalı bir KDF:

- **Extract:** ana sır + tuz → tekdüze bir ara anahtar (PRK).
- **Expand:** PRK + "info" etiketi → istenen boyda anahtar. Farklı `info` → farklı anahtar.



Farklı `info` → farklı anahtar. Aynı sırdan üretilseler de biri sızsarsa diğerleri güvende kalır.

✎ Kısa tarihçe: HKDF

2010 — Hugo Krawczyk, "Cryptographic Extraction and Key Derivation: The HKDF Scheme" makalesiyle Extract-then-Expand yapısını biçimselleştirir; IETF bunu **2010'da RFC 5869** olarak yayımlar. HKDF bugün TLS 1.3'ün anahtar çizelgesi dahil birçok protokolün temel yapıtaşdır.

i Kitap tarifi

Kitap **tarif 4.11**'de "tek bir ana sırdan algoritmayla anahtar üretme" ve **4.13**'te anahtar malzemesini güvenli yönetme konusunu işler. HKDF bu fikrin standartlaşmış hâlidir. Her türetmenin **benzersiz bir ayırt edici** (info/label) içermesi şarttır.

7.1 İleri gizlilik (forward secrecy)

Anahtarları bir **tek yönlü zincir** olarak türetirsek — $K_0 = \text{ana sır}$, $K_i = \text{HKDF}(K_{i-1})$ — ve her oturumdan sonra K_{i-1} 'i **silersek**, saldırgan bugünkü K_n 'i ele geçirse bile geriye gidip eski $K_1 \dots K_{n-1}$ 'i (ve o oturumların verisini) **hesaplayamaz**. TLS 1.3 bu güvenceyi her oturumda geçici Diffie-Hellman ile sağlar (kitap **tarif 8.20–8.21**).

✎ Terimin kökeni

"**Perfect forward secrecy**" terimi, Whitfield Diffie, Paul van Oorschot ve Michael Wiener'in **1992**'de yayımlanan "Authentication and Authenticated Key Exchanges" makalesine kadar uzanır; bugün kısaca **forward secrecy** (ileri gizlilik) olarak anılır.

İleri gizlilik: eski anahtarlar geri hesaplanamaz

zincirin her halkası bir öncekinden tek yönlü türetilir



K4'ten K3, K2, K1 GERİ HESAPLANAMAZ (tek yönlü)

Bugün anahtar çalınrsa bile dünkü trafik çözülemez — ileri gizliliğin tanımı budur.

Demo 5 — HKDF ile oturum anahtarları ve ileri gizlilik zinciri



Demo 5 · code/week-03/05-hkdf-session · HKDF, oturum anahtarı, ileri gizlilik

hkdf_session.exe — gerçek çıktı

```

Separate keys per purpose, from one master secret:
info='...encryption...' -> 5a5ed417c87e3566...0d37002cb
info='...MAC...' -> 4c63a348654a076c...0ce1750e6
^ Same secret, different info -> different key.
=====
FORWARD SECRECY chain: Ki = HKDF(Ki-1), Ki-1 is wiped
K1 (session 1) = cbbe8fdd0b6ca8be
K2 (session 2) = 1113abf7352881e0
K3 (session 3) = a99c7ad8e0e6fb6b
K4 (session 4) = 4974f0e86d9dcb37
^ We now only have K4. Because the chain is ONE-WAY, K3, K2, K1
  cannot be recomputed backwards from K4.
  
```

Aynı ana sırdan farklı `info` etiketleriyle farklı anahtarlar çıktı; bir amaç için üretilen anahtar başka amaca kullanılamaz. İleri gizlilik zincirinde her adımda eski anahtar `crypto_wipe` (OPENSSL_cleanse/SecureZeroMemory) ile silindi; elde yalnız son anahtar kaldı ve tek yönlülük geçmişini korudu.



Kural

Ana sırdan **amaca ve oturuma göre** ayrı anahtarlar türetin (anahtar ayrımı). Her türetmeye benzersiz bir `info` /label koyun. Uzun ömürlü sırdan kısa ömürlü oturum anahtarları üretip **kullanınca silin**; mümkünse geçici (ephemeral) DH ile gerçek ileri gizliliği hedefleyin.

**Sahada nasıl uygulanır?**

Örnek mimaride sunucuyla anlaşılan bir ana sırdan; şifreleme anahtarı, MAC anahtarı ve her oturum için ayrı bir oturum anahtarı HKDF ile türetilir. Anahtar hiyerarşisini (hangi anahtar hangi anahtardan türer, ne kadar yaşar) [Hafta 10](#)'da ayrıntılı işleyeceğiz; bu hafta ilke şudur: **tek bir anahtarı her işe koşmayın.**

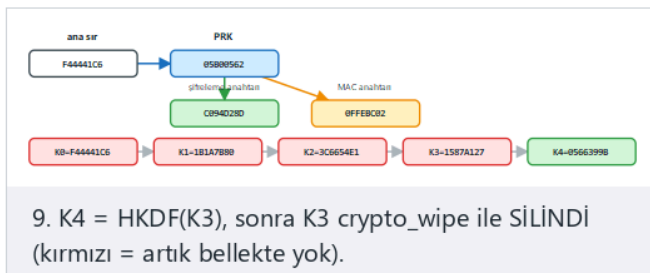
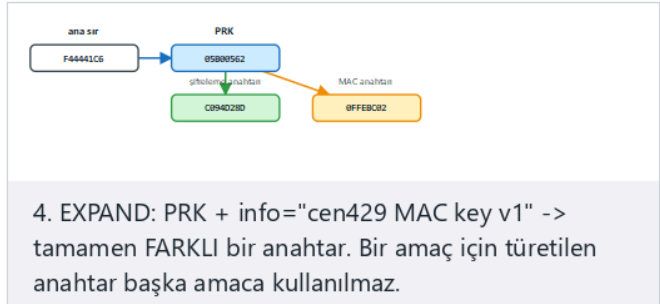
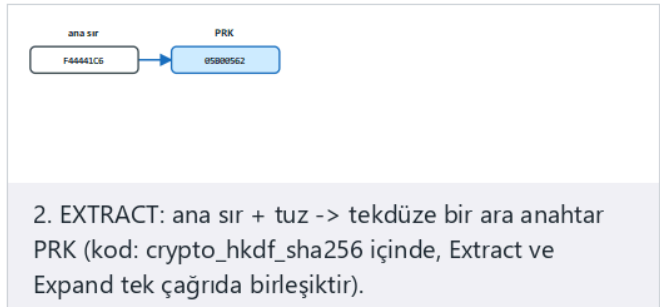
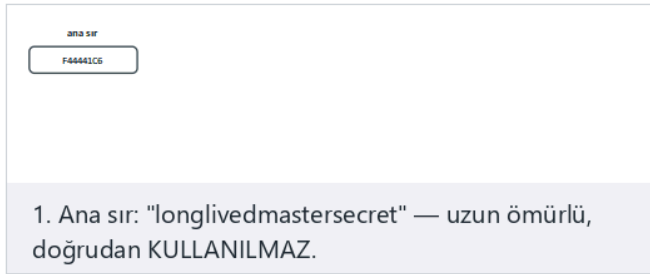
**Değerlendirici nasıl test eder?**

Değerlendirici anahtar hiyerarşisi belgesini ister ve her anahtarın **tek bir amacı** olduğunu, oturum anahtarlarının kısa ömürlü ve türetilmiş olduğunu, aynı türetilmiş anahtarın iki farklı yerde kullanılmadığını denetler. Ayrıca ileri gizliliğin sağlanıp sağlanmadığını (eski anahtarların silinip silinmediğini) sorar.

**Sık hatalar ve kontrol listesi**

- ✓ Her türetmenin benzersiz bir `info` /label'ı var mı?
- ✓ Şifreleme ve MAC için **ayrı** anahtarlar mı türetiliyor?
- ✓ Oturum anahtarları kullanımdan sonra siliniyor mu?
- ✓ Ana sır bellekte gereğinden uzun durmuyor, değil mi?

Aşağıdaki animasyonda `Extract` ve `Expand` 'in ana sırdan nasıl amaca özel anahtarlar ürettiğini, sonra ileri gizlilik zincirinin her adımda eski anahtarı nasıl sildiğini izleyin.



Örnek seçiciden **4 adımlık zincir** (uyar, gerçek demoyla aynı) ile **6 adımlık zincir** (zor) ve uç durumlardaki **tek adım** ile ****hiç ilerlemeyen zincir****i deneyin — ya da kendi ana_sır:adım çiftinizi yazın.

8. Dinamik anahtar yönetimi: yaşam döngüsü, hiyerarşi ve yenileme

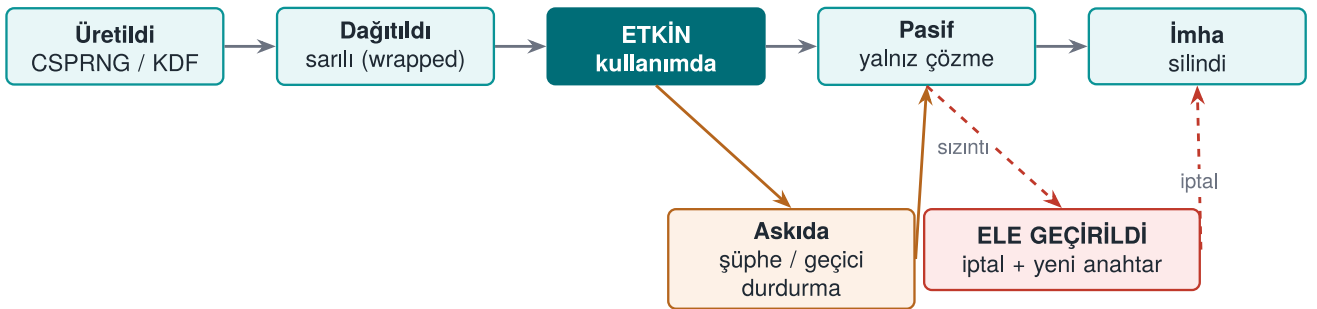
"Algoritma kırıldı" diye haber yapılan olayların çok küçük bir kısmı gerçekten algoritmanın kırılmasıdır. Büyük çoğunluk **anahtar yönetimi** hatasıdır: kaynak koda gömülmüş anahtar, hiç değişmeyen anahtar, her işe koşulan tek anahtar, silinmeyen eski anahtar. Kitap anahtar malzemesinin güvenli yönetimini Tarif 4.13'te kısaca işler; bu bölümde konuyu sahada kullanılan biçimiyle genişletiyoruz.

Bir anahtarın hayatı

NIST SP 800-57, bir anahtarın yaşam döngüsünü durumlarla tanımlar. Sade haliyle:

Bir anahtarın hayatı

üretimden imhaya — her geçiş kayıt altında



Kripto-periyodu dolan anahtar HEMEN silinmez: eski veri yeniden şifrelenene dek yalnız çözmede kalır.

Aşama	Sorulacak soru	Tipik hata
Üretim	Yeterli entropi var mı? Kim üretiyor (istemci mi, sunucu mu, HSM mi)?	<code>rand()</code> , sabit anahtar, istemcide üretilmemesi gereken anahtarı üretmek
Dağıtım	Anahtar yolda nasıl korunuyor?	Anahtarı düz metin olarak göndermek, yalnız TLS'e güvenmek
Saklama	Anahtar beklerken nerede, hangi korumayla duruyor?	Kaynak koda ya da yapılandırma dosyasına düz yazmak
Kullanım	Anahtar yalnız tek bir amaçla mı kullanılıyor?	Aynı anahtarla hem şifreleme hem MAC; aynı anahtarla sınırsız mesaj
Yenileme	Anahtar ne zaman ve nasıl değişiyor?	Hiç değişmemesi; değiştirtince eski verinin okunamaz hale gelmesi
İptal	Ele geçirildiğinde ne yapılıyor?	İptal mekanizmasının hiç olmaması
İmha	Bütün kopyaları siliniyor mu?	Yedeklerde, günlüklerde, bellekte kalan kopyalar

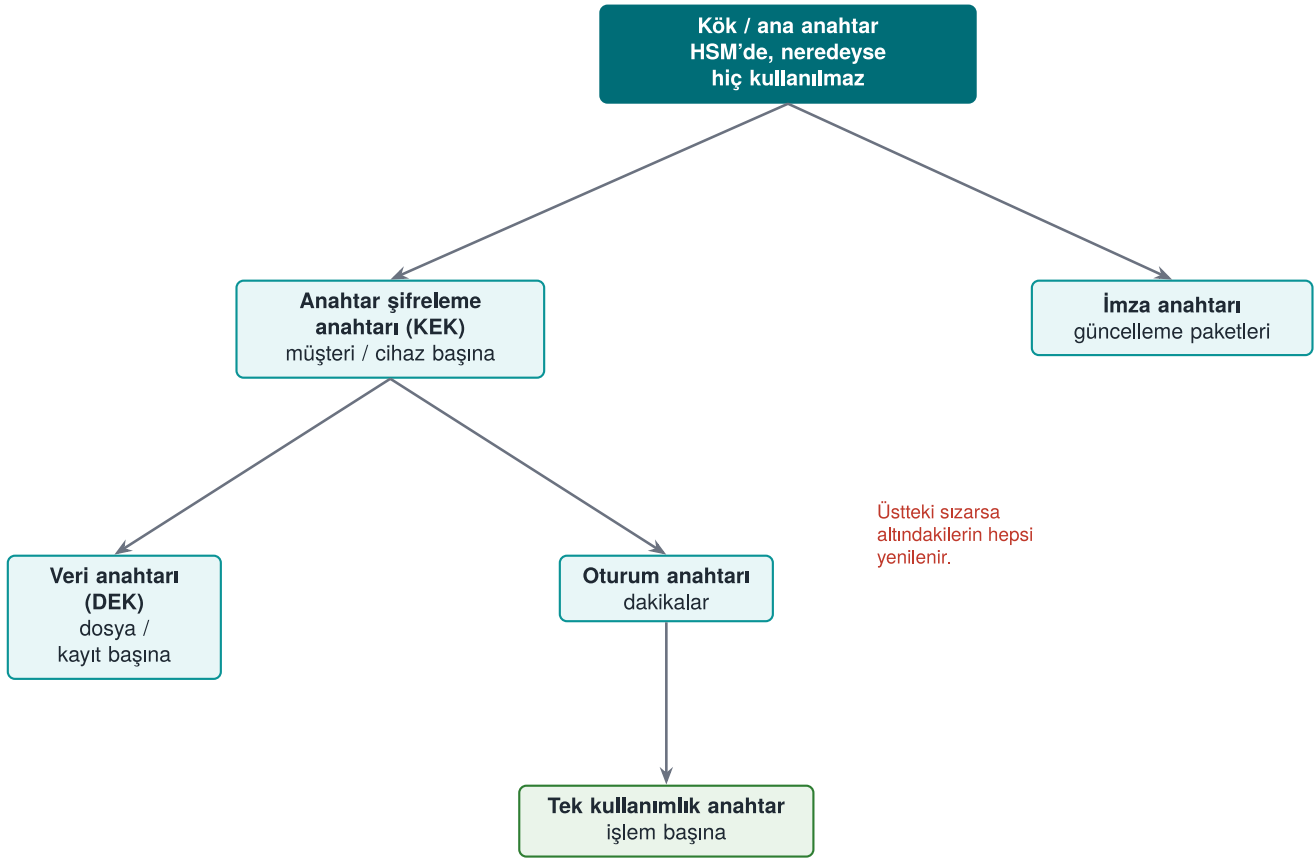
Kripto-periyot, bir anahtarın kullanılmasına izin verilen süredir. Süre dolduğunda anahtar yeni veriyi **şifrelemek** için kullanılmaz, ama eski veriyi **çözmek** için bir süre daha tutulabilir. Kripto-periyodu belirleyen etkenler: anahtarla korunan verinin miktarı, verinin ne kadar süre gizli kalması gerektiği, anahtarın ne kadar açıkta durduğu. İstemci cihazında duran bir anahtar, HSM'de duran bir anahtardan çok daha kısa ömürlü olmalıdır.

Anahtar hiyerarşisi: her işe ayrı anahtar

Tek bir anahtarla her şeyi yapmak yerine, anahtarlar bir **ağaç** halinde düzenlenir. Üstteki anahtarlar az kullanılır ve en iyi korunan yerde durur; alttakiler sık kullanılır, kısa ömürlüdür ve kaybedildiklerinde zarar küçüktür.

Anahtar hiyerarşisi: her işe ayrı anahtar

yukarıdakiler nadiren, aşağıdakiler sık kullanılır



İki temel fikir:

1. **Anahtar ayrımı:** Farklı amaçlar için farklı anahtarlar. Şifreleme anahtarı imzalamada, MAC anahtarı şifrelemede kullanılmaz. HKDF'nin `info` etiketi bu ayrımı kolaylaştırır (önceki bölüm).
2. **Zararın sınırlanması:** Alt düzeydeki bir anahtarın ele geçirilmesi yalnız o dalı etkiler. Tek kullanımlık bir ödeme anahtarı çalınırsa kaybedilen **tek bir işlemdir**.



Ödeme sistemlerinden bir hiyerarşi

EMV tarzı kart sistemlerinde tipik zincir şöyledir: kart çıkaran kurumun **ana anahtarı** (HSM'de) → kart numarası ve sıra numarasından türetilen **kart ana anahtarı** → işlem sayacından türetilen **oturum anahtarı** → her ödemede kullanılan **tek kullanımlık anahtar**. Telefonun güvenli elemanını kullanmayan mobil ödeme çözümlerinde telefona yalnız zincirin en altındaki, **sınırlı sayıda kullanılabilen** anahtarlar indirilir; anahtarlar tükenince sunucudan yenileri istenir (anahtar yenileme / replenishment). Telefon ele geçirilse bile saldırganın elinde birkaç işlemlik anahtar vardır ve bunlar sunucuda kolayca iptal edilir.

Zarflama (envelope encryption)

Büyük miktarda veriyi doğrudan ana anahtarla şifrelemek iki sorun yaratır: ana anahtar sık kullanılır (açıkta kalma süresi artar) ve anahtar değiştiğinde **bütün veri** yeniden şifrelenmelidir. Zarflama bu iki sorunu çözer:

Zarflama (envelope encryption)

veriyi DEK, DEK'i KEK şifreler



KEK yenilenince yalnız SARILMIŞ DEK'ler yeniden sarılır — terabaytlarca veri yerinde kalır.

1. Her dosya ya da kayıt için rastgele bir **veri anahtarı (DEK)** üretilir; veri bununla şifrelenir.
2. DEK, bir **anahtar şifreleme anahtarıyla (KEK)** sarılır (şifrelenir) ve şifreli verinin başlığına yazılır.
3. KEK güvenli bir yerde durur: işletim sisteminin anahtar deposu, TPM, HSM ya da bulut anahtar yönetim hizmeti.
4. KEK yenilediğinde yalnız küçük DEK'ler yeniden sarılır.

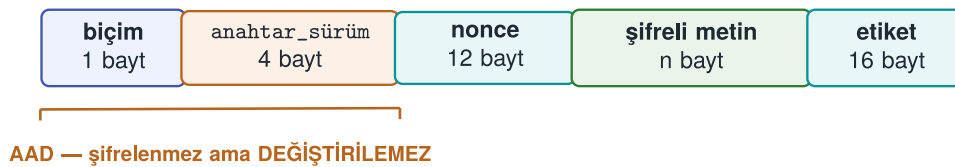
Bulut sağlayıcılarının anahtar yönetim hizmetleri (KMS) ve disk şifreleme sistemleri bu yöntemi kullanır.

Anahtar sürümü: yenilemeyi mümkün kılan küçük alan

Bir anahtarı yenileyebilmek için, şifreli verinin **hangi anahtarla** şifrelendiğini bilmek gerekir. Bu yüzden şifreli verinin başlığına bir **anahtar kimliği / sürüm** alanı konur:

Kayıt biçimi: anahtar yenilemeyi mümkün kılan alan

sürüm alanı olmadan anahtar değiştirilemez



Okurken önce sürüme bakılır, o sürümün anahtarıyla çözülür → eski kayıtlar bozulmadan anahtar yenilenir.

Okuyucu önce sürüme bakar, o sürümün anahtarını bulur ve çözer. Yeni yazmalar hep en yeni anahtarla yapılır; eski kayıtlar okundukça (ya da arka planda) yeni anahtarla yeniden şifrelenir. Başlık AAD olduğu için saldırgan sürüm alanını değiştirip uygulamayı eski, zayıf bir anahtara yönlendiremez.

Anahtarı nerede saklamalı?

Seçenek	Platform	Koruma	Not
Kaynak koda gömülü	Hepsi	Yok: <code>strings</code> ile bulunur	Asla (zorunlu değilse; zorunluysa gizleme + parçalama, 4. hafta)
Yapılandırma dosyası	Hepsi	Dosya izinleri	Yedeklere, depoya sızar
Ortam değişkeni	Hepsi	Zayıf	Alt süreçlere, çökme raporlarına sızar
DPAPI (<code>CryptProtectData</code>)	Windows	Kullanıcı ya da makine hesabına bağlı	Aynı kullanıcıyla çalışan zararlıya karşı korumaz
Anahtarlık / Keystore	macOS, iOS, Android	İşletim sistemi, bazen donanım destekli	Mobilde önerilen
TPM	Windows, Linux	Donanım; anahtar çıkarılamaz	Anahtar cihaza bağlanır
HSM / bulut KMS	Sunucu	Donanım; anahtar HSM'den hiç çıkmaz	Sunucu tarafının standardı

Windows DPAPI — bir anahtarı kullanıcı hesabına bağlı olarak sarmak

```
#include <windows.h>
#include <dpapi.h>
#pragma comment(lib, "crypt32.lib")

int wrap_key(const BYTE *key, DWORD n, DATA_BLOB *out)
{
    DATA_BLOB in = { n, (BYTE *)key };
    /* out->pbData is freed with LocalFree once you're done with it */
    return CryptProtectData(&in, L"vault-key", NULL, NULL, NULL,
        CRYPTPROTECT_UI_FORBIDDEN, out) ? 0 : -1;
}
```

Sahadan: kart şeması gereksinimleri ve bir anahtar envanteri

Mobil ödeme uygulamalarının uyması gereken kart şeması güvenlik gereksinimleri, bu bölümde anlattıklarımızı madde madde ister. İki şemanın (burada **Şema-A** ve **Şema-B**) kripto ve anahtar gereksinimlerinin özeti:

Gereksinim (özet)	Bu haftadaki karşılığı
Endüstri standardı algoritmalar ve protokoller kullanılmalı, güvenli yapılandırılmalı	Kripto API'lerini doğru kullanmak; TLS'i kodda kurmak
Anahtarların gizliliği ve bütünlüğü korunmalı ya da anahtarlar gizlenmeli	Güvenlik kabukları, whitebox (11. hafta)
Anahtarlar güvenli kurulmalı, güncellenmeli ve iş bitince bellekten ve geçici depolardan silinmeli	Yaşam döngüsü; kullanımda veri
Her anahtarın hiyerarşisi, kullanımı ve kripto-periyodu tanımlı olmalı; anahtar yalnız tek amacı için kullanılmalı	Anahtar hiyerarşisi, anahtar ayrımı
Yeterli entropili, tahmin edilemez rastgele sayılar kullanılmalı (ör. arka uçtan)	Rastgele sayılar bölümü
Rastgele sayı kaynağına yapılan çağrı kancalanamamalı	6. hafta (kanca algılama)
Veri şifreleme ve sınırlı kullanım anahtarları, uygulamaya ve cihaza bağlı, birbirinden farklı anahtarlarla korunmalı	Cihaz ve sürüm bağlama

Bu gereksinimleri karşılayan bir kütüphanenin kılavuzu, anahtarları dört gruba ayırır:

Grup	Nerede üretilir?	Nasıl korunur?
Dinamik cihaz anahtarları (whitebox anahtarları)	Sunucuda	Cihaza bağlı olarak indirilir ve saklanır; kullanımdan hemen önce bağdan çözülür
Dinamik ödeme anahtarları (sınırlı kullanım anahtarları)	Sunucuda	Cüzdan anahtarının (whitebox) altında indirilir ve aynı anahtarla veritabanında saklanır
Statik cihaz anahtarı (tek bir tane: dize gizleme anahtarı)	Derleme anında	Kaynak kodda parçalanmış ve gizlenmiş olarak durur
Sunucu sertifikası	Sertifika otoritesi	Açık anahtarı uygulamada sabitlenir

Aynı kılavuz iki önemli **olumsuz** beyanda da bulunur: kütüphane hesap parametreleri için **anahtar üretmez** ve ödeme için istemcide **rastgele sayı üretmez**. Bir değerlendirici için "yapmıyoruz" demek de "yapıyoruz" demek kadar değerlidir; saldırı yüzeyini daraltır.

Oturum anahtarları: sahadan bir örnek ve eleştirel okuma

İstemci ile sunucu arasında TLS'in **üstüne** ikinci bir koruma katmanı kurulduğunda, bu katmanın anahtarları oturum başına türetilir. Sahadan bir tasarımın ana hatları (adlar genelleştirilmiştir):

1. Sunucu 32 baytlık bir **oturum kimliği** üretir, bunu istemciye özgü **yapılandırma anahtarıyla** şifreler ve anlık bildirim kanalıyla (ya da istemcinin yeni oturum isteğine yanıt olarak) gönderir.
2. İstemci oturum kimliğini korunan anahtarla (whitebox AES, 11. hafta) çözer.

3. **Kimlik doğrulama kodu:** çözülmüş oturum kimliğinin bir kısmı, cüzdan kimliği ve cihaz parmak izi birlikte SHA-256'dan geçirilir. Sunucu aynı değeri kendisi hesaplar; iki taraf da karşısındakinin gerçek olduğunu doğrulamış olur.
4. **Oturum anahtarı:** yapılandırma anahtarının özeti bir **ana anahtar** olarak kullanılır; çözülmüş oturum kimliği ile bu özetten HMAC yoluyla 16 baytlık bir AES oturum anahtarı türetilir.
5. Mesajlar bu anahtarla **AES-CTR** kipinde şifrelenir ve her mesaja ayrıca 256 bitlik bir MAC eklenir; alıcı MAC'i doğrulamadan mesajı kullanmaz.

Bu tasarım derinlemesine savunmanın iyi bir örneğidir. TLS kırılrsa bile mesajlar ayrıca şifreli ve bütünlük korumalıdır; oturum anahtarı her oturumda değişir; kimlik doğrulama kodu cihaza bağlıdır. Bugün aynı tasarım yeniden yapılsaydı şu iyileştirmeler önerilirdi:

Tasarımda	Bugün önerilen	Neden?
AES-CTR + ayrı MAC (elle birleştirme)	Kütüphanenin AEAD kipi: AES-GCM, AES-CCM ya da ChaCha20-Poly1305	Tek çağrıda gizlilik ve bütünlük; "önce şifrele, sonra MAC" sırasını ve iki ayrı anahtarı elle doğru yönetme yükü kalkar
Özete dayalı özel HMAC türetmesi, çıktının kırılması	HKDF; şifreleme ve MAC için ayrı info	Standart, analiz edilmiş; anahtar ayrımı açıkça görünür
Bazı adımlarda SHA-1	SHA-256	SHA-1'e çakışma saldırıları pratik hale geldi
Oturum anahtarı simetrik bir kökten	İleri gizlilik için geçici (EC)DH	Kök anahtar çalınırsa geçmiş oturumların da çözülmesini önler



Eleştirel okuma bir beceridir

Birkaç yıl önce yazılmış, sertifikasyondan geçmiş bir tasarımı bugünün ölçütleriyle okumak, onu "yanlış" ilan etmek değildir. Tasarım kendi dönemindeki gereksinimleri karşılamıştır. Mühendislik becerisi, hangi seçimin **bugün** neden değişmesi gerektiğini gerekçesiyle söyleyebilmektir. [12. haftada](#) bir güvenlik kılavuzunu bu gözle inceleyeceğiz.



Kural

Anahtarları bir hiyerarşi içinde düzenleyin; her anahtarın tek bir amacı, bir krypto-periyodu ve bir imha anı olsun. Veriyi zarflayın; şifreli verinin başlığına anahtar sürümünü yazıp AAD ile koruyun. Anahtarı mümkün olan en güçlü depoda tutun ve istemci cihazına yalnız zararı sınırlı, kısa ömürlü anahtarlar indirin.



Değerlendirici nasıl test eder?

Anahtar yönetimi belgesini (her anahtar için: üretildiği yer, boyut, algoritma, amaç, saklandığı yer, krypto- periyot, imha anı) ister ve kaynak kodla karşılaştırır. Aynı anahtarın iki amaçla kullanılıp kullanılmadığını, iki farklı cihazda aynı anahtarın bulunup bulunmadığını (ortak statik anahtar), anahtar yenilemenin gerçekten çalışıp çalışmadığını ve eski anahtarların silinip silinmediğini denetler.

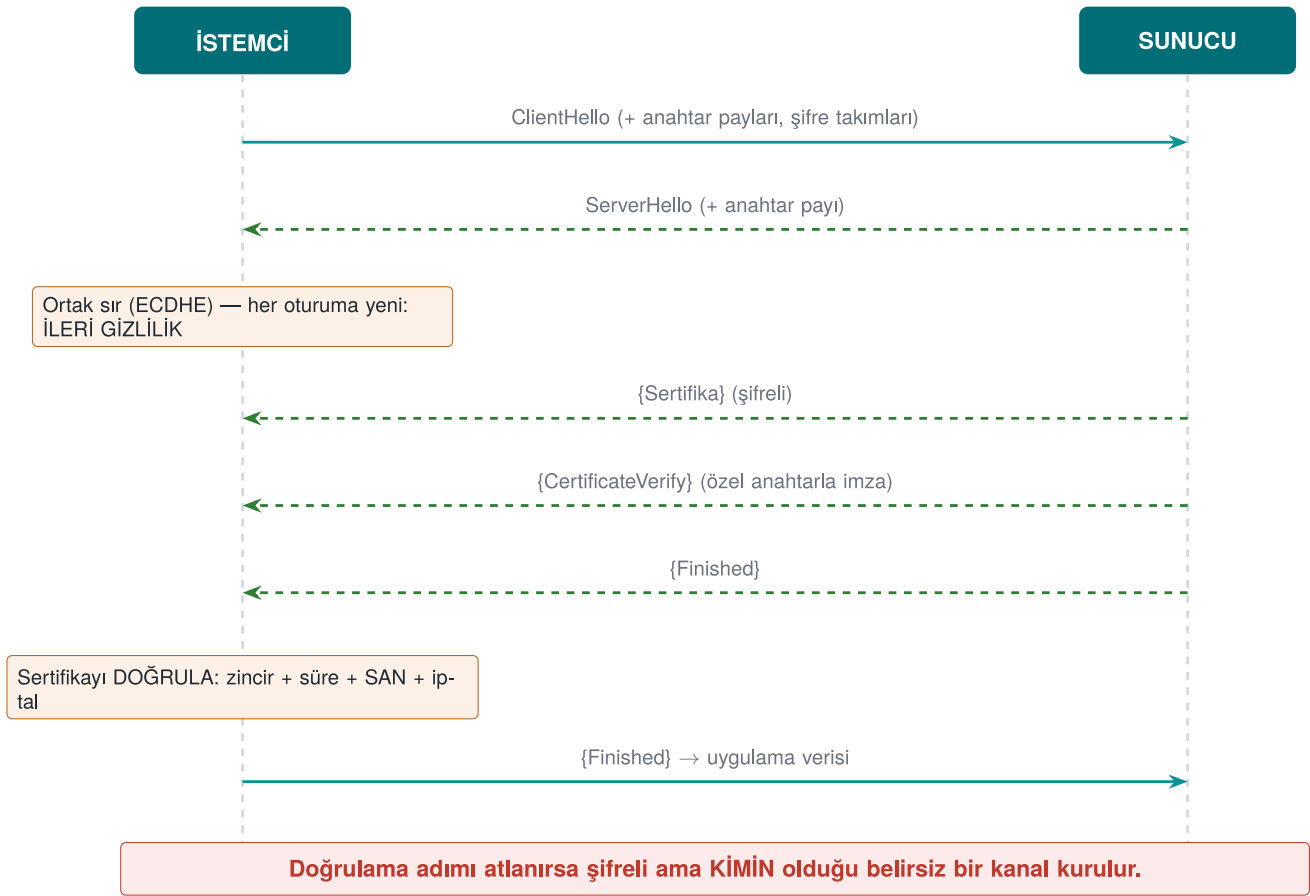
9. Aktarımda veri: TLS 1.3, sertifika doğrulama ve sabitleme

TLS (Transport Layer Security), iki uç arasında **anahtar anlaşmasını**, **AEAD ile şifrelemeyi** ve **sertifika doğrulamasını** birleştirip ağ üzerinde güvenli iletişim sağlayan protokoldür — HTTPS'in altındaki katmandır (eski adıyla SSL). Aktarımdaki veriyi koruyan standart budur. Bugün **TLS 1.3** kullanılır; SSLv2/SSLv3 ve TLS 1.0/1.1 kırık kabul edilir ve bırakılmıştır (POODLE, BEAST gibi saldırılar).

9.1 TLS 1.3 el sıkışması

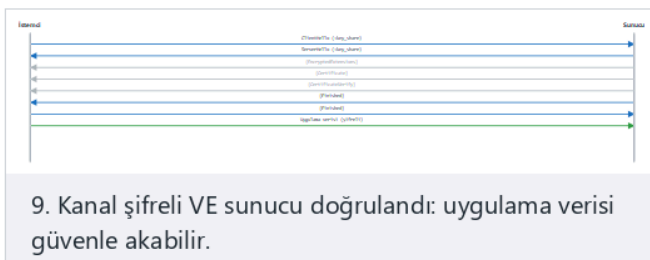
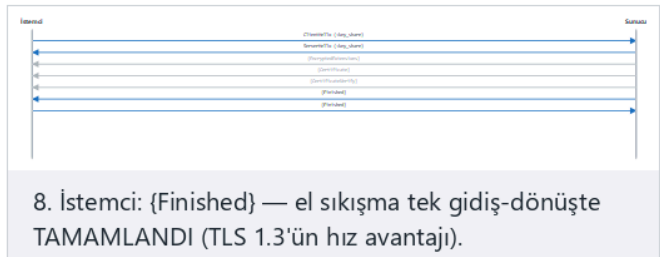
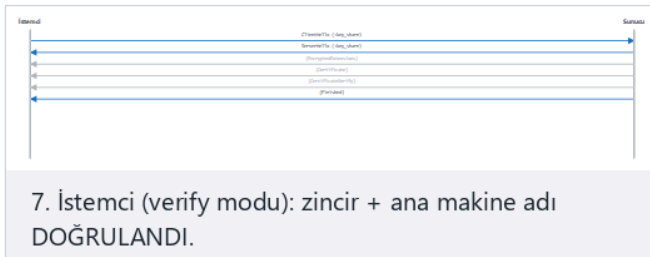
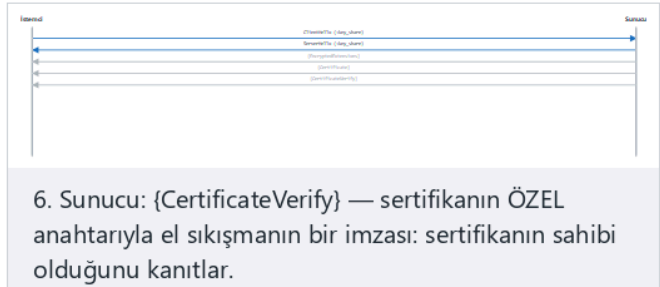
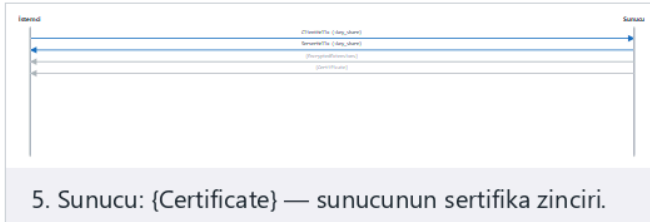
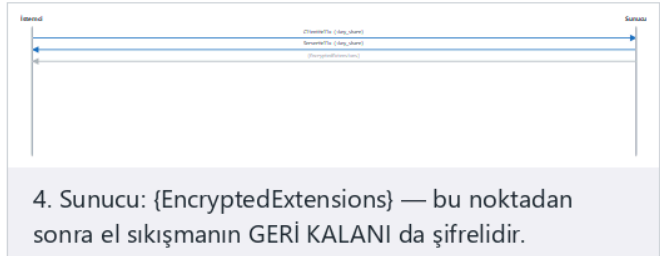
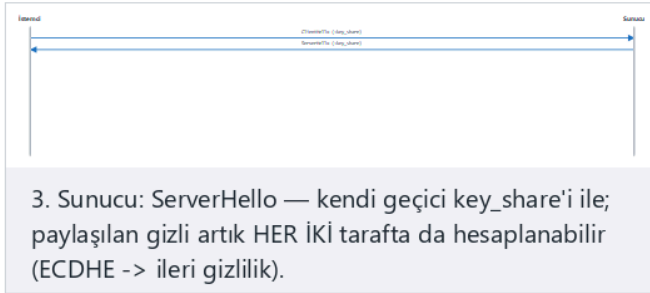
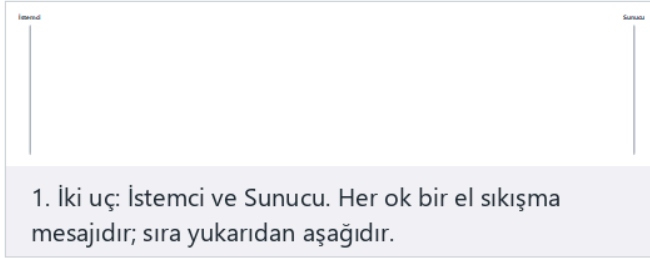
TLS 1.3 el sıkışması

tek gidiş-dönüşte anahtar anlaşması ve kimlik doğrulama



TLS 1.3'ün iki kilit özelliği: (1) el sıkışma **tek gidiş-dönüşte** biter (hızlı); (2) anahtar değişimi her zaman **geçici (ephemeral)** **ECDHE** ile yapılır, yani **ileri gizlilik** varsayılan olarak açıktır (RFC 8446).

Aşağıdaki animasyonda el sıkışma mesajlarının sırasını iki yaşam çizgisi (İstemci/Sunucu) üzerinde adım adım izleyin: normal el sıkışma, karşılıklı TLS (mTLS), sertifika reddiyle yarıda kesilen el sıkışma ve doğrulamasız (güvensiz) bir istemcinin yine de tamamlanan ama korunmasız el sıkışması.



Örnek seçiciden **normal el sıkışma** (uyar) ile **karşılıklı TLS** (zor) ve uç durumlardaki **yarıda kesilen** ile **doğrulamasız** senaryoları deneyin.

9.2 Doğrulamanın üç katmanı — ve en tehlikeli tuzak

Bir TLS bağlantısı, karşı tarafın sunduğu sertifikayı **doğrulamazsa** hiçbir şey ifade etmez. Doğrulama üç sorudan oluşur:

Katman	Soru	Araç
Zincir doğrulama	Sertifikayı güvenilir bir kök (CA) mı imzaladı, süresi geçmiş/iptal mi?	CA deposu, CRL/OCSP (H10)
Ana makine adı	Sertifika gerçekten bu sunucu için mi?	SAN/CN karşılaştırma (<code>SSL_set1_host</code>)
Sabitleme (pinning)	Beklediğim belirli anahtar/sertifika mı?	SPKI SHA-256 karşılaştırma



OpenSSL'in ölümcül varsayılanı

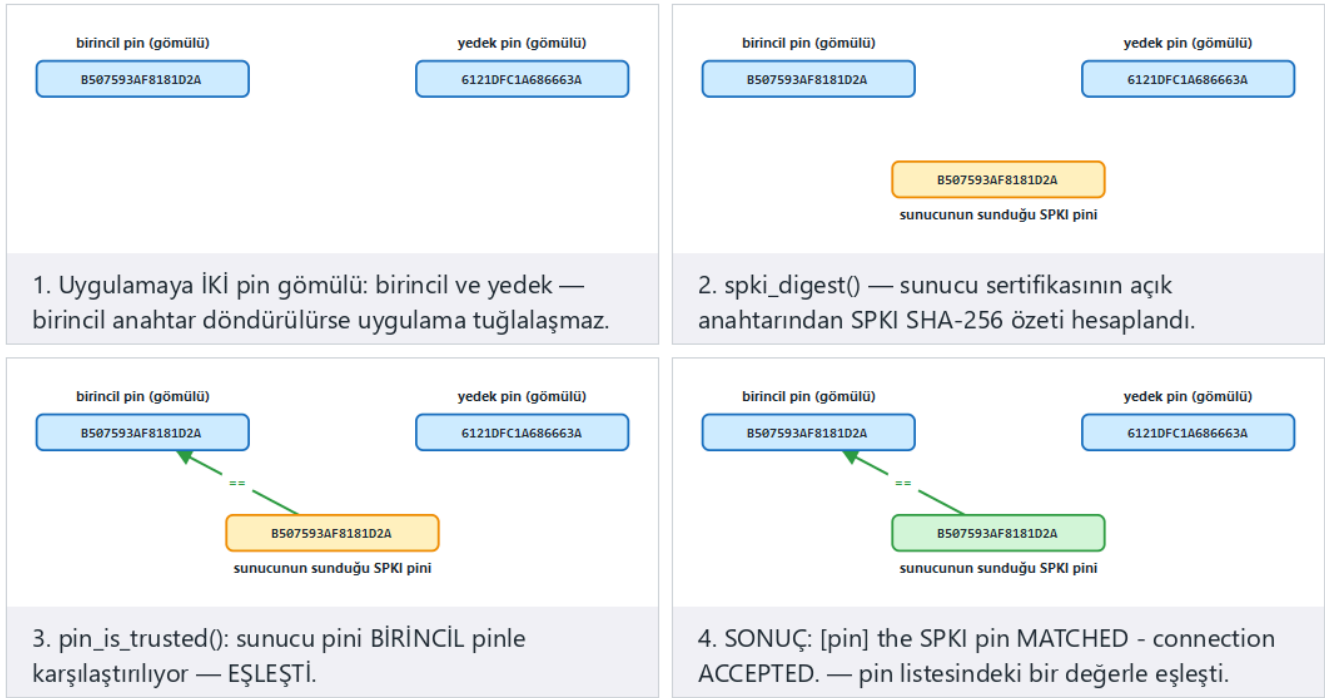
OpenSSL bir istemci socketinde sertifikayı **varsayılan olarak doğrulamaz** (kitap **tarif 10.7** bunu "olabilecek en kötü varsayılan" diye tanımlar). `SSL_CTX_set_verify` çağrılmazsa ve ana makine adı `SSL_set1_host` ile ayarlanmazsa, saldırgan araya girip **kendi sertifikasını** sunar ve istemci hiçbir şey anlamaz.

Zincir doğrulama şöyle işler: sunucu bir **uç (leaf) sertifika** sunar; bu sertifikayı bir **ara CA**, onu da güvenilir bir **kök CA** imzalamıştır. İstemci, elindeki güvenilir kök listesine (trust store) ulaşana kadar imzaları **yukarı doğru** doğrular; ayrıca her sertifikanın **süresini** ve **iptal durumunu** (CRL/OCSP) kontrol eder. Demo 6'da bu güven kökünü kendi ürettiğimiz küçük bir CA ile kuruyoruz: "gerçek" sunucu sertifikası bu CA tarafından imzalıdır, "saldırgan" sertifikası ise kendi imzalıdır (kök listesinde yoktur), bu yüzden zincir doğrulaması onu reddeder. PKI, sertifika zincirleri, CRL ve OCSP'yi **Hafta 10**'da ayrıntılı işleyeceğiz.

9.3 Sabitleme (pinning) ve TOFU

- **Sertifika sabitleme:** İstemci, sunucunun **beklenen açık anahtarının** (SPKI) SHA-256 özetini gömer. Zincir geçerli olsa bile (ör. saldırgan sahte ama "geçerli" bir sertifika elde etmiş) anahtar tutmuyorsa bağlantı reddedilir. Mobil uygulamalarda yaygındır (OWASP MASVS-NETWORK). Kitap **tarif 10.9** buna "beyaz liste ile doğrulama" der.
- **TOFU** (Trust On First Use): PKI yoksa, ilk bağlantıdaki anahtar hatırlanır; sonraki bağlantılarda **değişirse** uyarılır. SSH'in yaptığı budur (kitap **tarif 8.19**). PKI kadar güçlü değildir ama ilk bağlantıdan sonra araya girmeyi zorlaştırır.

Aşağıdaki animasyonda uygulamaya gömülü **birincil** ve **yedek** pin ile sunucunun sunduğu pinin nasıl karşılaştırıldığını izleyin: eşleşme, planlı anahtar rotasyonuyla yedeğe düşme ve hiçbir pinle eşleşmeyen bir saldırı denemesi.



Örnek seçiciden **birincil pinle eşleşme** (uyar) ile **yedek pine düşme** (zor) ve uç durumlardaki **hiçbir pinle eşleşmeme** ile ****çok kısa sunucu adı****nı deneyin.

9.4 TLS 1.2'den 1.3'e ne değişti?

TLS 1.3 yalnız bir sürüm artışı değil, **temiz bir sadeleşmedir**. Eski sürümlerde onlarca "şifre takımı" (cipher suite) vardı ve birçoğu zayıftı; yanlış yapılandırma yaygın bir açıktı. TLS 1.3 bunu köklü biçimde temizledi:

Konu	TLS 1.2	TLS 1.3
El sıkışma	2 gidiş-dönüş	1 gidiş-dönüş (daha hızlı)
Anahtar değişimi	RSA ya da DH (ileri gizlilik seçime bağlı)	Yalnız geçici (EC)DHE → ileri gizlilik varsayılan
Şifre takımları	Çok sayıda, bazıları zayıf (RC4, CBC, export)	Yalnız birkaç güçlü AEAD takımı
Kırık özellikler	Sıkıştırma, yeniden anlaşıma, statik RSA	Kaldırıldı

⚠ Düşürme (downgrade) ve 0-RTT tuzakları

Saldırgan bazen istemciyi **daha eski, zayıf bir sürüm**e düşürmeye çalışır (downgrade attack). Bu yüzden istemcide **minimum sürümü** (en az TLS 1.2, tercihen 1.3) zorlamak önemlidir. TLS 1.3'ün "0-RTT" hızlandırması, ilk mesajın **yeniden oynatmaya** (replay) açık olabileceği bir ödünleşmedir; ödeme gibi tekrar edilmemesi gereken işlemlerde 0-RTT kullanılmaz.

Demo 6 — Doğrulamasız istemci → localhost MITM → hostname + SPKI pinning

Demo 6 · `code/week-03/06-tls-pinning` · yalnız Linux/WSL · CWE-295 (yanlış sertifika doğrulama)

Demo kendi kök CA'mızı, CN=localhost için **CA tarafından imzalı "gerçek"** bir sertifika ve **kendi imzalı "saldırgan"** bir sertifika üretir; iki yerel TLS sunucusu (127.0.0.1) başlatır. İstemci üç modda dener: doğrulamasız, zincir+hostname doğrulayan ve SPKI pinleyen. **Windows'ta bu demo WSL'de çalıştırılır** (`.\demo.ps1` bunu anlatır).

İstemcinin özü:

tls_client.c (özet)

```
SSL_CTX_set_min_proto_version(ctx, TLS1_2_VERSION); /* at least TLS 1.2 */
if (verify) {
    SSL_CTX_load_verify_locations(ctx, ca_pem, NULL); /* the trusted CA */
    SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);
    SSL_set1_host(ssl, host); /* the hostname! */
}
/* in pin mode: the server's PUBLIC KEY's (SPKI) SHA-256 must match */
i2d_X509_PUBKEY(X509_get_X509_PUBKEY(cert), &der);
EVP_Digest(der, len, actual, NULL, EVP_sha256(), NULL);
if (memcmp(actual, expected_pin, 32) != 0) { /* REJECT */ }
```

sh demo.sh — gerçek çıktı (WSL)

```
The real server's SPKI pin (SHA-256): 1dabd67747366675...ac36b3f7c
=====
SCENARIO 1 - Insecure client to the ATTACKER's server (port 14444):
[insecure] no verification - ACCEPTS whatever it gets (DANGEROUS).
^ The fake certificate was accepted. A MITM would have succeeded.
SCENARIO 2 - Verifying client to the REAL server (port 14443):
[verify] chain + hostname VERIFIED - ACCEPTED.
SCENARIO 3 - Verifying client to the ATTACKER's server (port 14444):
[verify] HANDSHAKE FAILED - connection REJECTED.
Reason: certificate verification error (18: self signed certificate)
SCENARIO 4 - Pinning client to the REAL server, with the CORRECT pin:
[pin] the SPKI pin MATCHED - connection ACCEPTED.
SCENARIO 5 - Pinning client to the ATTACKER's server, with the REAL pin:
[pin] the SPKI pin did NOT match - connection REJECTED.
^ The key did not match: pinning stopped the MITM.
```

Beş senaryo tek bir hikâye anlatır: **doğrulamasız istemci** saldırıncının sahte sertifikasını sorgusuz kabul eder (MITM başarılı olurdu). **Zincir + hostname doğrulaması** saldırıncının kendi imzalı sertifikasını reddeder. **SPKI sabitleme** ise saldırıncı zincir açısından "geçerli" bir sertifika bulsa bile, **beklenen anahtar** tutmadığı için bağlantıyı keser.

Kural

İstemcide doğrulamayı **elle açın**: güvenilir CA deposunu yükleyin, `SSL_VERIFY_PEER` verin, **ana makine adını** (`SSL_set1_host`) ayarlayın ve minimum sürümü TLS 1.2 (tercihen 1.3) yapın. Yüksek riskli uygulamalarda **SPKI sabitleme** ekleyin. Hata durumunda **kapalı kalın** (fail-closed): istisnayı yutup güven deposuna düşmeyin.

**Aktarımda 'yalnız TLS'e güvenmeme'**

TLS kanalı korur ama **uç noktalar** güvenilirmez olabilir (kök alınmış telefon, araya konmuş vekil, hatalı yapılmış pinning). Yüksek güvenli tasarımlarda **TLS'in içine** bir kat daha konur: mesaj düzeyinde AEAD şifreleme + MAC + sunucu kimlik doğrulama kodu. Böylece TLS aşılsa bile mesajın gövdesi hâlâ şifrelidir. Bu, Bölüm 14'daki güvenlik kabuğunun en dış iki katmanıdır.

**Değerlendirici nasıl test eder?**

Değerlendirici localhost'ta bir vekil ile araya girer, **kendi CA'sını** sunar ve uygulamanın bağlantıyı reddedip reddetmediğine bakar. Pinlenmiş uygulamada, geçerli ama farklı anahtarlı bir sertifika sunup reddi doğrular; ayrıca kod incelemesinde "istisna yutuluyor mu, güven deposuna düşülüyor mu?" diye bakar.

**Sık hatalar ve kontrol listesi**

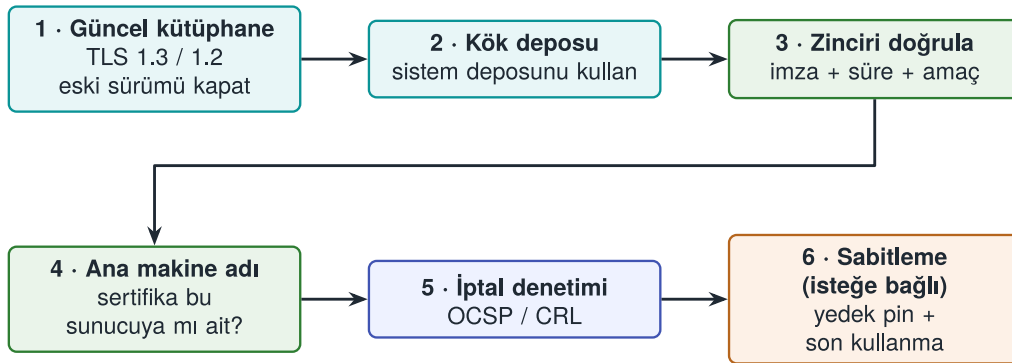
- ✓ `SSL_CTX_set_verify(..., SSL_VERIFY_PEER, ...)` çağrıldı mı?
- ✓ Ana makine adı (`SSL_set1_host`) ayarlandı mı? (Zincir doğru ama ad yanlış olabilir.)
- ✓ Minimum protokol sürümü $\geq$ TLS 1.2 mi?
- ✓ Sertifika hatası **yutulmuyor**, bağlantı gerçekten kesiliyor mu?
- ✓ Pinlenen değer **SPKI** mı (sertifikanın kendisi değil; sertifika yenilenince anahtar aynı kalır)?

10. TLS'i kodda doğru kurmak: adım adım istemci, sabitleme ve sık hatalar

Önceki bölüm TLS 1.3'ün **ne yaptığını** ve Demo 6 doğrulamasız bir istemcinin nasıl kandırıldığını gösterdi. Bu bölümde bir TLS istemcisini **satır satır** doğru kurmayı, sabitlemeyi sahada sürdürülebilir biçimde uygulamayı ve gerçek uygulamalarda en sık görülen hataları işliyoruz. Kitap bu konuyu Tarif 9.1 (SSL istemcisi), 10.7 (karşı tarafın sertifikasını doğrulama), 10.8 (ana makine adı denetimi) ve 10.9 (beyaz listeyi doğrulama) ile anlatır; aşağıdaki kod bu tariflerin OpenSSL 1.1.1.x'e güncellenmiş halidir.

TLS istemcisini doğru kurmanın adımları

tek bir adım atlanırsa MITM açık kalır



En sık hata: zincir doğrulanır ama ana makine adı denetlenmez — sertifika geçerli, sunucu yanlış.

OpenSSL ile doğru bir istemci: yedi adım

Doğru bir TLS istemcisi, adım adım (öğretici örnek — OpenSSL 1.1.1 / 3.x)

```

#include <openssl/ssl.h>
#include <openssl/err.h>

/* Returns a BIO that can be read/written on success; call BIO_free_all + SSL_CTX_free when done. */
BIO *tls_connect(SSL_CTX **ctx_out, const char *host, const char *port)
{
    BIO *bio = NULL;
    SSL *ssl = NULL;
    SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());           /* 1 */
    if (!ctx) return NULL;
    SSL_CTX_set_min_proto_version(ctx, TLS1_2_VERSION);        /* 2 */
    SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);            /* 3 */
    if (SSL_CTX_set_default_verify_paths(ctx) != 1) goto fail; /* 4 */

    if ((bio = BIO_new_ssl_connect(ctx)) == NULL) goto fail;
    BIO_get_ssl(bio, &ssl);
    SSL_set_tlsext_host_name(ssl, host);                        /* 5: SNI */
    if (SSL_set1_host(ssl, host) != 1) goto fail;               /* 6: hostname check */
    BIO_set_conn_hostname(bio, host);
    BIO_set_conn_port(bio, port);

    if (BIO_do_connect(bio) <= 0 || BIO_do_handshake(bio) <= 0) goto fail;
    if (SSL_get_verify_result(ssl) != X509_V_OK) goto fail;     /* 7 */

    *ctx_out = ctx;
    return bio;
fail:
    ERR_print_errors_fp(stderr);
    BIO_free_all(bio);
    SSL_CTX_free(ctx);
    return NULL;
}
  
```

**Bu, `tls_client.c`'in kendisi değil**

`tls_client.c` doğrulamayı tek bir `SSL_connect()` çağrısına bırakır (OpenSSL el sıkışmayı içeride yürütür); yukarıdaki `tls_connect()` aynı yedi adımı **BIO arayüzüyle elle** göstermek için yazılmış öğretici bir örnektir (kitabın Tarif 9.1/10.7/10.8/10.9'unun güncellenmiş hâli). İkisi de aynı yedi adımı uygular.

Adım	Ne yapar?	Atlanırsa
1	Sürümden bağımsız istemci bağlamı	—
2	En düşük sürümü TLS 1.2 yapar	Sürüm düşürme saldırısına açık kalır
3	Karşı tarafın sertifikasını doğrula	OpenSSL varsayılan olarak doğrulamaz: herhangi bir sertifika kabul edilir
4	İşletim sisteminin güvenilir kök deposunu yükler	Hiçbir sertifika doğrulanamaz; geliştirici "çalışmıyor" diye 3. adımı kapatır
5	Sunucuya hangi adı istediğini söyler (SNI)	Aynı IP'de birden fazla site varsa yanlış sertifika gelir
6	Sertifikadaki adın bu sunucu olduğunu denetler	Başka bir site için alınmış geçerli bir sertifika kabul edilir
7	Doğrulama sonucunu açıkça sorar	Bir yapılandırma hatası sessizce geçebilir (kemer + askı)

**Adım 6 neden bu kadar önemli?**

Zincir doğrulaması yalnız "bu sertifikayı güvenilir bir CA imzalamış mı?" sorusunu cevaplar. Saldırgan kendi alan adı için **tamamen geçerli** bir sertifika alabilir. Ad denetimi yapılmazsa, istemci `bank.example` yerine `attacker.example` sertifikasını da kabul eder. 2012'de yayımlanan "dünyanın en tehlikeli kodu" başlıklı çalışma, çok sayıda gerçek uygulamanın ve kütüphanenin tam olarak bu adımı atladığını göstermişti.

Sabitlemeyi eklemek: SPKI özeti

Doğrulama başarılı olduktan sonra sunucunun **açık anahtarının** özeti, uygulamaya gömülü listeyle karşılaştırılır. Sertifikanın kendisi yerine açık anahtar bilgisinin (SPKI) sabitlenmesi, sertifika aynı anahtarla yenilendiğinde uygulamanın çalışmaya devam etmesini sağlar.

SPKI SHA-256 sabitleme

```

#include <openssl/x509.h>
#include <openssl/sha.h>

static const unsigned char TRUSTED_PINS[][32] = {
    { /* the primary key's SPKI SHA-256 digest (32 bytes) */ },
    { /* the BACKUP key's digest: generated offline, not yet in use */ },
};

int check_pin(SSL *ssl)
{
    X509 *cert = SSL_get_peer_certificate(ssl);    /* 3.x: SSL_get1_peer_certificate */
    if (!cert) return 0;
    X509_PUBKEY *pk = X509_get_X509_PUBKEY(cert);
    int n = i2d_X509_PUBKEY(pk, NULL);
    unsigned char *der = OPENSSL_malloc((size_t)n), *p = der;
    i2d_X509_PUBKEY(pk, &p);                        /* p advances; der stays at the start */
    unsigned char digest[32];
    SHA256(der, (size_t)n, digest);
    OPENSSL_free(der);
    X509_free(cert);

    for (size_t i = 0; i < sizeof TRUSTED_PINS / sizeof TRUSTED_PINS[0]; i++)
        if (CRYPTO_memcmp(digest, TRUSTED_PINS[i], 32) == 0)
            return 1;
    return 0;                                        /* no match: CUT the connection */
}

```

Bir sunucunun SPKI özetini komut satırından hesaplamak:

```

openssl s_client -connect server.example:443 -servername server.example </dev/null 2>/dev/null \
| openssl x509 -pubkey -noout \
| openssl pkey -pubin -outform der \
| openssl dgst -sha256 -binary | base64

```

Sabitlemeyi sahada sürdürmek

Sabitleme güçlü ama **kırılgan** bir önlemdir: sunucunun anahtarı değiştiğinde, eski sürümdeki bütün uygulamalar bağlanamaz hale gelir. Tarayıcılar bu yüzden HTTP üzerinden dinamik sabitlemeyi (HPKP) 2018'de bıraktı; mobil ve masaüstü uygulamalarda ise sabitleme hâlâ yaygındır (OWASP MASVS-NETWORK). Sürdürülebilir bir sabitleme için:

Kural	Neden?
Sertifikayı değil SPKI 'yi sabitle	Aynı anahtarla yenilenen sertifika çalışmaya devam eder
Her zaman en az bir yedek pin göm	Birincil anahtar ele geçerse ya da kaybolursa yedeğe geçilir; uygulama "tuğlalaşmaz"
Yedek anahtarı çevrimdışı üret ve sakla	Birincil anahtarla aynı anda ele geçmesin
Pin listesini güncelleme sürecine bağla	Yeni pin, eski pin kaldırılmadan önce bir sürümle dağıtılır
Pin hatasını raporla	Pin hatası ya bir saldırı ya da bir yapılandırma hatasıdır; ikisini de bilmek istersiniz
Pinlemeyi bir yapılandırma bayrağıyla kapatılabilir bırakma	Saldırgan önce bayrağı değiştirir



Uç sertifika mı, ara CA mı?

Uç (leaf) sertifikanın anahtarını sabitlemek en sıkı seçenektir ama en çok bakım gerektirir. Ara CA'nın anahtarını sabitlemek, o CA'nın imzaladığı her sertifikayı kabul eder; bakım kolaylaşır ama güvence azalır. Seçim, koruma planına bir **ödünleşim kaydı** olarak yazılır (1. hafta).

Windows'ta TLS: WinHTTP ve Schannel

Windows'un yerleşik TLS yığını **Schannel**'dir; uygulamalar çoğunlukla onu **WinHTTP** ya da WinINet üzerinden kullanır (kitabın Tarif 9.4'te anlattığı WinINet'in güncel karşılığı). OpenSSL'in tersine WinHTTP, sertifikayı ve ana makine adını **varsayılan olarak doğrular**. Tehlike, bu doğrulamayı kapatan bayraklardadır:

Asla sürüm derlemesine girmemesi gereken satır

```
DWORD flags = SECURITY_FLAG_IGNORE_UNKNOWN_CA |
               SECURITY_FLAG_IGNORE_CERT_CN_INVALID |
               SECURITY_FLAG_IGNORE_CERT_DATE_INVALID;
WinHttpSetOption(request, WINHTTP_OPTION_SECURITY_FLAGS, &flags, sizeof flags);
```

Bu satır genellikle geliştirme sırasında kendi imzalı bir test sunucusuna bağlanmak için eklenir ve **unutulur**. Doğrusu, test sunucusunun CA'sını geliştirme makinesinin güven deposuna eklemektir; kod aynı kalır. WinHTTP'de sabitleme için, istek gönderildikten sonra `WinHttpQueryOption(..., WINHTTP_OPTION_SERVER_CERT_CONTEXT, ...)` ile sunucu sertifikası alınır ve açık anahtar bilgisinin özeti yukarıdaki gibi karşılaştırılır.

Eleştirel okuma: sessizce açılan kapı (fail-open)

Sahada kullanılmış bir sabitleme gerçekleştiriminin sadeleştirilmiş ve yeniden yazılmış hali aşağıdadır. Kod ilk bakışta doğru görünür: önce zinciri doğruluyor, sonra sunucunun açık anahtarını uygulamada saklanan anahtarla karşılaştırıyor. Hatayı bulabilir misiniz?

Sabitleme — sadeleştirilmiş örnek (Java)

```

public void checkServerTrusted(X509Certificate[] chain, String authType) throws CertificateException
{
    defaultValidator.checkServerTrusted(chain, authType);           // 1. chain validation
    try {
        PublicKey expected = keyStore.getCertificate("ca").getPublicKey();
        if (!Arrays.equals(expected.getEncoded(), chain[0].getPublicKey().getEncoded()))
            throw new CertificateException("pin did not match");    // 2. pinning
    } catch (KeyStoreException e) {
        e.printStackTrace();                                       // 3. ???
    }
}

```

3 numaralı satır, anahtar deposundan okuma **başarısız olursa** hatayı yazdırıp metodu **normal biçimde** bitirir. Metot istisna fırlatmadan döndüğü için bağlantı **kabul edilir**: sabitleme hiç yapılmamış olur. Bir güvenlik denetiminin hata durumunda "geçti" sonucuna düşmesine **fail-open** (açık kalarak başarısız olma) denir. Aynı kod tabanında dikkat çeken başka noktalar da vardı:

- Zincir doğrulaması başarısız olduğunda, uygulamaya gömülü depo ile ikinci bir doğrulama yapılıyor ve bu ikinci doğrulamada **iptal denetimi kapatılıyordu**.
- Sertifika dosyası ya da onun bütünlük belirteci bulunamazsa, yalnız bir günlük satırı yazılıp bağlantı kurulmaya **devam ediliyordu**.
- Tek bir pin vardı; **yedek pin** yoktu.

Doğrusu **fail-closed** (kapalı kalarak başarısız olma) tasarımıdır: bir güvenlik denetiminin sonucunu belirleyemiyorsanız, sonuç "reddet"tir.

Düzeltilmiş: her belirsizlik reddedilir

```

    } catch (KeyStoreException | RuntimeException e) {
        throw new CertificateException("pin check could not be performed", e);    // ambiguity =
reject
    }

```

✓ Kural: güvenlik denetimleri kapalı kalarak başarısız olur

Bir denetimin her çıkış yolunu (istisna, boş değer, eksik dosya, zaman aşımı) tek tek düşünün ve her birinin **reddetmeye** vardığını doğrulayın. Kod incelemesinde `catch` bloklarını ve "bulunamadı, devam ediliyor" günlük satırlarını ayrıca arayın: fail-open hatalarının neredeyse hepsi oradadır.

Gerçek uygulamalarda en sık TLS hataları

Hata	Nerede görülür?	Sonuç
Doğrulama geri çağırısının her zaman "geçerli" döndürmesi	OpenSSL <code>verify_callback</code> , Java'da "her şeye güvenen" <code>TrustManager</code>	Her sertifika kabul edilir
Ana makine adı denetiminin kapatılması	<code>SSL_set1_host</code> yok; Java <code>HostnameVerifier</code> her zaman <code>true</code>	Başka sitenin geçerli sertifikası kabul edilir
Sertifika hatasında "devam et"	Mobil web görünümünde hata olayında işlemi sürdürmek	Araya giren saldırgan fark edilmez
Hata ayıklama bayrağının sürüme sızması	<code>#ifdef DEBUG</code> yerine çalışma anı bayrağı	Saldırgan bayrağı açar
Yalnız yedeksiz uç sertifika pini	Mobil uygulamalar	Sertifika yenilenince uygulama çalışmaz; ekip pinlemeyi tamamen kaldırır
Dönüş değerlerinin denetlenmemesi	Her yerde	El sıkışma başarısız olsa bile veri gönderilir
TLS'in tek koruma sanılması	Mimari	Sunucuda ya da istemcide açık duran veri korunmaz



'Nasılrsa TLS var' yanılgısı

TLS yalnız **yoldaki** veriyi korur. Veri sunucuya ulaştığında, günlüğe yazıldığında ya da istemcide bir dosyaya kaydedildiğinde TLS'in koruması biter. Bu yüzden hassas veri için TLS'in üstüne **mesaj düzeyinde** şifreleme (güvenlik kabuğu) eklenir ve veri beklemede de şifrelenir.

Kendi bağlantınızı test etmek

```
# Sürüm, şifre takımı ve sertifika zincirini göster; doğrulama hatasında dur
openssl s_client -connect server.example:443 -servername server.example \
    -verify_return_error -brief </dev/null

# TLS 1.1'i dene: sunucu reddetmeli
openssl s_client -connect server.example:443 -tls1_1 </dev/null
```

1. Kendi imzalı bir sertifika sunan yerel bir sunucuya bağlanın: uygulama **reddetmeli** (Demo 6).
2. Geçerli ama başka bir ad için alınmış sertifika sunun: uygulama **reddetmeli**.
3. Test cihazının güven deposuna kendi CA'nızı ekleyip araya bir proxy koyun: sabitleme yapan uygulama yine **reddetmeli**.
4. Pin hatasının raporlandığını ve kullanıcıya anlaşılır bir ileti gösterildiğini doğrulayın.

? Değerlendirici nasıl test eder?

Test cihazına kendi kök sertifikasını kurar ve trafiği bir araya-girme proxy'sinden geçirir. Uygulama bu durumda bağlanabiliyorsa sabitleme yok ya da etkisizdir. Sonra RASP ve gizleme katmanlarını aşarak sabitleme denetimini çalışma anında devre dışı bırakmayı dener (6. hafta); bu denemenin ne kadar sürdüğü, korumanın gücünü gösterir. Ayrıca sunucu tarafında TLS sürümlerini, şifre takımlarını ve sertifika zincirini tarar.

✎ Sahada nasıl uygulanır?

Mobil ödeme kütüphanelerinde aktarımdaki veri tipik olarak iki dış kabukla korunur: TLS + açık anahtar karşılaştırmalı sertifika sabitleme + sunucu sertifika zincirinin doğrulanması (en dış kabuk), onun içinde de oturum anahtarıyla mesaj düzeyinde şifreleme ve MAC. Sunucunun adresi, sertifika ana makine adı ve sabitlenen değerler kütüphaneye üst uygulama tarafından parametre olarak verildiğinde, bu parametrelerin korunması **üst uygulamanın sorumluluğu** olarak kılavuzda açıkça yazılır; sorumluluğun kimde olduğu belirsiz bırakılmaz.

11. Beklemede veri: dosya ve veritabanı şifreleme

Diskteki dosya ya da veritabanı **çalınabilir** (kayıp telefon, çalınan yedek, ele geçirilen sunucu). Beklemede veriyi korumanın iki düzeyi vardır:

Beklemede veri: iki düzey, iki tehdit modeli

hangi tehdide karşı koruduğunuzu önce yazın

TÜM DİSK / DOSYA

LUKS, BitLocker, XTS-AES
cihaz kapalıyken korur
açıkken her şey okunur
yedek çalınrsa yeter

ALAN / SÜTUN

yalnız hassas alan AEAD ile
uygulama açıkken bile korur
anahtar yönetimi gerekir
arama/indeks zorlaşır

Kayıp telefon → disk şifreleme yeter. Ele geçirilen sunucu → alan şifreleme şart.

- **Tüm disk/dosya şifreleme:** İşletim sistemi düzeyinde (LUKS, BitLocker, XTS-AES) ya da uygulama düzeyinde tüm dosyayı AEAD ile şifreleme (bkz. Demo 1). Kitap **tarif 5.15** disk şifrelemeyi işler; sabit IV tuzağına dikkat çeker.
- **Alan/sütun şifreleme:** Yalnız **hassas alanları** (kart no, TC kimlik, sağlık verisi) uygulama katmanında şifreleyip veritabanına şifreli yazma. Veritabanı çalınsa bile o alanlar okunamaz.

Veri maskeleye ayrı bir tekniktir: veriyi geri döndürülemez ya da kısmi biçimde gösterme. Şifreleme **değildir** (anahtarla geri açılmaz); amacı, verinin **gereksiz yere maruz kalmasını** azaltmaktır. Başlıca biçimleri:

- **Kısmi maskeleye:** yalnız bir kısmını gösterme, ör. kart no **** * 4242 , e-posta a***@ornek.com . Kullanıcı arayüzünde ve müşteri hizmetleri ekranlarında kullanılır.
- **Loglama maskesi:** loglara hassas alan **hiç** yazılmaz ya da maskeli yazılır. (Hafta 1'de gördüğümüz "sırrı loglama" hatasının veri tarafı.)
- **Tokenizasyon:** gerçek değeri anlamsız bir "token" ile değiştirip gerçek değeri ayrı, güvenli bir kasada tutma. Ödeme sistemlerinde kart numarası yerine token dolaşır; asıl numara yalnız gerekince kasadan çözülür.
- **Test verisi maskeleye:** üretim verisini test/geliştirme ortamına kopyalarken gerçek kişisel veriyi sahte ama gerçekçi değerlerle değiştirme.

Maskeleye şifrelemenin **yerine geçmez**; onu **tamamlar**. Veri diskte AEAD ile şifreli durur (gizlilik), ekrana/loga giderken maskelenir (maruz kalmayı azaltma). İkisi farklı tehditlere karşıdır: biri dosyanın çalınmasına, diğeri "yanlış kişinin ekranda/logda görmesine".

Demo 7 — SQLite'ta hassas alanı AES-GCM ile şifreleme

Demo 7 · `code/week-03/07-sqlite-field` · beklemede veri, alan şifreleme

Program bir SQLite veritabanına iki müşteri kaydı ekler: `name` alanı **açık**, `card_encrypted` alanı AES-256-GCM ile şifreli BLOB. Anahtar koda gömülü değil; `SIM_KEY` ortam değişkeninden gelir. Saldırgan DB'yi çalıp açtığında kart alanı yalnız şifreli bayt yığınıdır. SQLite Linux'ta `sqlite3`, Windows'ta Windows SDK'nın yerleşik **winsqlite3** kütüphanesidir; kript o ortak başlıktan.

.\demo.ps1 — gerçek çıktı (Windows)

```
STEP 1 - Set up the database (the sensitive field is encrypted)
Database created: output\customers.db (2 records)
STEP 2 - An attacker stole the raw DB file and is searching it:
'Jane' (name) in the raw DB      : FOUND (the name field is plain)
'4242-4242' (card) in the raw DB : NOT THERE - encrypted
STEP 3 - The application reads with the correct key (decrypt and verify):
1 | Jane Doe      | 4242-4242-4242-4242
2 | John Smith    | 5555-4444-3333-2222
STEP 4 - A read attempt with the WRONG key (the GCM tag will not match):
1 | Jane Doe      | (COULD NOT DECRYPT - wrong key?)
2 | John Smith    | (COULD NOT DECRYPT - wrong key?)
```

Saldırgan `sqlite3` ile DB'yi açsa bile `name` alanını görür ama `card_encrypted` alanı yalnız şifreli bayttır. Doğru anahtarla uygulama açıp doğrular; **yanlış** anahtarla GCM etiketi tutmadığı için çözme reddedilir (sessizce çöp döndürmez).

Windows demosu (`.\demo.ps1`) `sqlite3` aracı yerine ham DB dosyasında arama yaparak aynı noktayı gösterir: `Jane` (açık ad) bulunur, `4242-4242` (kart) **bulunmaz** çünkü şifrelidir.

Kural

Hassas alanları veritabanına yazmadan **önce** uygulama katmanında AEAD ile şifreleyin. Anahtarı koda gömmeyin; cihaza/kullanıcıya bağlı bir KDF'den türetin (Bölüm 13). Loglara ve ekrana giden değerleri **maskeleyin**. Beklemede şifreleme, kullanımda korumanın (Demo 8) yerine geçmez; ikisi birlikte gerekir.

Sahada nasıl uygulanır? / Değerlendirici testi

Örnek mimaride yerel veritabanı SQLite'tır; kart profili, işlem sayacı ve anahtar listesi şifreli sütunlarda tutulur, anahtar cihaza bağlıdır. Değerlendirici DB dosyasını çıkarır, açık hassas veri arar ve bulursa bulgu yazar; ayrıca anahtarın nereden geldiğini (koda gömülü mü, cihaza bağlı mı) inceler.

Sık hatalar ve kontrol listesi

- ☒ Hassas alan DB'ye yazılmadan **önce** mi şifreleniyor?
- ☒ Anahtar koda gömülü değil, cihaza/kullanıcıya bağlı bir kaynaktan mı geliyor?
- ☒ Yanlış anahtarla çözme sessizce çöp döndürmüyor, reddediyor mu (AEAD)?
- ☒ Loglara/ekrana giden hassas alanlar maskeleniyor mu?
- ☒ Şifreli alan başka bir kayda taşınmıyor mu (AAD ile bağlama)?

Aşağıdaki animasyonda iki müşteri satırının `card_encrypted` alanını doğru ve yanlış anahtarla okuma denemesini izleyin: doğru anahtarla her satır çözülür, yanlış anahtarla HER satır reddedilir.

ad	kart_şifreli (kilitli)
Jane Doe	88CE8E73...
John Smith	89C98F74...

1. Veritabanı satırları: name AÇIK, card_encrypted bir [nonce|şifreli metin|etiket] BLOB.

DOĞRU anahtarla okunuyor

ad	kart_şifreli (kilitli)
Jane Doe	88CE8E73...
John Smith	89C98F74...

2. SIM_KEY ortam değişkeni: gerçek anahtar.

DOĞRU anahtarla okunuyor	
ad	kart_şifreli (kilitli)
Jane Doe	4242-4242-4242-4242
John Smith	5555-4444-3333-2222

3. gcm_unpack(): her satırda yeniden hesaplanan etiket = saklanan etiket — DOĞRULANDI, kart numaraları çözüldü.

Örnek seçiciden **gerçek demonun kayıtları**, **doğru anahtar** (uyar) ile **farklı kayıtlar** (zor) ve uç durumlardaki **yanlış anahtar** ile ****çok kısa kart alanları****nı deneyin — ya da kendi ad:kart çiftlerinizi yazın.

12. Veri maskeleyme teknikleri: veriyi göstermeden kullanmak

Şifreleme, veriyi **anahtarı olmayan** herkesten saklar. Ama birçok durumda veriyi **anahtarı olan** sistemlerin ve kişilerin bile tam olarak görmesi gerekmez: çağrı merkezi çalışanı müşterinin kart numarasının tamamını değil son dört hanesini

görmelidir; test ortamındaki veritabanı gerçek kişilerin kimlik numaralarını içermemelidir; günlük dosyası parolayı hiç görmemelidir. **Veri maskeleye**, verinin kullanım amacına yetecek kadarını gösterip gerisini gizleme teknikleri ailesidir. Kişisel verilerin korunması mevzuatı (KVKK, GDPR) ve ödeme kartı standardı (PCI DSS), bu teknikleri açıkça ister.

Dört temel teknik

Teknik	Ne yapar?	Geri döndürülebilir mi?	Tipik kullanım
Maskeleye (gösterimde)	Değerin bir kısmını * ile değiştirir	Hayır (gösterilen kopya için)	Ekran, fiş, günlük
Tokenizasyon	Değeri anlamsız bir belirteçle değiştirir; gerçek değer ayrı ve korunan bir kasada durur	Evet, yalnız kasaya erişimi olan için	Ödeme sistemleri, kart saklama
Takma adlandırma (pseudonymization)	Değeri anahtarlı bir fonksiyonla sabit bir takma ada çevirir	Anahtarı olan için eşleştirilebilir	Analiz, raporlama, test verisi
Anonimleştirme	Kişiyi bağı kalıcı olarak koparır (genelleme, gürültü, silme)	Hayır	Açık veri, istatistik

Aralarındaki fark hukuken de önemlidir: takma adlandırılmış veri hâlâ **kişisel veridir** (anahtarı olan kişiyi bulabilir); gerçekten anonimleştirilmiş veri ise kişisel veri sayılmaz. Birçok "anonim" veri kümesinin, başka verilerle birleştirilerek kişilere geri bağlanabildiği gösterilmiştir. Bu yüzden "anonimleştirdik" demek, "takma ad verdik" demekten çok daha güçlü bir iddiadır.

1. Gösterimde maskeleye

Kart numarası (PAN) için PCI DSS kuralı nettir: numara görüntülendiğinde en fazla ilk altı (ya da standardın yeni sürümlerinde kart türüne göre sekiz) ve son dört hane gösterilebilir; işi gereği tam numarayı görmesi gereken roller ayrıca tanımlanır.

Kart numarasını görüntüleme için maskeleye (son 4 hane)

```
#include <string.h>

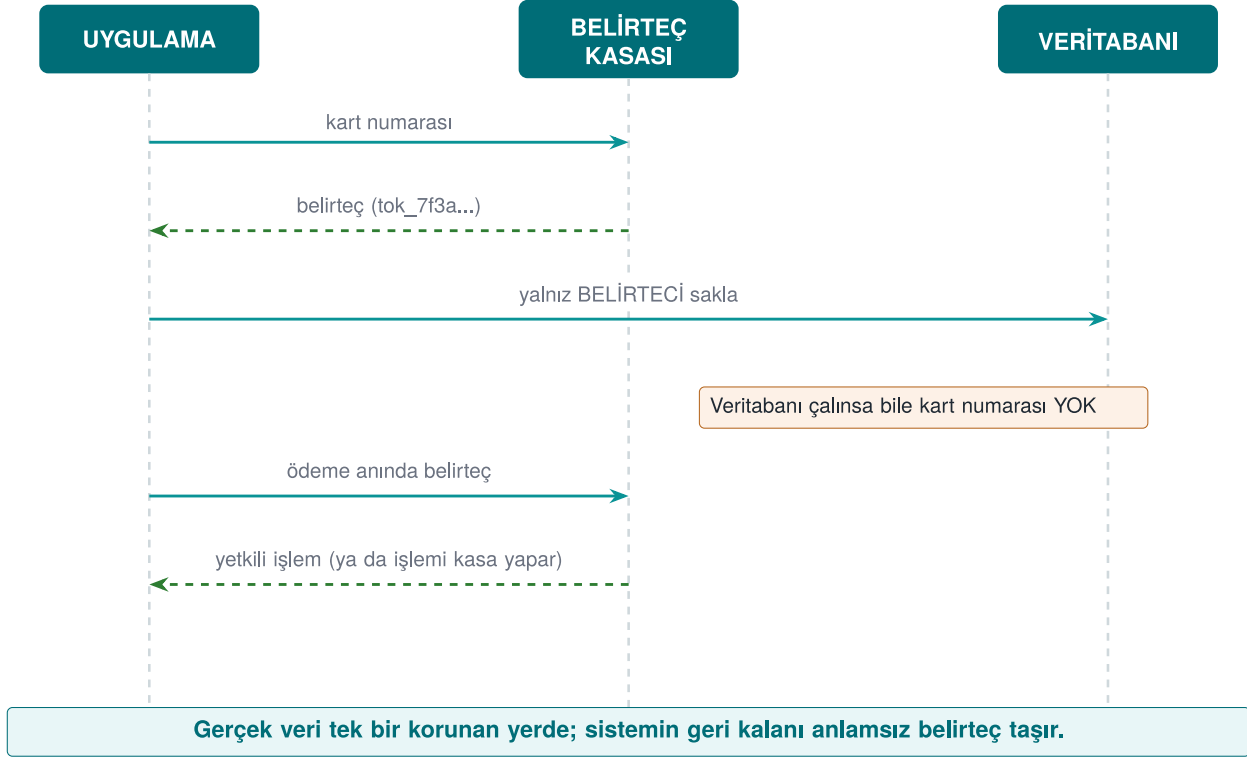
/* "1234567812345678" -> "*****5678"; out must be at least n+1 bytes */
void mask_pan(const char *pan, char *out, size_t out_size)
{
    size_t n = strlen(pan, 19);
    if (out_size < n + 1) { if (out_size) out[0] = '\0'; return; }
    for (size_t i = 0; i < n; i++)
        out[i] = (i + 4 < n) ? '*' : pan[i];
    out[n] = '\0';
}
```

Maskeleye **kaynağa yakın** yapılmalıdır: tam değer sunucudan istemciye gönderilip ekranda maskelenirse, ağ trafiğini ya da istemci belleğini inceleyen biri tam değeri görür. Doğrusu, istemciye zaten maskelenmiş değeri göndermektir.

2. Tokenizasyon

Tokenizasyon: gerçek değeri kasada tut

PCI DSS denetim kapsamı kasaya daralır



Tokenizasyonun gücü, gerçek verinin **tek bir küçük ve çok iyi korunan** sistemde toplanmasıdır. Uygulamanın geri kalanı, veritabanları, raporlar ve günlükler yalnız belirteci görür; bunlar ele geçirilse bile kart verisi sızmaz. Bu, [1. haftadaki](#) "tehdidi aktarma" yanıtının teknik karşılığıdır: kart verisini hiç saklamayıp ödeme sağlayıcısının belirteç hizmetini kullanmak, uygulamanın PCI DSS kapsamını büyük ölçüde küçültür. Mobil ödemede telefona indirilen kart bilgisi de gerçek kart numarası değil, cihaza özgü bir **belirteç numarasıdır**.

Belirtecini **biçimi korunabilir**: 16 haneli bir kart numarası yerine yine 16 haneli ama anlamsız bir sayı. Bu, eski sistemlerin (alan uzunluğu, doğrulama kuralları) değiştirilmeden kullanılmasını sağlar. Biçimi koruyan şifreleme (format-preserving encryption, NIST SP 800-38G'deki FF1 algoritması) aynı işi bir anahtar ve kasa olmadan yapar.

3. Takma adlandırma: anahtarlı özet

Analiz ya da test için "aynı kişinin kayıtlarını bir araya getirebilmek ama kim olduğunu bilmemek" gerekir. Kimlik numarasını düz bir özetten (SHA-256) geçirmek **yetmez**: 11 haneli bir kimlik numarasının olası değer sayısı küçüktür ve saldırgan hepsinin özetini hesaplayıp karşılaştırabilir. Doğrusu, **gizli bir anahtarla HMAC** kullanmaktır:

Anahtarlı takma ad: HMAC-SHA256(key, id_number)

```

/* code/common/cen429_crypto.h */
unsigned char pseudonym[32];
crypto_hmac_sha256(pseudonym_key, 32, id_number, strlen(id_number), pseudonym);
/* the data set stores the first 16 bytes of hex(pseudonym) instead of id_number */

```

Anahtar veri kümesinden **ayrı** tutulur. Anahtarı olmayan için takma addan kimlik numarasına dönmek, olası bütün kimlik numaralarını denemek anlamına gelir ve anahtar bilinmeden bu mümkün değildir. Anahtar kaybolursa ya da imha edilirse, veri fiilen anonimleşir.

4. Test ve geliştirme verisi

Gerçek üretim verisinin test ortamına kopyalanması, en sık görülen veri sızıntısı nedenlerinden biridir: test ortamları daha az korunur, daha çok kişi erişir. Test verisi için sırasıyla tercih edilecekler:

1. **Tamamen sentetik veri:** Gerçek kayıtlardan hiç türetilmemiş, üretilmiş veri. En güvenli seçenek.
2. **Statik maskeleye:** Üretim verisinin bir kopyası alınır, kopyadaki hassas alanlar kalıcı olarak değiştirilir (takma ad, karıştırma, genelleme) ve ancak ondan sonra test ortamına taşınır.
3. **Dinamik maskeleye:** Veri değiştirilmez; veritabanı sorgu sonucunu kullanıcının rolüne göre maskeleyerek döndürür. Kullanışlıdır ama veri tabanında tam haliyle durmaya devam eder.

5. Günlüklerde ve hata iletilerinde maskeleye

İkinci haftada denetim kaydının "asla kaydetme" listesini görmüştük: parola, PIN, belirteç, anahtar, tam kart numarası. Kural bir şeydir, onu **garanti etmek** başka. İki yaklaşım birlikte kullanılır:

- **Kaynakta önleme:** Hassas veriler özel bir türde tutulur ve bu türün yazdırma fonksiyonu her zaman maskeli çıktı verir. Programcı yanlışlıkla günlüğe yazsa bile tam değer çıkmaz.
- **Çıkışta süzme:** Günlük kütüphanesinin son adımında, kart numarasına benzeyen (13–19 haneli, Luhn denetimini geçen) diziler ve `password=`, `token=` gibi anahtarlardan sonra gelen değerler otomatik olarak maskelenir.

Günlük satırında 'password=' ve 'pin=' değerlerini maskeleye (basit süzgeç)

```
#include <string.h>
#include <ctype.h>

void log_filter(char *line)
{
    static const char *keys[] = { "password=", "pin=", "token=" };
    for (size_t k = 0; k < sizeof keys / sizeof keys[0]; k++) {
        char *p = line;
        while ((p = strstr(p, keys[k])) != NULL) {
            p += strlen(keys[k]);
            while (*p && !isspace((unsigned char)*p))
                *p++ = '*';
            /* mask the value in place */
        }
    }
}
```



Süzgeç ikinci savunma hattıdır

Çıkışta süzme, programcının hatasını yakalamak için bir **emniyet ağıdır**; asıl önlem hassas veriyi günlüğe hiç göndermemektir. Süzgeç her biçimi tanıyamaz (ör. JSON içinde `"parola": "..."`) ve süzülmeden önce veri zaten bellekte ve belki bir tamponda durmuştur. Sahada sürüm derlemelerinde günlük kodunun tamamen kaldırılması bu yüzden tercih edilir (1. hafta ödünleşim kaydı).

Hangi tekniği ne zaman?

Durum	Önerilen teknik
Çağrı merkezi ekranında kart numarası	Gösterimde maskeleyme (son 4), tam değer istemciye hiç gelmez
Kart numarasını tekrarlı ödemeler için saklamak	Tokenizasyon (tercihen ödeme sağlayıcısında)
Aynı müşterinin davranışını analiz etmek	Anahtarlı takma ad (HMAC)
Test veritabanı	Sentetik veri; olmazsa statik maskeleyme
Günlük dosyası	Kaynakta önleme + çıkışta süzme
Açık veri olarak paylaşmak	Anonimleştirme (genelleme, toplulaştırma) ve yeniden tanımlama riskinin değerlendirilmesi

? Değerlendirici nasıl test eder?

Uygulamanın ekranlarında, istemciye giden ağ yanıtlarında, günlük dosyalarında, çökme raporlarında ve yedeklerde tam kart numarası, kimlik numarası ya da parola arar. Maskeleymenin istemcide mi (zayıf) yoksa sunucuda mı yapıldığına bakar. Takma adlandırmada anahtarsız bir özet kullanılıp kullanılmadığını denetler ve test ortamında gerçek veri bulunup bulunmadığını sorar.

✎ Sahada nasıl uygulanır?

Ödeme kütüphanelerinde kullanıcı arayüzüne kartın yalnız son dört hanesi ve kart türü gibi maskeli bilgiler verilir; gerçek kart numarası yerine cihaza özgü belirteç numarası kullanılır. Kütüphanenin kılavuzunda "hata kodları saldırganı ipucu vermemeli" ve "hassas veri açık olarak günlüğe yazılmamalı" gereksinimlerinin nasıl karşılandığı (ör. bütün günlüklerin derleme anında kaldırılması) ayrıca belgelenir.

Aşağıdaki animasyonda aynı değer üç farklı teknikle nasıl işlendiğini izleyin: kısmi maskeleyme (yalnız ilk/son karakterler kalır), tokenizasyon (anlamsız bir simge + korumalı kasa) ve takma adlandırma (anahtarlı özet).

değer = 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 1. Gerçek değer, 16 karakter: "42424242424242". Anahtarlı olan sistemler bile bunu HER ZAMAN görmek zorunda değil.	değer = 4 2 4 2 4 2 4 2 4 2 4 2 4 2 4 2 maskeli = 4 2 4 2 * * * * * * * * 4 2 4 2 2. mask_pan(): yalnız ilk ve son 4 karakter kalır, ortası * ile değiştirilir — GÖRÜNTÜLENEN kopyadan GERİ DÖNÜLEMEZ.
--	---

Örnek seçiciden **kısmi maskeleyme** (uyar) ile **tokenizasyon** (zor) ve uç durumlardaki **takma adlandırma** ile **çok kısa bir değer**i deneyin — ya da kendi değer|teknik çiftinizi yazın.

13. Kullanımda veri: bellekte güvenli silme ve cihaz bağlama

Bir sır **kullanılmak için** eninde sonunda bellekte açılır. O anda risk altındadır: bellek dökümü, hata ayıklayıcı, takas alanı (swap), aynı belleği sonra kullanan başka kod. [Hafta 1, §17](#)'de `explicit_bzero` ile "ölü yazma giderme"yi ve sırrı bellekten silmeyi (Tarif 13.2), ayrıca [belleğin diske yazılmasını önlemeyi](#) (Tarif 13.3) görmüştük; bu hafta **kullanımda veri** için döküm engelleme, bellek kilitleme ve **cihaz bağlama** katmanlarını ekliyoruz.

Kullanımda veri: açılma penceresini daraltan katmanlar

sır kullanılmak için bir an açılır; o an yönetilir

Kısa ömür	anahtar işin hemen öncesinde aç, bitince sil (<code>explicit_bzero</code>)
Bellek kilidi	<code>mlock</code> / <code>VirtualLock</code> — takas alanına yazılmasını engeller
Döküm kapatma	<code>core dump</code> ve hata raporu devre dışı
Cihaz bağlama	anahtar cihazın kimliğine bağlanır; kopyalanırsa açılmaz
Donanım (TEE/SE)	anahtar hiç uygulamaya girmez — en güçlü katman

Hiçbiri pencereyi sıfırlamaz; amaç saldırganın o ana yetişmesini pahalı kılmaktır.

Katman	Amaç	Linux	Windows
Döküm engelleme	Sır core/crash dosyasına düşmesin	<code>setrlimit(RLIMIT_CORE, 0)</code>	<code>SetErrorMode</code>
Bellek kilitleme	Sayfa takas alanına yazılmasın	<code>mlock</code>	<code>VirtualLock</code>
Güvenli silme	İş bitince üzerine yaz (kaldırılamaz)	<code>OPENSSL_cleanse</code>	<code>SecureZeroMemory</code>
Kısa ömür	Sırrı en kısa süre tut, kopyalama	—	—

Demo 8 — mlock + güvenli silme



Demo 8 · [code/week-03/08-memory-wipe](#) · kullanımda veri (kitap tarif 13.2–13.3)

Program üç katmanı kurar: dökümü kapatır, belleği kilitlet, işi bitince `crypto_wipe` ile siler; silme sonrası tamponun gerçekten sıfırlandığını doğrular. (Hafta 1'deki gcore gösterimini tekrarlamaz; yeni katmanlar ekler.)

`sh demo.sh` — gerçek çıktı (WSL)

```

Layer 1 - crash dump turned off (RLIMIT_CORE=0): OK
Layer 2 - mlock blocked writing to swap: OK
Secret in use: length = 23, non-zero bytes = 23
Layer 3 - non-zero bytes after secure wipe = 0
==> the secret was cleared from memory.
-----
OPENSSL_cleanse call count -> 2

```

Windows'ta (`.\demo.ps1`) aynı üç katman `SetErrorMode + VirtualLock + SecureZeroMemory` ile kurulur; çıktı da "TAMAM"dır. `memset`'in aksine `OPENSSL_cleanse / SecureZeroMemory` çağrısı derleyici tarafından "ölü yazma" olarak kaldırılamaz — makine kodunda gerçek bir çağrı olarak kalır (demo bunu `objdump` ile sayar).

13.1 Cihaz bağlama

Kullanımda korumanın bir parçası da anahtarın **belirli bir cihaza/sürüme bağlanmasıdır**. Anahtarı, cihazın parmak izinden (üretici, model, cihaz kimliği, sürüm dizesi) türetilen bir değerle sararsak, dosyalar başka bir cihaza kopyalansa bile orada **açılmaz**. Doğru yol: XOR ya da tuzsuz özet değil, **HKDF ile tuzlanmış türetme** ve AEAD ile sarma. Bölüm 14'deki güvenlik kabuğunun en iç katmanı tam olarak budur; bunu [Hafta 6](#)'da RASP ile birleştireceğiz.

✓ Kural

Sırrı **en son anda** aç, **en kısa süre** tut, **kopyalama**, işin bitince derleyicinin kaldıramayacağı bir işlevle **sil**. Mümkünse belleği kilitler ve dökümü kapat. Depolama anahtarını **cihaza ve sürüme bağla** (HKDF + AEAD).

🔧 Sahada nasıl uygulanır?

Mobil ödeme kütüphanesinde tek kullanımlık ödeme anahtarı yalnız ödeme anında, yalnız yerel (native) katmanda açılır ve kullanımdan hemen sonra rastgele veriyle üzerine yazılarak silinir. Depolama anahtarı cihaz parmak izine bağlıdır; telefon klonlansa bile başka cihazda çözülemez (Demo 9, Saldırı 2). Java tarafında hassas veri, silinemeyen `String` yerine silinebilen `byte[]` dizilerinde tutulur.

❓ Değerlendirici nasıl test eder?

Değerlendirici ödeme sırasında ve sonrasında bellek dökümü alıp anahtarı arar; ayrıca dosyaları başka bir cihaza kopyalayıp orada açılıp açılmadığını dener. Güvenli silmenin gerçekten yapıldığını (ve derleyicinin kaldırmadığını) binary dosyanın makine kodundan doğrular.

⚠️ Sık hatalar ve kontrol listesi

- ✓ Sırrı `memset` ile değil, kaldırılamaz bir işlevle mi siliniyor?
- ✓ Sırrın gereksiz kopyaları (loglar, geçici tamponlar, `String`) var mı?
- ✓ Depolama anahtarı cihaza/sürüme bağlı mı?
- ✓ Çökme dökümü kapalı ve hassas bellek kilitli mi?

Aşağıdaki animasyonda üç katmanın (döküm engelleme, bellek kilitleme, güvenli silme) bir sırrı bellekte adım adım nasıl koruduğunu ve silme sonrası her baytın gerçekten sıfırlandığını izleyin.

1. `char secret[BUF_LEN]; memset(secret, 0, sizeof(secret));` — arabellek her zaman ÖNCE sıfırlanır.

2. Katman 1 — `setrlimit(RLIMIT_CORE, 0)` (Linux): sır artık bir çökme dosyasına asla düşemez.

3. `strcpy(secret, "temp-password1")` — 14 bayt yazıldı; kalan 2 bayt hâlâ sıfır.

4. Katman 2 — `mlock(secret, sizeof(secret))` (Linux): bu sayfa artık takas alanına (swap) hiç yazılmaz.

5. Sır kullanımda: `strlen(secret) = 14`, `nonzero_bytes(secret) = 14`.

6. `crypto_wipe(secret, sizeof(secret))` — her bayt 0x00 ile üzerine yazılır (derleyici bu çağırımı "ölü yazma" olarak KALDIRAMAZ).

7. `nonzero_bytes(secret) = 0` — sır bellekten tamamen temizlendi.

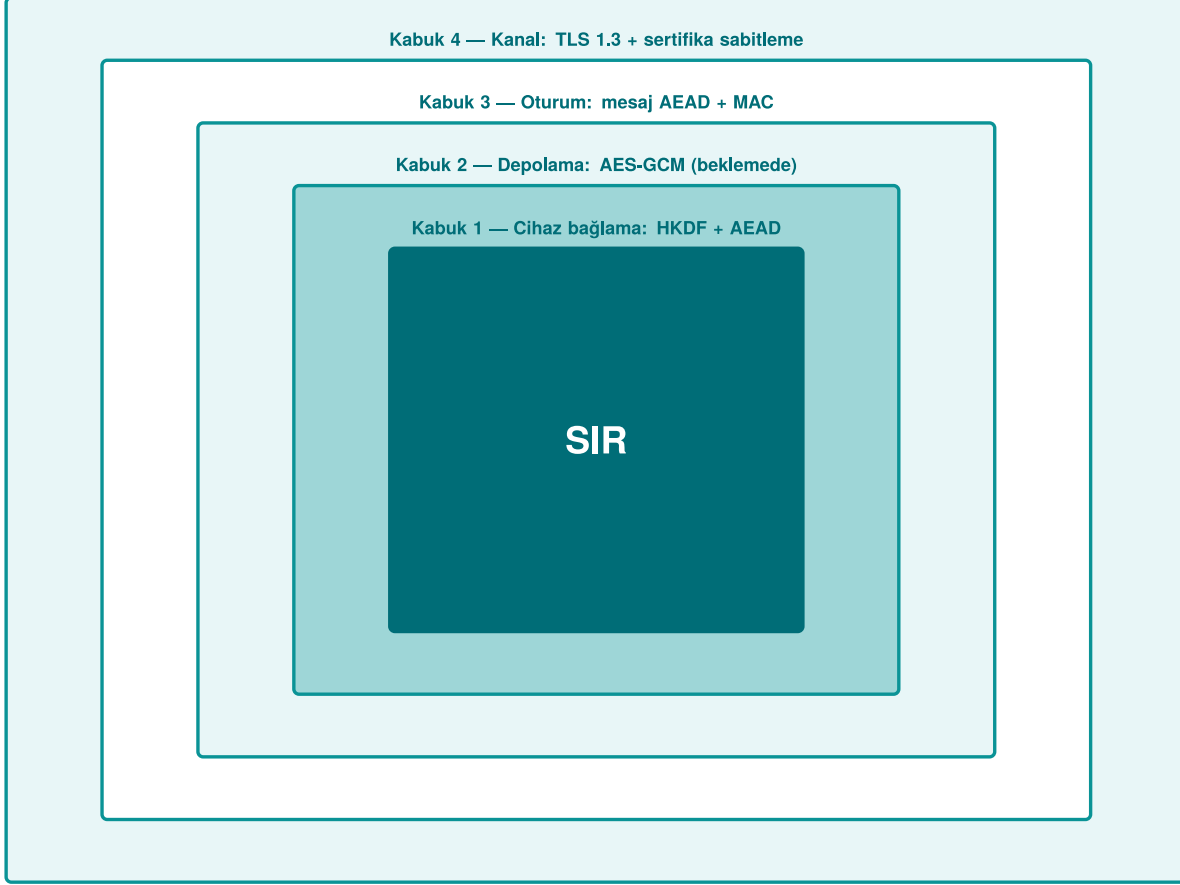
Örnek seçiciden **orta uzunlukta bir sır** (uyar) ile **demonun gerçek sırrına yakın, uzun bir sır** (zor) ve uç durumlardaki **tek karakterlik sır** ile ****özel karakterler****i deneyin — ya da kendi sır metninizi yazın.

14. Güvenlik kabukları: derinlemesine savunmanın somut hâli

Şimdi bu haftanın bütün parçalarını tek bir fikirde birleştiriyoruz. Hassas bir varlığı **tek** bir korumaya bırakmayız; onu, yaşam döngüsünün her aşamasına karşılık gelen **iç içe kabuklarla** sararız:

Güvenlik kabukları: derinlemesine savunma

sırra ulaşmak için dört kabuğu da kırmak gerekir



Bir kabuk delinse bile sır hâlâ diğerlerinin altındadır.

Saldırganın sırra ulaşması için **dört kabuğu da** sırayla açması gerekir: TLS'i kır, mesaj şifrelemesini kır, depolama şifrelemesini kır **ve** doğru cihazda ol. Bu, "aktarımda yalnız TLS'e güvenmeme", "beklemede alan şifreleme" ve "kullanımda cihaz bağlama" ilkelerinin tek tabloda birleşmiş hâlidir.

Demo 9 — Bir anahtarı dört kabukla sarıp açmak

Demo 9 · [code/week-03/09-security-layers](#) · derinlemesine savunma

Program 16 baytlık bir sırrı sırayla dört AES-GCM kabuğuyla sarar (en iç: cihaza bağlı HKDF anahtarı), sonra ters sırada açar. İki saldırı: (1) paketin bir biti kurcalanırsa en dış kabuk açılmaz; (2) paket başka bir cihaza (farklı parmak izi) kopyalanırsa dış kabuklar açılabilir **en iç cihaz kabuğu** açılmaz.

security_layers.exe — gerçek çıktı

```

WRAPPING (outward): secret -> K1 -> K2 -> K3 -> K4
  Layer 1 (device binding)      : 44 bytes
  Layer 4 (channel/TLS-like)    : 128 bytes <- transmitted packet
UNWRAPPING (inward): K4 -> K3 -> K2 -> K1 -> secret
  Layer 4 opened: OK
  Layer 3 opened: OK
  Layer 2 opened: OK
  Layer 1 opened: OK
  Recovered SECRET : deadbeef0123456789abcdeffedcba98 -> secret CORRECT
=====
ATTACK 1 - if one bit of the packet is tampered with (Layer 4):
  Layer 4 opened: REJECTED (the GCM tag did not match)
=====
ATTACK 2 - if the packet is copied to ANOTHER device (wrong fingerprint):
  Outer 3 layers opened: OK
  Layer 1 (device binding) opened on the OTHER device: REJECTED
  ^ Even with the keys copied, the secret CANNOT be opened on another device.

```

Normal akışta dört kabuk sırayla açıldı ve sır doğru geri geldi. Tek bir bit kurcalanınca en dış GCM etiketi tuttu ve o katmanda reddedildi. Paket başka bir cihaza taşındığında dış üç kabuk (anahtarları taşınabilir) açıldı ama **cihaza bağlı en iç kabuk** açılmadı — sır ele geçmedi. Gerçek ürünlerde bu en iç kabuk çoğu zaman **whitebox krypto** olur ([Hafta 11](#)).

Bu kabuk yaklaşımı dönemin geri kalanına köprüdür: native kod sağlamlaştırma (hardening) ve RASP ([Hafta 4](#), 6, 9) kabukları **çalışma zamanında** korur; krypto ve anahtar hiyerarşisi ([Hafta 10](#)) kabukların anahtarlarını doğru üretir ve yönetir; whitebox ([Hafta 11](#)) en iç kabuğu güçlendirir; sertifikasyon ([Hafta 12–13](#)) bütün bu katmanların **kanıtını** ister. Yani bu haftanın veri güvenliği bölümü, güvenlik kılavuzunuzun (S5, S7) omurgasıdır ve ilerideki her hafta bir kabuğu daha somutlaştırır.

Sahadaki kabuk matrisi: bir varlık, beş aşama

Demo 9, fikri sade tutmak için dört kabuğun hepsini AES-GCM ile kurdu. Bağımsız bir laboratuvarın sertifikasyonundan geçmiş bir mobil ödeme kütüphanesinde ise kabuklar şöyle tanımlanmıştı (adlar genelleştirilmiştir):

Kabuk	İçerik
1	Ödeme anahtarının whitebox DES biçiminde tutulması: anahtar hiçbir aşamada açık halde bulunmaz
2	Cüzdan anahtarıyla whitebox AES-CBC + cihaz bağlama + uygulama bağlama
3	Beklemede: cüzdan anahtarıyla whitebox AES-CBC + SHA-256 MAC + cihaz ve sürüm bağlama . Aktarımda: yapılandırma anahtarından türetilen oturum anahtarıyla AES + SHA-256 MAC + cihaz ve sürüm bağlama
4	TLS + açık anahtar sabitleme + sunucu sertifikası denetimi
Dış	Kimlik doğrulama kodu: oturum anahtarı + cihaz ve sürüm bağlama (iki taraf birbirini doğrular)

Asıl öğretici olan, aynı varlığın yaşam döngüsündeki **her aşamada** hangi kabukların bulunduğunu gösteren matristir:

Aşama	Kabuk 1	Kabuk 2	Kabuk 3	Kabuk 4
Sunucudan kütüphaneye aktarım	✓	✓	✓ (oturum anahtarı)	✓
Yerel veritabanında saklama	✓	✓	✓ (MAC)	—
Java tarafında çözme ve native API çağırısı	✓	✓	—	—
Ödeme sırasında Java tarafı	✓	✓	—	—
Ödeme sırasında native taraf	✓	—	—	—

Matrisin okunuşu:

- Varlık dışarıdan içeriye, yani kullanım anına yaklaştıkça kabuklar birer birer açılır. Ama en içteki aşamada bile (native tarafta ödeme hesabı) **bir kabuk kalır**: anahtar, whitebox biçimiyle kullanılır ve hiçbir zaman açık hale gelmez.
- Kabuklar yalnız **hassas** varlıklar için dört katlıdır. Daha az hassas ("normal") varlıklar beklemede tek bir kabukla, aktarımda yalnız dış iki kabukla korunur. Her varlığa en güçlü korumayı uygulamak performans ve bakım maliyetini gereksiz yere artırır; varlık tablosundaki C/I sınıflandırması bu kararın dayanağıdır.
- Kullanımda kalan açıklığı (bellekteki işlem anı) kabuklar değil, **RASP ve kod sağlama** korur (4, 6 ve 9. haftalar).
- Kütüphane yerel veritabanında (SQLite) kart profillerini, sınırlı kullanım anahtarlarını, kalan anahtar sayısını, kart durumunu, işlem kayıtlarını ve cihaz parmak izini tutar. Kayıt silindiğinde **ya da bir saldırı algılandığında** bu tabloların tamamı silinir.

Bugünün gözüyle

Kabuk 1'deki DES tabanlı biçim, ödeme şemalarının o dönemki kriptogram algoritmalarına uyum için seçilmişti; CBC + ayrı MAC birleşimi de dönemin yaygın tercihiydi. Bugün yeni bir tasarımda AES tabanlı bir whitebox ve AEAD kipleri tercih edilirdi. Matris fikrinin kendisi ise zamansızdır: projenizin güvenlik kılavuzunda (S7) her hassas varlık için bu tabloyu dolduracaksınız.

Kural

Kritik bir varlığı korurken "hangi tek önlem?" diye değil, "hangi **katmanlar**?" diye sorun. Her aşamaya (taşıma, depolama, kullanım) bir kabuk koyun; kabukları **birbirinden bağımsız anahtarlarla** kurun ki birinin kırılması diğerini vermesin. En içe cihaz/sürüm bağlaması koyun.

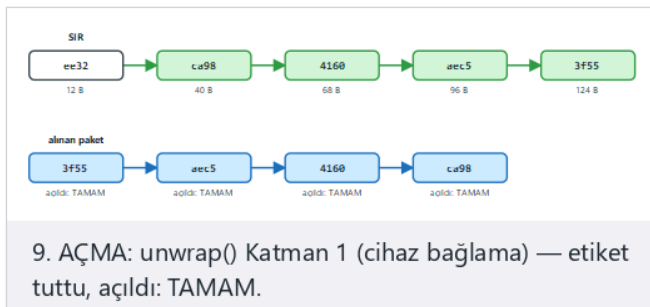
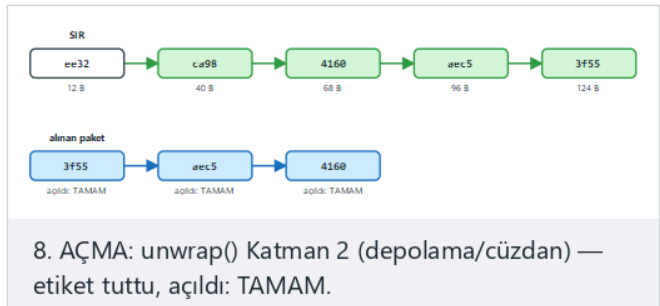
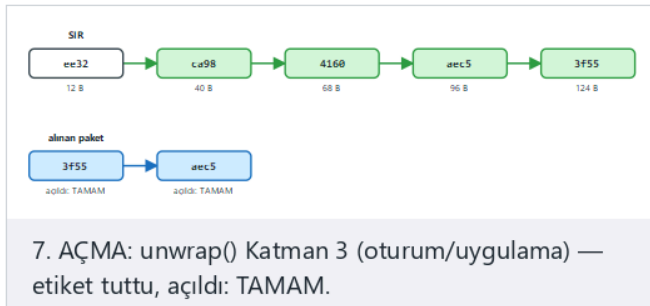
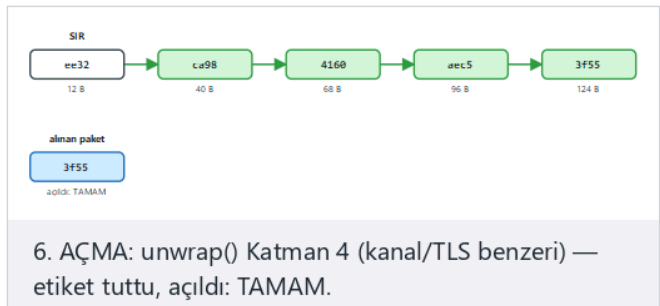
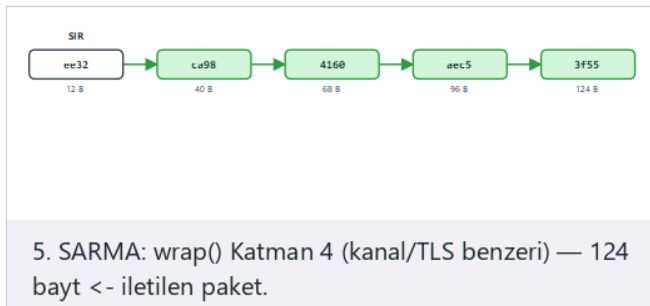
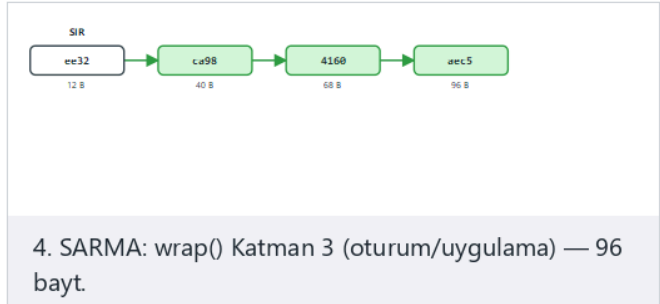
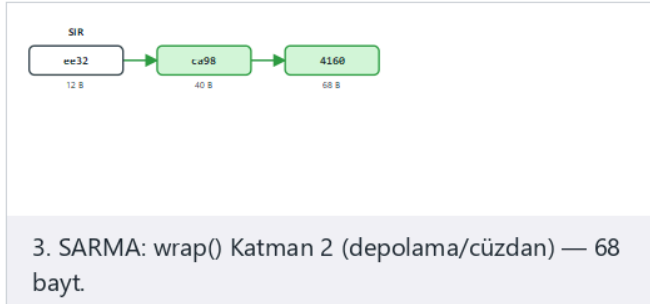
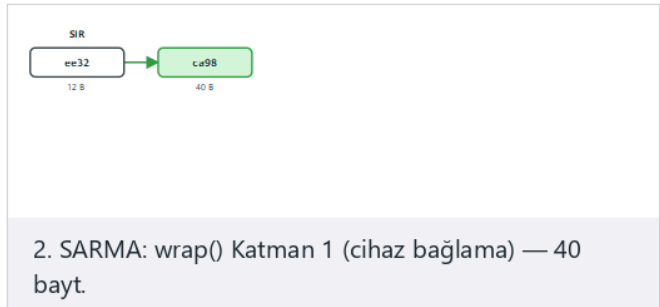
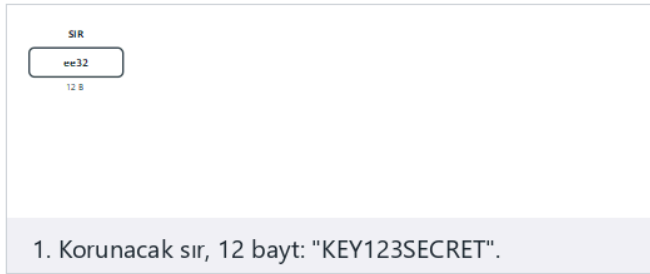
Değerlendirici nasıl test eder?

Değerlendirici güvenlik kabuğu matrisini ister ve her hassas varlık için **her aşamada** en az bir kabuğun varlığını doğrular; kabukların **doğru sırada** uygulandığını ve anahtarların **paylaşılmadığını** denetler. Sızma testinde tek bir kabuğu kırmak yetmez; korumaların **birlikte** (zincirleme) kırılması gerektiğini raporlar — bu, derinlemesine savunmanın çalıştığının kanıtıdır.

**Sık hatalar ve kontrol listesi**

- ✓ Her hassas varlık için üç aşamada da (aktarım/depolama/kullanım) kabuk var mı?
- ✓ Kabuklar bağımsız anahtarlarla mı kuruldu (biri diğerini vermiyor)?
- ✓ En iç kabuk cihaza/sürüme bağlı mı?
- ✓ "Yalnız TLS'e güvenmeme" için mesaj düzeyi bir kabuk var mı?

Aşağıdaki animasyonda bir sırrın dört kabukla nasıl sarılıp açıldığını, kurcalanan bir paketin en dış katmanda nasıl reddedildiğini ve başka bir cihaza kopyalanan paketin neden en iç katmanda takılıp kaldığını izleyin.



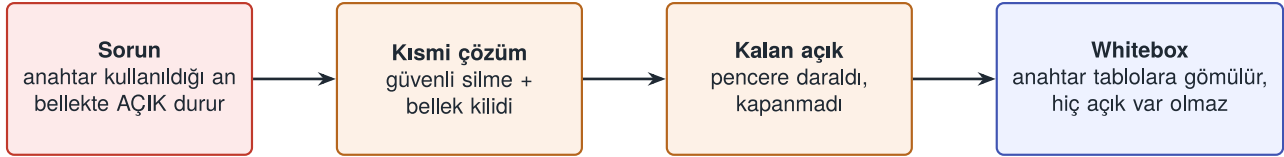
Örnek seçiciden **dört katman da başarıyla açılıyor** (uyar/zor) ile uç durumlardaki **paket kurcalandı** ve **başka cihaz** senaryolarını deneyin — ya da kendi sır|senaryo çiftinizi yazın.

15. Whitebox kriptografiye giriş (motivasyon)

Bütün bu haftanın gizli varsayımı şudur: **anahtar kullanıldığı an bellekte açık durur**. "Cihazın sahibi saldırgan" ([Hafta 1](#), beyaz kutu) senaryosunda saldırgan belleği okuyabilir, hata ayıklayıcı bağlayabilir; o anı yakalayıp anahtarı çalabilir. Güvenli silme ve bellek kilitleme bu pencereyi **daraltır** ama kapatmaz.

Whitebox kriptografiye neden ihtiyaç duyuldu?

11. haftanın kapısı burada açılıyor



Whitebox sihir değildir: anahtar çıkarmayı geciktirir, diğer katmanlarla anlam kazanır.

Whitebox kriptografi bu soruna radikal bir cevaptır: algoritmayı (ör. AES) öyle bir tabloya gömer ki, çalışırken bile **ham anahtar hiçbir zaman bellekte açık görünmez**; anahtar tabloların içine "pişirilmiştir". Böylece saldırgan belleği tamamen görse bile anahtarı doğrudan okuyamaz. Bu, Bölüm 14'daki güvenlik kabuğunun **en iç** katmanının gerçek dünyadaki hâlidir.

Bu yalnız bir giriş

Whitebox'ın nasıl kurulduğunu, saldırılarını (BGE, DFA, DCA) ve neden "yayımlanmış tüm tasarımların kıldığını" [Hafta 11](#)'de ayrıntılı işleyeceğiz. Bu hafta yalnız **neden var olduğunu** bilmeniz yeterli: bellekte açık anahtar problemi.

Whitebox sihirli değnek değildir

Whitebox kriptu tek başına yeterli değildir: saldırgan anahtarı okuyamasa bile whitebox işlevinin **kendisini** bir "kâhin" (oracle) gibi çağırıp kullanabilir ya da kod parçasını kopyalayıp (code lifting) başka yerde çalıştırabilir. Bu yüzden whitebox her zaman **diğer kabuklarla** birlikte gelir: cihaz bağlama, RASP ([Hafta 6](#)), sunucu tarafı risk denetimi ve anahtar yenileme. Yine bir **güvenlik kabuğu**.

16. Yaygın yanlış anlamalar

Dört yaygın yanlış anlama

her biri sahada gerçek bir açığa dönüşmüştür

“Şifreledim, güvendeyim.”

Şifreleme yalnız GİZLİLİK verir; bütünlük için AEAD gerekir.

“TLS var, MITM imkânsız.”

Doğrulama yapılmazsa TLS kandırılır (Demo 6).

“Kendi algoritmam daha güvenli.”

Açık tasarım ilkesi: gizlilik anahtarda, algoritmada değil.

“Anahtarı koda gömdüm, bulunmaz.”

strings ve entropi taraması saniyeler içinde bulur.

Dördünün ortak kökü aynı: güvenliği bir ÜRÜN sanmak, oysa o bir KARAR zinciridir.

✗ “Şifreledim, o hâlde güvende.”

Şifreleme yalnız **gizlilik** verir. Bütünlük için AEAD (ya da encrypt-then-MAC) gerekir; yoksa saldırgan veriyi fark edilmeden değiştirebilir. Ayrıca şifreleme, anahtar kötü yönetiliyorsa (koda gömülü, tuzsuz türetilmiş) işe yaramaz.

✗ “HTTPS/TLS var, o hâlde MITM imkânsız.”

TLS yalnız **doğru yapılandırıldığında** korur. İstemci sertifikayı doğrulamıyorsa, ana makine adını kontrol etmiyorsa ya da hataları yutuyorsa MITM hâlâ mümkündür (Demo 6). Ayrıca uç nokta ele geçmişse TLS'in içi de korunmalıdır (“yalnız TLS'e güvenmeme”).

✗ “SHA-256 ile parolayı özetledim, güvenli.”

Düz (tuzsuz, tek turlu) SHA-256 bir parola koruması **değildir**: hızlıdır, tuz yoktur, aynı parola herkeste aynı özeti verir. Argon2id/scrypt/yüksek turlu PBKDF2 + rastgele tuz gerekir.

✗ "Nonce/IV gizli olmalı."

Hayır. Nonce ve IV **gizli değildir** (şifreli metinle birlikte açık taşınır); yalnız **benzersiz/rastgele** olmaları gerekir. Gizli olan tek şey **anahtardır**.

✗ "Kendi şifreleme algoritmamı yazarsam kimse kıramaz."

Güvenlik gizli algoritmaya değil (Kerckhoffs), **anahtarın gizliliğine** dayanmalıdır. Ev yapımı kriptu neredeyse her zaman kırılır. Standart, açık, denenmiş yapıları kullanın.

17. Dönem projesi: bu hafta

Bu hafta güvenlik kılavuzunuzun **veri güvenliği** bölümlerini yazacaksınız. Vize teslimi için:

- ✓ **S5 — Varlık listesi (taslak):** Projenizdeki her hassas varlığı tabloya dökün. Her varlık için: nerede durur (dosya/DB/bellek), ne zaman oluşur/sililir, **gizlilik (C)** mi **bütünlük (I)** mi (ya da **I+**) gerekir, hangi mekanizmayla korunur.
- ✓ **S7 — Veri güvenliği + güvenlik kabuğu matrisi:** Her varlık için üç durumu (aktarımda/beklemede/kullanımda) ve her durumda hangi kabuğun sarıldığını gösterin. Güvenli silme politikanızı yazın.

S5 varlık tablosu sablonu (surekli kolonlar için genişletin)

Varlık	Konum	C/I	Koruma mekanizması
Oturum anahtarı	bellek	C,I	HKDF + AES-GCM, kullanınca sil
Yerel veri anahtarı	bellek	C,I	KDF(parola,tuz) + cihaz baglama
Kart no (ornek)	DB (sifreli)	C,I	AES-256-GCM alan sifreleme

S7 guvenlik kabugu matrisi sablonu

Varlık	Asama	Ic kabuk -> ... -> Dis kabuk
Oturum anahtarı	aktarım	cihaz -> depo -> mesaj -> TLS+pin
Oturum anahtarı	depolama	cihaz -> AES-GCM
Oturum anahtarı	kullanım	kisa omur + guvenli silme

🔥 Kurs gereksinim aileleri

Bu bölümler şu gereksinim ailelerini karşılar: CEN429-DT (aktarımda veri), CEN429-DR (beklemede veri), CEN429-DU (kullanımda veri), CEN429-AS (varlık koruma), CEN429-CR (kripto ve anahtarlar). Her gereksinim için S17 uyum matrisine bir satır ekleyin: gereksinim → durum → belge bölümü → kanıt (test/kod).

18. Sınıf etkinlikleri

Etkinlik 1 — MITM'i kendi elinizle durdurun (15 dk, binary)

Demo 6'yı WSL'de çalıştırın. Sonra `tls_client.c`'de `verify` modundaki `SSL_set1_host` satırını yoruma alıp yeniden derleyin. Saldırgan sunucuya (14444) bağlanınca ne değişir? Ana makine adı denetimi olmadan **hangi** saldırı hâlâ yakalanır, hangisi kaçır? (İpucu: kendi imzalı vs. farklı ad ama geçerli zincir.)

Etkinlik 2 — Nonce'u tekrarlatın (10 dk, bireysel)

Demo 1'in kodunda `crypto_random(nonce, 12)` yerine `memset(nonce, 0, 12)` yazıp iki farklı dosyayı şifreleyin. İki şifreli metnin başındaki nonce alanları aynı mı? Şifreli metinlerin XOR'unu alıp düz metinlerin XOR'uyla karşılaştırın (Demo 2'deki gibi). Ne görüyorsunuz? Neden bu bir güvenlik açığı?

Etkinlik 3 — Üç veri durumu haritası (15 dk, 3-4 kişilik grup)

Bir "**sağlık takip uygulaması**" düşünün: kullanıcı ölçümlerini telefona kaydeder, buluta yükler, doktoru görür. Bir tablo çizin: her hassas veri (ölçüm, kimlik, doktor notu) için üç durumdaki (aktarımda/beklemede/kullanımda) tehdidi ve önlemi yazın. En az **iki** varlık için güvenlik kabuğu matrisini doldurun.

Etkinlik 4 — KDF maliyeti ayarı (10 dk, bireysel)

Demo 4'te tur sayılarına bir de 5.000.000 ekleyin. Kullanıcı girişinde kabul edilebilir en yüksek gecikme sizce ne (100 ms? 500 ms?)? O gecikmeyi veren tur sayısını bulun. Saldırganın makinesi sizinkinden 1000 kat hızlıysa bu tur sayısı onu ne kadar yavaşlatır?

19. Kod okuma alıştırmaları

Okuma 1 — Bu TLS istemcisi güvenli mi?

```
SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
SSL *ssl = SSL_new(ctx);
SSL_set_fd(ssl, sock);
if (SSL_connect(ssl) == 1)
    printf("Connection secure!\n");
```

Cevap

Hayır. `SSL_CTX_set_verify` ve `SSL_set1_host` yok; hiçbir doğrulama yapılmıyor. Bu kod her sertifikayı (saldırganınki dahil) kabul eder. "Connection secure!" mesajı yanıltıcıdır — yalnız pasif dinleyiciye karşı koruma vardır, aktif MITM'e karşı yoktur (Demo 6, Senaryo 1).

? Okuma 2 — Bu çözme neden tehlikeli?

```
EVP_DecryptUpdate(c, plain, &len, ct, ct_len);
EVP_DecryptFinal_ex(c, plain + len, &len); /* dönüş değeri okunmuyor */
use(plain);
```

✓ Cevap

EVP_DecryptFinal_ex'in dönüş değeri (GCM'de etiket doğrulaması) **kontrol edilmiyor**. Etiket tutmasa bile use(plain) çağrılıyor — kurcalanmış veri kabul edilir. Doğrulama başarısızsa çıktı **kullanılmamalı** (Demo 1).

? Okuma 3 — Bu anahtar türetme nerede zayıf?

```
unsigned char key[32];
SHA256((unsigned char*)password, strlen(password), key);
```

✓ Cevap

Tek turlu, **tuzsuz** SHA-256 bir parola KDF'si değildir: hızlıdır (kaba kuvvet ucuz), tuz yoktur (rainbow tablo çalışır), aynı parola herkeste aynı anahtarı verir. PBKDF2/scrypt/Argon2id + rastgele tuz gerekir (Demo 4).

20. Kendi başına çalış

Bu alıştırmalar not verilmez; pekiştirme içindir. Hepsi kendi bilgisayarınızda, code/week-03 demoları üzerinde yapılır.

? Alıştırma 1 — Kolay: AAD'nin gücü

Demo 1'e ilişkili veri (AAD) ekleyin: şifrelerken crypto_gcm_encrypt 'ye AAD olarak "version=1" verin, çözerken "version=2" verin. Ne olur? AAD ne işe yarar?

✓ Beklenen sonuç

Etiket tutmaz, çözme reddedilir. AAD şifrelenmez ama **doğrulandır**; şifreli gövde başka bir bağlama (farklı sürüm, farklı kayıt) taşınamaz.

? Alıştırma 2 — Kolay: Tuzu sabitleyin

Demo 4'te iki tuzu da aynı yapın. İki "farklı kullanıcı" artık aynı anahtarı mı alıyor? Bu neden kötü?

? Alıştırma 3 — Orta: CBC + rastgele IV

Demo 1'i AES-256-CBC ile yeniden yazın (kip değişecek; bütünlük için ayrıca HMAC ekleyin — "encrypt-then-MAC"). Neden CBC tek başına yetmez? GCM ile karşılaştırın: hangisi daha az hata yapmaya açık?

İpucu

CBC yalnız gizlilik verir; MAC'siz CBC'de saldırgan bitleri değiştirebilir (padding oracle). GCM tek çağrıda ikisini birden yapar.

? Alıştırma 4 — Orta: Pinning'i kırın (kendi sertifikanızla)

Demo 6'da saldırgan sertifikasını **gerçek** sunucunun anahtarıyla üretin (aynı `gercek.key`'i kullanın). Pinleyen istemci artık kabul eder mi? Bu neden pinning'in "anahtarı" pinlediğini, sertifikayı değil, gösterir?

? Alıştırma 5 — Orta: Maskeleye

Demo 7'ye bir "maskeli görünüm" ekleyin: kart numarasını çözüp yalnız son 4 hanesini gösterin (`**** * 4242`). Maskeleye ile şifrelemenin farkı ne? Hangisi loglar için uygun?

? Alıştırma 6 — Orta: Kabuğa beşinci katman

Demo 9'a bir HMAC katmanı ekleyin (her kabuğun dışına ayrı bir MAC). Toplam boyut nasıl değişir? Bir kabuğun MAC'i tutmazsa hangi katmanda durursunuz?

? Alıştırma 7 — Zor: HKDF'yi elle doğrulayın

`cen429_crypto.h`'daki HKDF'yi RFC 5869 Test Case 1 ile karşılaştırın (IKM, tuz, info değerleri RFC'de). Çıktınız RFC'nin OKM'siyle birebir aynı mı? (İpucu: `code`'daki test bunu zaten yapıyor.)

? Alıştırma 8 — Zor: Nonce sayacı taşması

Bir sistem 96 bitlik nonce'u sayaç olarak kullanıyor ve her mesajda 1 artırıyor. Kaç mesajdan sonra nonce tekrar eder? Rastgele nonce'ta çakışma olasılığı ne zaman kaygı verir (doğum günü sınırı)?

? Alıştırma 9 — Zor: Whitebox motivasyonu

Demo 8'i çalıştırırken (WSL) programı `gdb` ile durdurup belleği inceleyin ([Hafta 1](#) Demo 2'deki `gcore` yöntemi). Sır silinmeden **önce** bellekte görünüyor mu? Bu, whitebox kriptografinin hangi problemi çözmeye çalıştığını nasıl açıklar?

? Alıştırma 10 — Orta: AES-GCM mi ChaCha20-Poly1305 mi?

Bir mobil uygulama, AES donanım hızlandırması **olmayan** eski bir telefonda çalışacak. Hangi AEAD'yi seçersiniz ve neden? Aynı uygulama modern bir sunucuda çalışsaydı seçiminiz değişir miydi?

✓ Beklenen sonuç

Donanım AES'i olmayan cihazda **ChaCha20-Poly1305** genelde daha hızlı ve yan kanala dirençlidir. Modern sunucuda AES-NI olduğu için **AES-GCM** çok hızlıdır. İkisi de AEAD'dir; güvenlik açısından ikisi de kabul edilir, seçim performans/platform kaynaklıdır.

? Alıştırma 11 — Zor: Sabit zamanlı karşılaştırma

Bir MAC doğrulaması `if (memcmp(beklenen, gelen, 32) == 0)` ile yapılıyor. Bu neden bir **zamanlama** sızıntısı olabilir? Nasıl düzeltirsiniz?

🔥 İpucu

`memcmp` ilk farklı baytta durur; saldırgan cevap süresini ölçerek doğru MAC'i bayt bayt tahmin edebilir. Çözüm: `CRYPTO_memcmp` gibi **sabit zamanlı** karşılaştırma (tüm baytları her zaman karşılaştırır).

? Ek alıştırma A — Kolay: rand() ne kadar tahmin edilebilir?

`srand(time(NULL))` ile 16 baytlık bir "anahtar" üreten küçük bir program yazın ve anahtarı onaltılık yazdırın. Sonra ikinci bir program yazın: son 10 dakikanın her saniyesini tohum olarak deneyip aynı anahtarı bulsun. Kaç denemede buldu? Ardından ilk programı `crypto_random()` (`code/common/cen429_crypto.h`) ile yeniden yazın.

? Ek alıştırma B — Kolay: modulo sapmasını ölçün

0–255 arasında tekdüze baytlar üretilip `bayt % 10` ile bir milyon rakam çıkarın; her rakamın kaç kez çıktığını sayın. Sonra "Rastgele sayılar" bölümündeki `aralikta_rastgele(10)` ile tekrarlayın. Farkı tablo olarak yazın.

🔥 Beklenen sonuç

$256 = 25 \cdot 10 + 6$ olduğu için 0–5 rakamları 6–9'dan yaklaşık %4 daha sık çıkar (26/256'ya karşı 25/256). Reddetme yöntemiyle bütün rakamlar istatistiksel olarak eşit çıkar.

? Ek alıştırma C — Orta: AEAD'de Final'ı unutmak

"Kripto API'lerini doğru kullanmak" bölümündeki `aes_gcm_coz` fonksiyonundan `EVP_DecryptFinal_ex` denetimini kaldırın (yalnız `EVP_DecryptUpdate`'in yazdığı tamponu kullanın). Şifreli metnin bir baytını değiştirip çözün. Program ne döndürüyor? Sonra denetimi geri koyup aynı testi yapın. Bu farkı bir cümleyle bir değerlendirme raporuna yazılacak bulgu olarak ifade edin.

? Ek alıştırma D — Orta: zarflama ile anahtar yenileme

Beş küçük dosyayı her biri kendi rastgele DEK'ile AES-GCM ile şifreleyin; DEK'leri bir KEK ile sarıp dosya başlığına anahtar sürümüyle birlikte yazın. Sonra KEK'i yenileyin: yalnız sarılmış DEK'leri yeniden sarın. Dosya içeriklerinin (şifreli metinlerin) hiç değişmediğini özetleriyle gösterin.

? Ek alıştırma E — Orta: fail-open avı

"Eleştirel okuma: sessizce açılan kapı" bölümündeki deseni kendi yazdığınız ya da daha önce yazdığınız bir projede arayın: bir güvenlik denetiminin `catch` bloğunda ya da "bulunamadı" dalında işleme devam eden bir yer var mı? Her bulgu için "hangi girdiyle tetiklenir, sonuç ne olur, nasıl kapalı kalarak başarısız olur" sorularını cevaplayın.

? Ek alıştırma F — Orta: günlükte kart numarası

Bir günlük dosyasında 13–19 haneli, Luhn denetimini geçen sayı dizilerini bulup son dört hanesi dışında maskeleyen bir süzgeç yazın. Test için yalnız **sentetik** numaralar kullanın (ör. Luhn'u geçen ama gerçek bir karta ait olmayan, kendi ürettiğiniz diziler).

? Ek alıştırma G — Zor: kendi projeniz için kabuk matrisi

Projenizin en hassas iki varlığı için "Sahadaki kabuk matrisi" biçiminde bir tablo doldurun: aşamalar (aktarım, saklama, bellekte işlem) × kabuklar. Her hücrede hangi algoritmanın, hangi anahtarın ve hangi bağlamanın kullanıldığını yazın; boş kalan hücreler için gerekçe verin.

21. Kendini sına

? 1. Verinin üç hâli nedir? Her biri için bir tehdit ve bir önlem söyleyin.

Aktarımda (dinleme/MITM → TLS+pinning), beklemede (dosya çalınması → AEAD alan şifreleme), kullanımda (bellek dökümü → güvenli silme + kısa ömür).

? 2. AEAD nedir? AES-CTR'den farkı?

AEAD gizlilik **ve** bütünlük+kimliği birlikte sağlar (şifreli metin + doğrulanan etiket). AES-CTR yalnız gizlilik verir; MAC eklenmezse kurcalama fark edilmez.

? 3. Nonce yeniden kullanımı GCM/CTR'de neden felakettir?

Aynı anahtar akışı iki mesajda kullanılır; $C1 \oplus C2 = P1 \oplus P2$ olur, anahtar akışı sızar. Bir düz metin bilinirse diğeri çözülür (GCM'de kimlik doğrulama da çöker).

? 4. ECB neden kullanılmaz?

Her bloğu bağımsız şifreler; aynı düz metin bloğu aynı şifreli bloğu verir, desenler sızar (ECB pengueni).

? 5. Tuz ile nonce arasındaki fark nedir? İkisi de gizli mi?

Tuz paroladan anahtar türetmede rainbow tabloyu engeller; nonce şifreleme kipinde anahtar akışını benzersizleştirir. İkisi de **gizli değildir**; tuz rastgele+benzersiz, nonce anahtar başına tekrarsız olmalıdır.

? 6. PBKDF2'de tur sayısı neden yüksek olmalı? Bugünkü öneri?

Her parola denemesini pahalılaştırır. OWASP 2023: PBKDF2-HMAC-SHA256 için ≥ 600.000 tur; mümkünse Argon2id.

? 7. HKDF'nin iki aşaması nedir? 'info' ne işe yarar?

Extract (ana sır+tuz $\rightarrow$ PRK) ve Expand (PRK+info $\rightarrow$ anahtar). **info** amaç/oturum ayrımı sağlar; farklı info farklı anahtar verir.

? 8. İleri gizlilik (forward secrecy) nedir?

Bugünkü anahtar sızsa bile eski oturumların çözülememesi. Tek yönlü zincir/geçici DH ile sağlanır; TLS 1.3'te varsayılan.

? 9. TLS 1.3 el sıkışmasında sunucunun kimliği nasıl doğrulanır?

Sunucu sertifikasını sunar ve özel anahtarıyla el sıkışmayı imzalar (CertificateVerify). İstemci zinciri, süreyi ve **ana makine adını** doğrular (gerekirse pin).

? 10. OpenSSL istemcisinde en tehlikeli varsayılan nedir?

Sertifikayı **doğrulamamak**. `SSL_VERIFY_PEER` ve `SSL_set1_host` açıkça ayarlanmazsa MITM mümkündür.

? 11. Sertifika sabitleme (pinning) neyi, hostname doğrulama neyi yakalar?

Hostname, yanlış ad taşıyan sertifikayı yakalar; pinning, geçerli zincirli ama **beklenmeyen anahtarlı** (mis-issued) sertifikayı yakalar.

? 12. TOFU nedir ve sınırı nedir?

İlk bağlantıdaki anahtarı hatırlayıp sonra değişirse uyarmak (SSH modeli). İlk bağlantı korumasızdır; PKI kadar güçlü değildir.

? 13. Alan şifreleme ile veri maskeleye farkı?

Alan şifreleme geri döndürülebilir (anahtarla açılır), gizlilik için. Maskeleye genelde geri döndürülemez/kısımdır, gereksiz maruz kalmayı azaltmak için (log, ekran).

? 14. Kullanımda veriyi korumanın üç katmanı?

Döküm engelleme (RLIMIT_CORE/SetErrorMode), bellek kilitleme (mlock/VirtualLock), güvenli silme (OPENSSL_cleanse/SecureZeroMemory) + kısa ömür.

? 15. Güvenlik kabuğu nedir? Bir örnek verin.

Bir sırrı iç içe bağımsız katmanlarla sarmak. Örnek: cihaz bağlama → depolama AES-GCM → mesaj AEAD → TLS+pinning (Demo 9). Saldırgan hepsini kırmalı.

? 16. Cihaz bağlama nasıl doğru yapılır?

Cihaz parmak izinden **HKDF ile tuzlanmış** türetme ve **AEAD ile sarma**; tuzsuz özet ya da XOR değil.

? 17. Whitebox kriptografi hangi problemi çözmeye çalışır?

Kullanım anında anahtarın bellekte açık durması problemini: anahtarı tablolara gömerek ham hâlini hiç göstermemek (beyaz kutu saldırılarına karşı).

? 18. 'Yalnız TLS'e güvenmeme' ne demektir?

Uç noktalar güvenilir olabildiği için TLS'in içine bir kat daha (mesaj düzeyi AEAD+MAC) koymak; TLS aşılsa bile gövde şifreli kalır.

? 19. GCM etiketi kaç bayttır ve neyin fonksiyonudur?

16 bayt; şifreli metnin, AAD'nin ve anahtarın fonksiyonudur. Bu yüzden saldırgan anahtarsız doğru etiket üretemez.

? 20. Aynı düz metni AES-GCM ile iki kez şifrelerseniz şifreli metinler aynı mı olur?

Hayır — nonce her seferinde farklı olduğu için şifreli metin ve etiket de farklı olur. Aynı çıkması nonce tekrarına işaretir (tehlike).

? Ek soru 1. `rand()` neden güvenlikle ilgili değerler için kullanılamaz? Yerine ne kullanılır?

İstatistiksel bir üreticidir; tohum ya da birkaç çıktı bilinirse bütün dizi hesaplanır. Yerine işletim sisteminin CSPRNG'si: Linux `getrandom`, Windows `BCryptGenRandom`, OpenSSL `RAND_bytes`.

? Ek soru 2. `EVP_DecryptUpdate` açık metni yazdı ama `EVP_DecryptFinal_ex` başarısız döndü. Ne yapmalısınız?

Açık metni kullanmadan silmeli ve işlemi reddetmelisiniz; etiket tutmamıştır, veri kurtulanmış olabilir.

? Ek soru 3. MAC karşılaştırmasında `memcmp` neden sakıncalıdır?

İlk farklı baytta durur; süre, kaç baytın doğru olduğunu sızdırır (zamanlama saldırısı). Sabit zamanlı karşılaştırma (`CRYPTO_memcmp`) kullanılır.

? Ek soru 4. Kripto-periyot nedir? Süresi dolan bir anahtar hemen silinir mi?

Anahtarın kullanılmasına izin verilen süredir. Süre dolunca yeni veriyi şifrelemekte kullanılmaz ama eski veri yeniden şifrelenene kadar çözme için tutulabilir; sonra imha edilir.

? Ek soru 5. Zarflamada KEK yenilendiğinde neden bütün veri yeniden şifrelenmez?

Veri DEK'lerle şifrelidir; KEK yalnız DEK'leri sarar. KEK değişince yalnız küçük sarılmış DEK'ler yeniden sarılır.

? Ek soru 6. OpenSSL istemcisinde `SSL_set1_host` çağrılmazsa ne olur?

Zincir doğrulaması yapılsa bile sertifikadaki ad denetlenmez; saldırganın kendi alan adı için aldığı geçerli bir sertifika kabul edilir.

? Ek soru 7. Sabitlemede neden sertifika yerine SPKI özeti ve neden bir yedek pin kullanılır?

SPKI özeti, sertifika aynı anahtarla yenilendiğinde değişmez. Yedek pin, birincil anahtar kaybolduğunda ya da ele geçtiğinde uygulamanın bağlanamaz hale gelmesini (tuğlalaşmasını) önler.

? Ek soru 8. 'Fail-open' ne demektir? Bir örnek verin.

Bir güvenlik denetiminin hata durumunda "geçti" sonucuna düşmesidir. Örnek: sabitleme kodunda anahtar deposu hatasının yakalanıp yalnız yazdırılması ve bağlantının kabul edilmesi.

? Ek soru 9. Takma adlandırmada kimlik numarasının düz SHA-256 özeti neden yetmez?

Olası değer sayısı küçüktür; saldırgan bütün kimlik numaralarının özetini hesaplayıp eşleştirebilir. Gizli anahtarlı HMAC kullanılır ve anahtar veri kümesinden ayrı tutulur.

? Ek soru 10. Tokenizasyon bir uygulamanın PCI DSS kapsamını neden küçültür?

Uygulama, veritabanları ve günlükler gerçek kart numarası yerine anlamsız belirteç tutar; gerçek veri yalnız küçük ve iyi korunan belirteç kasasında durur.

? Ek soru 11. Sahadaki kabuk matrisinde, ödeme sırasında native tarafta neden yalnız bir kabuk kalır ve bu yeterli midir?

Anahtarın hesaplamada kullanılabilmesi için dış kabuklar açılmıştır; kalan kabuk whitebox biçimindedir, yani anahtar yine açık değildir. Tek başına yeterli değildir: o andaki koruma RASP ve kod sağlamlaştırmayla tamamlanır.

22. Quiz-1 tarzı örnek sorular



Quiz-1 (8. hafta) bu tarz kısa soru + kod okuma içerir

1. (Çoktan seçmeli) Aşağıdakilerden hangisi **hem** gizlilik **hem** bütünlük sağlar? a) AES-CTR · b) SHA-256 · c) **AES-GCM** · d) AES-ECB
2. (Doğru/Yanlış) "Nonce gizli tutulmalıdır." → **Yanlış** (benzersiz olmalı, gizli değil).
3. (Kısa cevap) Bir istemci TLS'te sertifikayı doğruluyor ama `SSL_set1_host` çağırıyor. Hangi saldırgan hâlâ başarılı olur? → Geçerli zincirli ama **başka bir ad** için verilmiş sertifikaya sahip saldırgan.
4. (Kod okuma) `EVP_DecryptFinal_ex` 'in dönüşü kontrol edilmeyen bir GCM çözme kodunda hangi güvenlik özelliği kaybolur? → **Bütünlük/kimlik doğrulama** (kurcalanmış veri kabul edilir).
5. (Eşleştirme) PBKDF2 ↔ CPU maliyeti · Argon2id ↔ CPU+bellek · HKDF ↔ oturum anahtarı türetme · pinning ↔ SPKI.

23. Kaynaklar ve ileri okuma

Ders kitabı — J. Viega, M. Messier, *Secure Programming Cookbook for C and C++*, O'Reilly, 2003:

- Tarif 4.9 (tuz/nonce/IV), 4.10 (paroladan anahtar — PBKDF2), 4.11 (ana sırdan anahtar), 4.13 (anahtar malzemesi yönetimi)
- Tarif 5.15 (dosya/disk şifreleme), 6.18 (şifreleme + bütünlük birlikte)
- Tarif 8.19 (PKI'siz doğrulama / TOFU), 8.20–8.21 (ileri gizlilik)
- Tarif 9.1–9.3 (SSL istemci/sunucu), 10.7–10.9 (sertifika doğrulama, ana makine adı, beyaz liste = pinning)
- Tarif 13.2–13.3 (bellekten güvenli silme, belleğin diske yazılmasını önleme)
- Tarif 11.1–11.4 (rastgele sayı türleri, Unix ve Windows altyapısı), 11.11 (aralıkta sapmasız rastgele tamsayı)
- Tarif 9.4 (Windows'ta WinInet ile güvenli iletişim — bugün WinHTTP/Schannel)

Açık standartlar ve kaynaklar

- NIST SP 800-57 Part 1 (anahtar yönetimi, kripto-periyot), SP 800-90A (rastgele bit üreteçleri), SP 800-38G (FF1, biçimi koruyan şifreleme), SP 800-38C (CCM).
- PCI DSS v4.0 (kart numarasının gösterimde maskelenmesi, tokenizasyon); KVKK ve GDPR'da takma adlandırma ve anonimleştirme.
- M. Georgiev ve diğ., "The Most Dangerous Code in the World: Validating SSL Certificates in Non-Browser Software", ACM CCS 2012.
- MITRE CWE: CWE-330 (yetersiz rastgelelik), CWE-338 (zayıf PRNG), CWE-208 (zamanlama farkıyla bilgi sızması), CWE-297 (ana makine adı denetimi eksik), CWE-636 (fail-open: güvenli olmayan başarısızlık), CWE-359 (kişisel veri ifşası), CWE-532 (günlüğe hassas veri yazma).
- NIST SP 800-38D (AES-GCM), SP 800-132 (PBKDF2), SP 800-108 / RFC 5869 (HKDF), RFC 8439 (ChaCha20-Poly1305), RFC 9106 (Argon2), RFC 7914 (scrypt), RFC 8446 (TLS 1.3).
- OWASP: Password Storage Cheat Sheet, Transport Layer Security Cheat Sheet, Certificate and Public Key Pinning, MASVS-CRYPTO / MASVS-NETWORK.

- MITRE CWE: CWE-295 (yanlış sertifika doğrulama), CWE-323 (nonce/IV yeniden kullanımı), CWE-327 (zayıf kriptoloji), CWE-916 (yetersiz maliyetli parola özetleri), CWE-311 (şifrelemenin eksikliği).

**Sözlük**

Terim	Türkçe	Kısa tanım
AEAD	Kimlik doğrulamalı şifreleme	Gizlilik + bütünlük + kimliği birlikte veren şifreleme (AES-GCM)
Nonce	Bir kez kullanılan sayı	Anahtar başına tekrar etmemesi gereken, gizli olmayan değer
IV	Başlangıç vektörü	Kipe rastgelelik veren, gizli olmayan başlangıç değeri
Salt / Tuz	Tuz	Paroladan anahtar türetmede rainbow tabloyu engelleyen rastgele değer
KDF	Anahtar türetme fonksiyonu	Paroladan/sırdan anahtar üreten fonksiyon (PBKDF2, HKDF, Argon2id)
MAC	Mesaj kimlik doğrulama kodu	Paylaşılan anahtarla bütünlük+kimlik kanıtı (HMAC)
Forward secrecy	İleri gizlilik	Bugünkü anahtar sızsa bile eski oturumların güvende kalması
Pinning	Sabitlenme	Beklenen belirli anahtarı/sertifikayı zorunlu kılma (SPKI)
TOFU	İlk kullanımda güven	İlk görülen anahtarı hatırlayıp değişince uyarma (SSH modeli)
Data at rest / in transit / in use	Beklemede / aktarımda / kullanımda veri	Verinin üç hâli
Security shell	Güvenlik kabuğu	Bir sırrı iç içe bağımsız koruma katmanlarıyla sarma
Whitebox crypto	Beyaz kutu kriptografi	Anahtarı tablolara gömüp bellekte açık göstermeyen gerçekleştirim
Device binding	Cihaz bağlama	Anahtarı belirli cihaza/sürüme bağlayıp başka yerde açılmaz kılma
Hybrid encryption	Hibrit şifreleme	Asimetrik ile simetrik anahtar taşıyıp veriyi simetrik şifreleme (TLS)
Encrypt-then-MAC	Önce şifrele, sonra MAC	Şifreli metnin üzerine MAC koyma; AEAD'nin içerideki doğru sırası
Tokenization	Tokenizasyon	Gerçek değeri anlamsız bir token ile değiştirip kasada saklama
Constant-time compare	Sabit zamanlı karşılaştırma	Sırları/MAC'leri zamanlama sızdırmadan karşılaştırma (CRYPTO_memcmp)

Terim	Türkçe	Kısa tanım
AAD	İlişkili veri	AEAD'de şifrelenmeyen ama doğrulanan (bağlanan) ek veri

Bir sonraki hafta

4. hafta — Kod Sağlamaştırma: C/C++. Bu hafta veriyi (anahtar, açık metni) doğru şifreleme ve anahtar yönetimiyle korumayı öğrendik; ama şifreleme kodunun kendisi de bir C/C++ programıdır ve bir arabellek taşması, biçim dizesi (format string) açığı ya da tamsayı hatası, bu hafta özenle koruduğumuz anahtar ve açık metni belleğe sızdırabilir. 4. haftada SEI CERT C/C++ kurallarıyla tam olarak bu tür hataları önlemeyi, statik analiz ve sanitizier'larla yakalamayı işleyeceğiz.